VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

ASSIGNMENT REPORT

Software Engineering Submission #3

Smart Parking System at HCMUT - VNU-HCM

Instructor: Mr. Phan Trung Hiếu
Class: CC02 - **Group:** 02

Student's Name	Student ID
Nguyễn Tiến Hưng	2352441
Nguyễn Bùi Tuấn Anh	2352038
Phạm Lê Minh Khôi	2352622
Đặng Nguyên Quân	2352985
Nguyễn Từ Nhật Khang	2352492
Huỳnh Vỹ Quốc	2353025
Nguyễn Tuấn Kiệt	2352652
Lê Chương Quyền	2353034

Ho Chi Minh City, December 2025.

Member list & Workload

Group Leader: Nguyễn Từ Nhật Khang - khang.nguyentusteven@hcmut.edu.vn

No.	Full name	Contribution	Percentage
1.	Nguyễn Tiến Hưng	- Project details specification - Functional, Non-functional requirements - UI Design - Deployment View	100%
2.	Nguyễn Bùi Tuấn Anh	- Project details specification - Functional, Non-functional requirements - Sequence diagrams - Class diagrams	100%
3.	Phạm Lê Minh Khôi	- Project details specification - Functional, Non-functional requirements - Sequence diagrams - Component diagram	100%
4.	Đặng Nguyên Quân	- Project details specification - Functional, Non-functional requirements - Activity diagrams - Class diagrams	100%
5.	Nguyễn Từ Nhật Khang	- Project details specification - Functional, Non-functional requirements - State-chart diagrams - Class diagrams	100%
6.	Huỳnh Vỹ Quốc	- Project details specification - Functional, Non-functional requirements - Activity diagrams - Component diagram	100%
7.	Nguyễn Tuấn Kiệt	- Project details specification - Functional, Non-functional requirements - UI Design - Testcases	100%
8.	Lê Chương Quyền	- Project details specification - Functional, Non-functional requirements - UI Design - Class diagrams	100%

Lecture's Assessment

No.	Full name	Evaluation	Notes
1.	Nguyễn Tiến Hưng		
2.	Nguyễn Bùi Tuấn Anh		
3.	Phạm Lê Minh Khôi		
4.	Đặng Nguyên Quân		
5.	Nguyễn Từ Nhật Khang		
6.	Huỳnh Vỹ Quốc		
7.	Nguyễn Tuấn Kiệt		
8.	Lê Chương Quyền		

Contents

List of Figures

1 Project Details Specification

1.1 Project Context

Ho Chi Minh City University of Technology (HCMUT) accommodates a large volume of daily parking activities involving undergraduate students, graduate students, doctoral candidates, faculty members, staff, and external visitors. Due to increasing vehicle density and limited parking capacity, current parking operations face challenges such as congestion at entry and exit points, inefficient space utilization, limited real-time visibility of parking availability, and fragmented parking fee management.

To address these issues, the university proposes the development of an IoT-based Smart Parking Management System (IoT-SPMS1). The system aims to automate access control, monitor parking occupancy in near real time, support dynamic guidance within the campus, and provide integrated billing mechanisms. It must operate reliably under real-world constraints including high concurrency during peak hours, intermittent connectivity, and heterogeneous user groups.

University members access parking facilities using official identification cards integrated with campus infrastructure, enabling automatic recording of entry and exit activities. Parking fees are computed based on accumulated usage and user roles, with payment processing integrated through BKPay. The system must also support temporary visitors and exceptional cases via a temporary access mechanism at entry gates.

From an infrastructure perspective, the system integrates IoT sensors, local gateways, centralized services, HCMUT_SSO for authentication, and HCMUT_DATACORE for read-only synchronization of user and role information. Role-based access control and comprehensive logging are required to ensure security, traceability, and operational transparency.

1.2 Relevant Stakeholders

The system involves the following key stakeholders:

Stakeholder	Brief Description	Key Expectations
University Management	Strategic owners focusing on macro-efficiency. Introduces university management policies, including parking lot management.	High-level ROI reports and optimized space utilization.
Parking Management	Operational oversight and incident response.	Real-time monitoring and 100% automated visitor handling.
Finance Office	Financial controllers and pricing strategists.	Accurate fee calculation and zero-leakage payment auditing.
IT Management	Infrastructure and database maintainers.	99.9% system uptime and secure data integration (SSO/API).
System Admin	Technical configuration and security auditor.	Granular control over user roles and detailed audit trails.
Learners	Internal users with registered profiles.	Fast entry via card-swipe and seamless balance/subscription use.
Faculty / Staff	Internal users with registered profiles. The only group that can access privilege parking lot.	Fast entry via card-swipe and seamless balance/subscription use.
Temporary Visitor	External users with no pre-existing profile.	Intuitive automated kiosk for card issuance and flexible payment.
BKPay (Payment API)	External financial processing service.	Secure, PCI-compliant transactions and low-latency authorization.
HCMUT_SSO (University Identity API)	Source of truth for user data and balances.	Real-time validation of roles and instant synchronization of card balances.
HCMUT_DATACORE (University Database API)	Backend service interface for persistent data storage and retrieval.	High availability, ACID-compliant transactions, secure authentication, and low-latency query response.
IoT Gateway / Sensors	Hardware interface for physical lot control.	Accurate ALPR plate capture and immediate barrier response times.

Table 1: Stakeholders of the Smart Parking Management System

1.3 Project Objectives

The project aims to develop the **IoT-based Smart Parking Management System (IoT-SPMS1)** with the following main objectives:

- **Parking Information Management:** Allow management of parking sessions, vehicle information, user roles, and pricing policies across different parking zones. Registered users (learners, faculty, staff) and visitors have their own respective parking entries but use the same exit.
- **Process Automation:** Automate entry and exit control using ID card authentication and IoT sensors, enabling automatic recording of parking sessions and fee calculation.
- **Real-time Monitoring and Guidance:** Provide near real-time visibility of parking slot availability and support dynamic electronic signage for campus traffic guidance.
- **Integrated Billing and Payment Support:** Automatically compute parking fees based on user category and parking duration, and initiate payment processing through BKPay.
- **Administrative Control and Reporting:** Provide dashboards and reporting tools for Parking Management Office, Finance Office, and University Management to monitor operations and revenue.
- **System Integration:** Ensure seamless integration with HCMUT_SSO for authentication, HCMUT_DATACORE for user data synchronization, BKPay for payment processing, and IoT gateways for sensor data collection.
- **Dedicated Session and Visitor Data Management:** Maintain a separate operational database responsible for storing temporary visitor card information, active parking session data, and detailed user activity logs. This database records all interactions within the system, including card authentication events, entry and exit gate operations, parking session lifecycle, and payment transactions, ensuring reliable temporary data storage, traceability, and support for auditing and system monitoring.

1.4 Project Scope

1.4.1 In-Scope (Minimum Viable Product)

- **User and Role Management:**
  - Synchronize user data from HCMUT_DATACORE (read-only).
  - Enforce role-based access control for learners, faculty, staff, operators, and administrators.
- **Parking Session Management:**
  - Automatic entry and exit recording via ID card scanning.
  - Temporary access mechanism for visitors and exceptional cases.
  - Active parking session tracking and storage.
- **Operational Database and Activity Logging:**
  - Maintain a dedicated operational database for storing temporary visitor card information.
  - Store and manage active and historical parking session data.
  - Record detailed system activity logs including card authentication, gate operations, session lifecycle events, and payment transactions.
  - Maintain timestamps, user identifiers, event types, and system response status for audit and troubleshooting purposes.
- **IoT-based Occupancy Monitoring:**
  - Collect occupancy data from parking slot sensors.
  - Aggregate data via IoT gateways.
  - Maintain near real-time parking availability status.
- **Fee Calculation and Billing:**
  - Calculate parking fees based on duration and user role.
  - Support configurable pricing policies.
  - Trigger payment requests via BKPay.
- **Dashboard and Reporting:**
  - Operational dashboards for Parking Management Office.
  - Financial reports for Finance Office.
  - Summary reports for University Management.
- **System Integration:**
  - Integration with HCMUT_SSO (authentication).
  - Integration with HCMUT_DATACORE (user synchronization).
  - Integration with BKPay (payment processing).
  - Integration with IoT gateways and electronic signage systems.

1.4.2 Out-of-Scope

- Advanced AI-based parking prediction and optimization features.
- Dynamic demand-based pricing models.
- Campus-wide traffic management beyond designated parking zones.
- Physical hardware maintenance of sensors, barriers, and signage.
- External bank settlement processes beyond BKPay integration.
- Development of independent identity management systems.

2 Functional requirements

2.1 Whole system use-case diagram

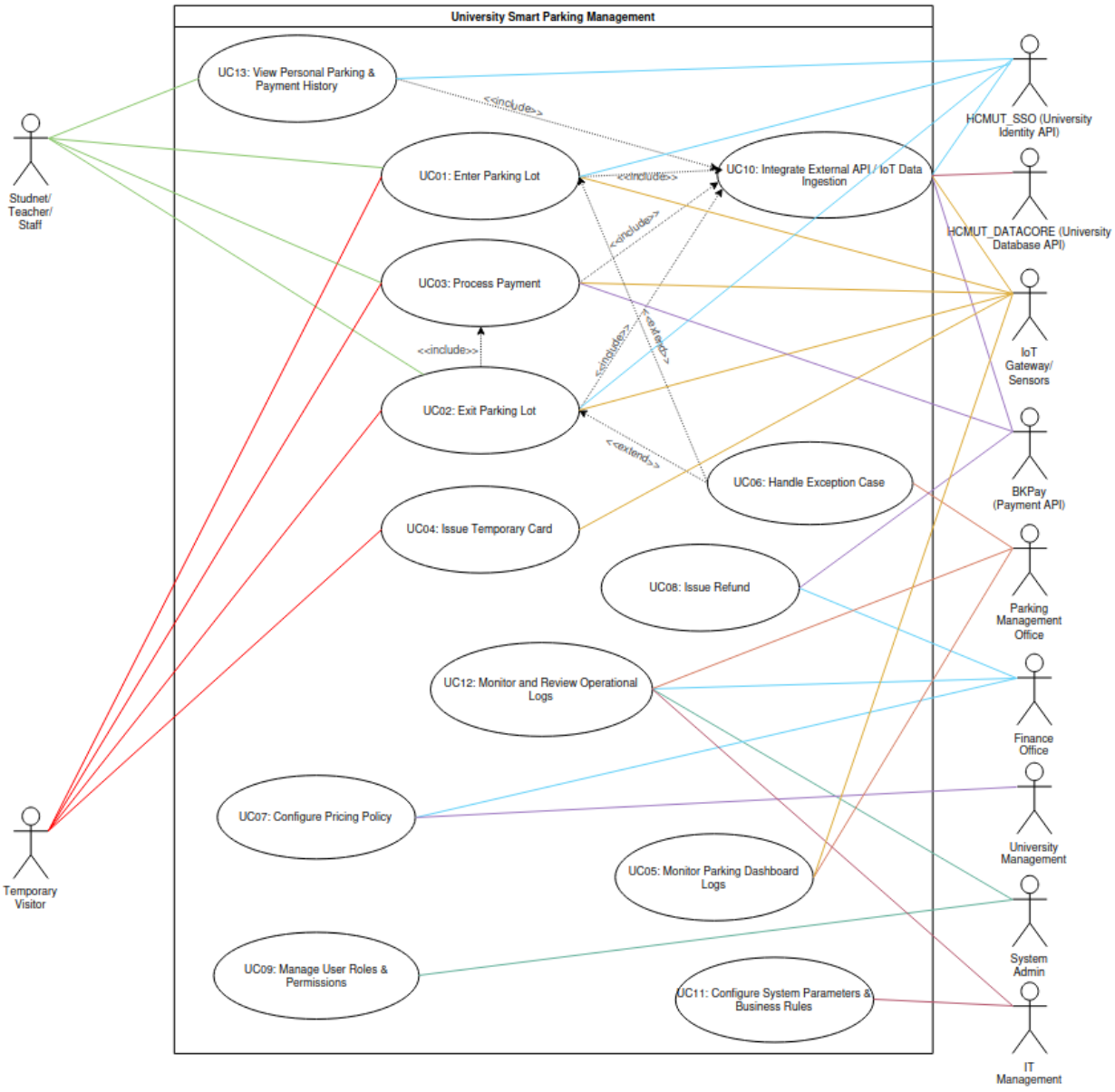

Figure 2.1: Use-case diagram for the whole system

2.2 Detailed use-case diagrams

2.2.1 UC-01: Enter Parking lot

a. Diagram

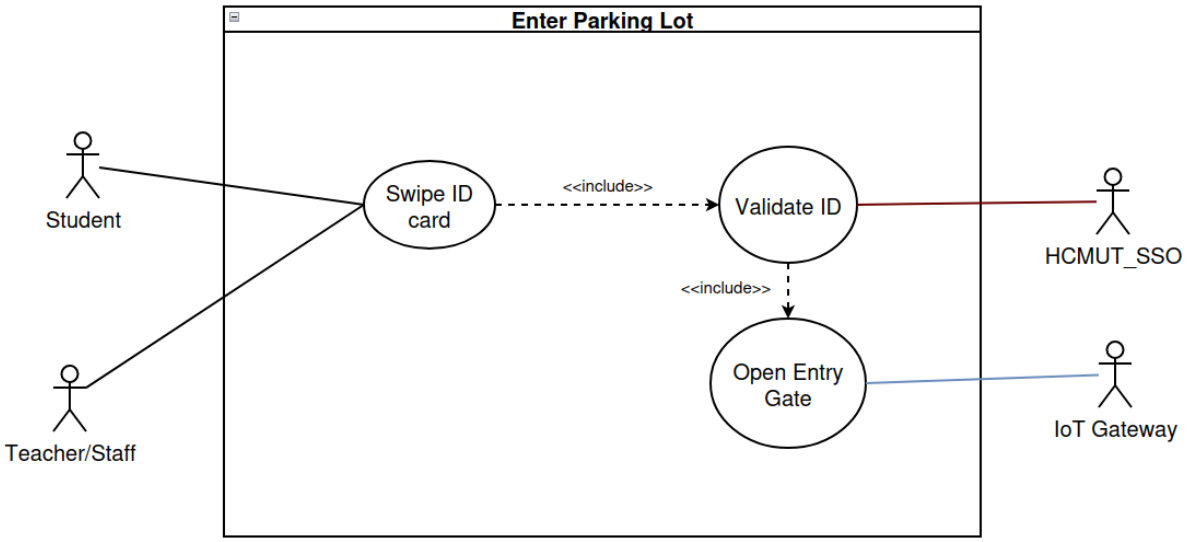

Figure 2.2: UC-01: Enter Parking Lot

b. Description

Name	Enter Parking Lot
ID	UC-01
Actors	Students, Staffs, Teachers, Visitors, IoT Gateway
Description	This use case describes the process of a user (students, staffs, teachers, or visitors) entering the campus parking facility. Staffs and teachers are granted access to a privileged parking lot, which is restricted to students and visitors.
Trigger	The driver arrives at the entry gate with an ID card.
Preconditions	- The system is operational, and the local IoT gateway is communicating with the central system. - The user possesses a valid ID card.
Postconditions	- A new parking session is successfully logged in the system. - The vehicle is inside the appropriate parking lot (general or privileged). - The system decrements the available parking slot count of the corresponding parking area.

Table 2: Enter Parking Lot

Normal Flow	1. The driver arrives at the entry gate and presents the ID card to the Card Reader. 2. The system captures the ID card data. 3. The system validates the user's ID and determines the user role (student, staff, teacher, visitor). 4. The system checks parking privileges: - If the user is staff or teacher, access to the privileged parking lot is granted. - If the user is student or visitor, access is limited to the general parking lot. 5. The system verifies slot availability in the corresponding parking area. 6. The system creates a new parking session with timestamp and entry location. 7. The system signals the Entry Gate to open. 8. The user drives into the parking facility. 9. The Entry Gate closes after the vehicle passes.
Exceptions	**Step 3a (Invalid ID card):** - The system fails to validate the ID. - The system displays an error message and denies entry. **Step 5e (Parking Area Full)** - The system indicates that the corresponding parking area (general or privileged) is at maximum capacity. - The system notifies the user and denies entry.
Notes	The privileged parking lot is strictly reserved for staff and teachers. Students and visitors are not permitted to access this area under any circumstances.

Table 2 – Continue from previous page

2.2.2 UC-02: Exit Parking Lot

a. Diagram

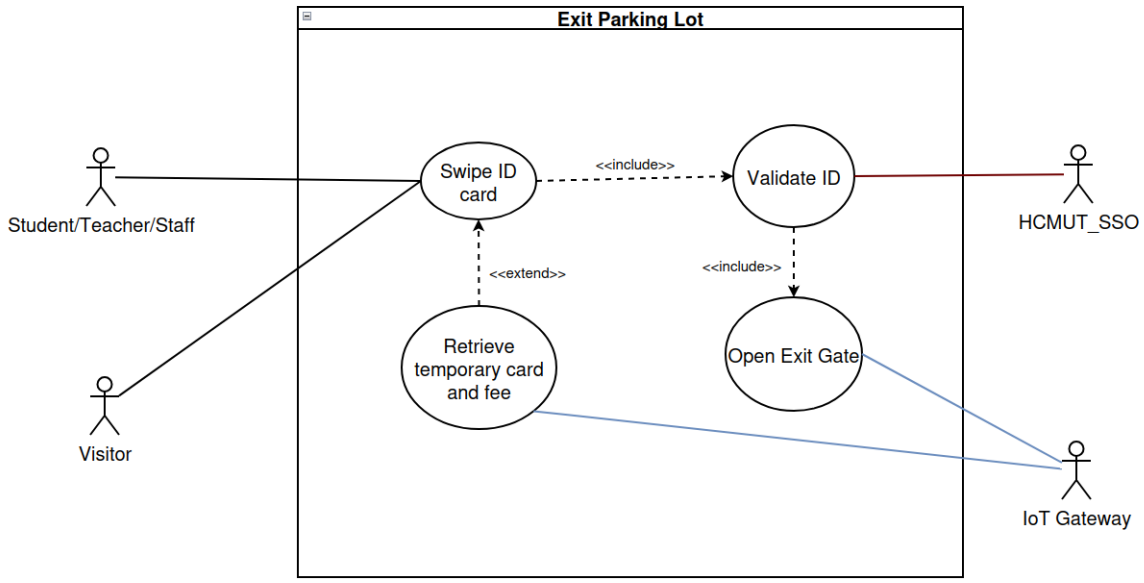

Figure 2.3: UC-02: Exit Parking Lot

b. Description

Name	Exit Parking Lot
ID	UC-02
Actors	Students, staffs, visitors, IoT Gateway
Description	This use case describes the process of a user (students, staffs, or visitors) entering the campus parking facility
Trigger	The driver arrives at the exit gate with an ID card.
Preconditions	- The system is operational, and the local IoT gateway is communicating with the central system.
Postconditions	- End the parking session successfully - logged out the system. - The vehicle is outside the parking lot. - The system increase the available parking slot count.
Normal Flow	1. The driver arrives at the exit gate and presents the ID card to the Card Reader. 2. System capture the ID Card data 3. The system successfully validates the user's ID and privileges. 4. The system creates end the parking session with timestamp. 5. The system signals the Exit Gate to open. 6. The user get out. 7. The Exit Gate closes after the vehicle passes.
Alternative Flow	**Step 3a (Visitor Exit):** - The system validate the visitor's temporary card - The system retrieve the temporary card and process a external payment
Exceptions	**Step 3e (Invalid ID card):** - The system fails to validate the ID - Displays an error message.

Table 3: Exit Parking Lot

2.2.3 UC-03: Process Payment

a. Diagram

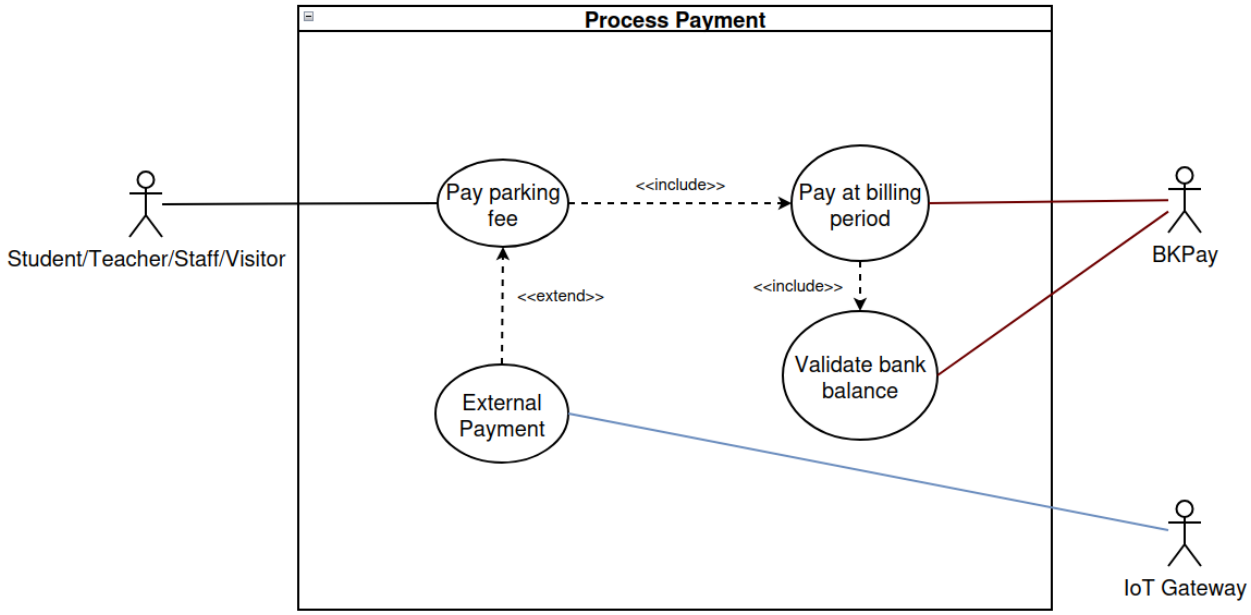

Figure 2.4: UC-03: Process Payment

b. Description

Name	Process Payment
ID	UC-03
Actors	Students, staffs, visitors, IoT Gateway, BKPay
Description	This use case describes the process of a user (students, staffs, or visitors) paying the parking fee
Trigger	The driver arrives at the exit gate, or after the billing period
Preconditions	- The system is operational, and the local IoT gateway and BKPay is communicating with the central system.
Postconditions	- The payment is done successfully
Normal Flow	1. The user want to pay the parking fee at the billing period. 2. The system compute the total parking fee throughout the period. 3. The system validate the user bank account balance 4. The user confirm and process payment.
Alternative Flow	**Step 1a (External Payment):** - The user want to pay the fee right after exiting the parking lot - User pay cash into the cash register.
Exceptions	**Step 3e (Insufficient Bank account balance)** - The system notify user, then record a debt.

Table 4: Process Payment

2.2.4 UC-04: Issue Temporary Card

a. Diagram

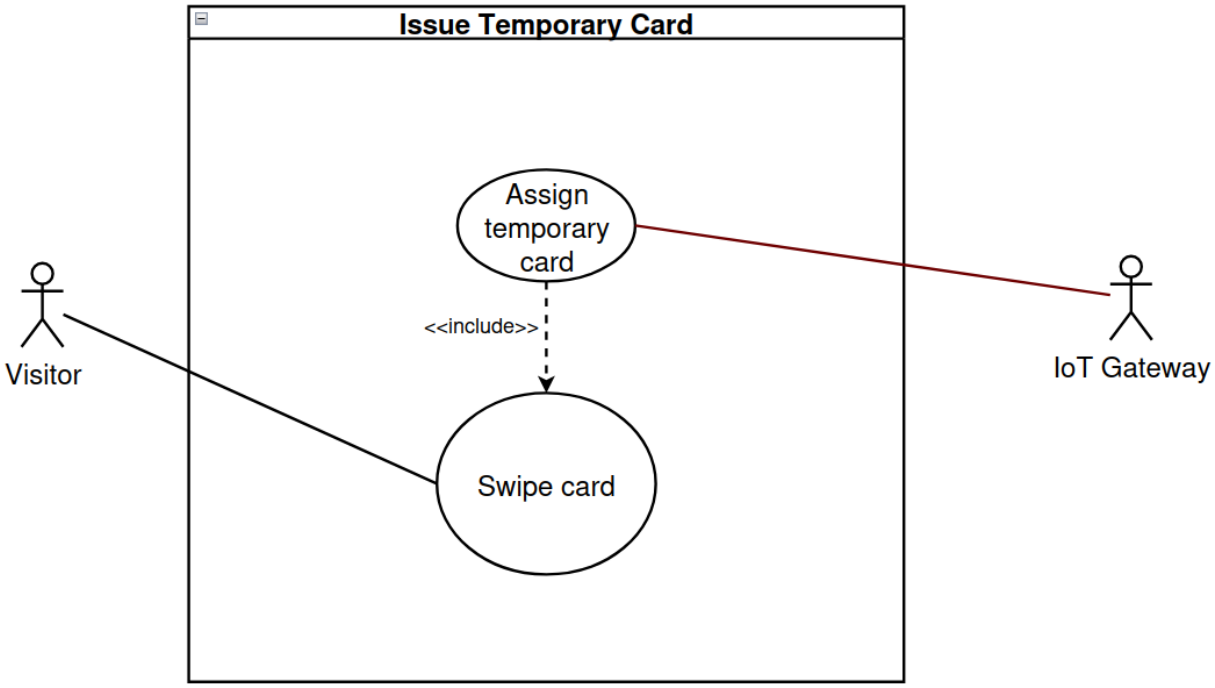

Figure 2.5: UC-04: Handle Exception Case

b. Description

Name	Issue Temporary Card
ID	UC-04
Actors	Visitors, IoT Gateway
Description	An automated process where the parking system identifies a non-registered visitor arriving through the visitor gate and provide a temporary access card to enable entry, exit.
Trigger	The visitor arrives at the visitor entry gate
Preconditions	- The Kiosk is stocked with physical visitor cards; IoT sensors and the card dispenser mechanism are online.
Postconditions	- A temporary visitor profile is created in the database
Normal Flow	1. A vehicle triggers the sensor at the gate. 2. The system automatically capture the license plate. 3. The system checks the system database, finding no match, it creates a "Temporary Visitor" profile. 4. The system selects the next available temporary card ID from the dispenser's internal inventory. 5. The system writes the entry timestamp to the card (or database record) and dispenses the physical card to the visitor. 6. The visitor swipe the card for entrance.
Alternative Flow	**Step 4a (Dispenser Empty):** - If the kiosk runs out of cards, the system automatically notify the Parking Management to insert more cards. - The system dispenses the card for user.
Exceptions	**Step 1e (Parking Lot Full)** - The system indicates that the parking lot is at maximum capacity, notify user and no temporary card is dispensed. The user cannot entry.

Table 5: Issue Temporary Card

2.2.5 UC-05: Monitor Parking Dashboard

a. Diagram

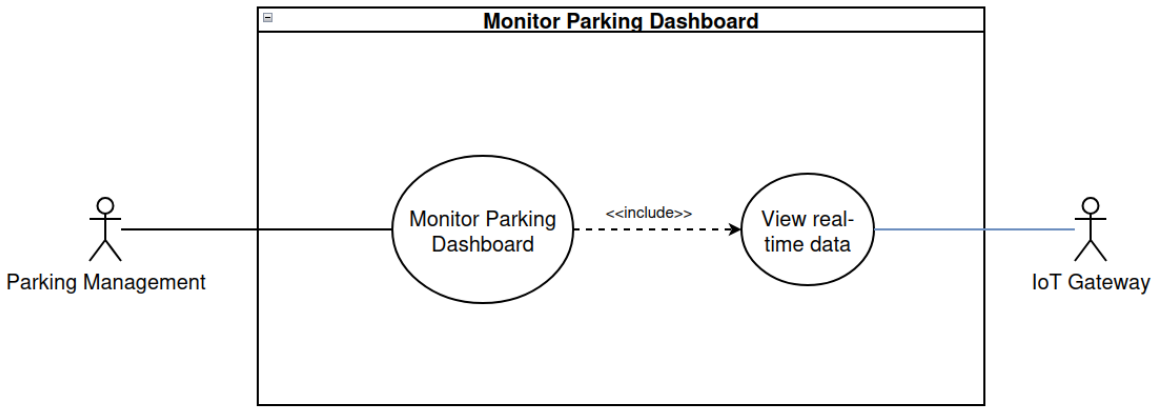

Figure 2.6: UC-05: Monitor Parking Dashboard

b. Description

Name	Monitor Parking Dashboard
ID	UC-05
Actors	Parking Management, IoT Gateway
Description	An administrative process where authorized personnel access a centralized interface to view real-time parking availability, monitor IoT hardware health, and oversee system operations.
Trigger	Parking management navigates to the parking management system URL/application.
Preconditions	- The parking manager has active, valid login credentials. - The central server and local IoT gateways are actively running.
Postconditions	- The parking manager successfully views the current state of the parking infrastructure.
Normal Flow	1. The parking manager accesses the dashboard portal. 2. The system retrieves real-time occupancy data aggregated from the IoT gateways. 3. The system retrieves the current connection status of all hardware. 4. The system renders the interactive dashboard displaying availability, active sessions, and hardware health to the Administrator.
Alternative Flow	**Step 7a (Filter Zone Data)** - The parking manager selects a specific parking zone (e.g., Faculty Lot A) from a dropdown menu. - The system filters the visual data and updates the dashboard to show only the selected zone's occupancy and hardware status.

Table 6: Monitor Parking Dashboard

2.2.6 UC-06: Handle Exception Case

a. Diagram

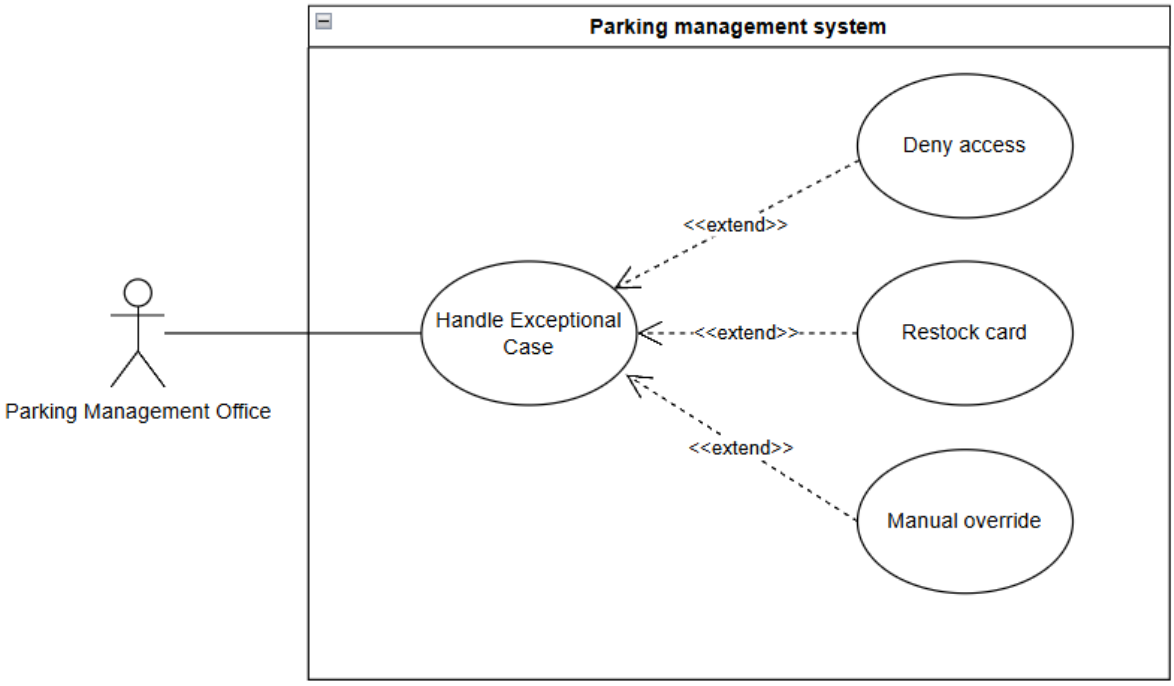

Figure 2.7: UC-06: Handle Exception Case

b. Description

Name	Handle Exception Case
ID	UC-06
Actors	Parking Management Office
Description	Manual intervention to resolve gate blocks, plate mismatches, hardware failures, or ticket/card dispenser issues.
Trigger	An automated alert (e.g., Mismatch_Plate_Error, Dispenser_Empty) or a manual support call.
Preconditions	An alert has been generated or a user has requested help via intercom.
Postconditions	The exception is resolved (gate opened, vehicle denied, or dispenser restocked) and logged.
Normal Flow	1. Actor receives the alert and opens the "Event Detail" view. 2. System displays live camera footage and the data recorded at entry/exit. 3. Actor compares the visual plate with the database record. 4. Actor selects "Manual Override": Open gate (e.g., for valid staff with a temporary car). 5. System logs the Actor's ID, the action taken, and the required reason note for audit.
Alternative Flow	**Step 4a (Deny Access):** - Actor determines the vehicle/ID is invalid or suspects fraud. - Actor selects "Deny Access": Keep gate closed. - System logs the Actor's ID, the action taken, and the reason.
Exceptions	**Step 2a (System/Hardware Failure):** - The system cannot load live footage or database records due to hardware/network failure. - Actor notifies technical staff and dispatches security to the gate for manual check. **Step 2b (Ticket/Card Dispenser Empty):** - The system detects that no physical entry cards/tickets remain in the dispenser. - An automated alert (Dispenser_Empty) is sent to the Parking Management Office. - Actor physically goes to the gate and restocks the dispenser with new cards/tickets. - Actor confirms restocking in the system interface. - System logs the Actor's ID, timestamp, and restocking action for maintenance records.

Table 7: Handle Exception Case

2.2.7 UC-07: Configure Pricing Policy

a. Diagram

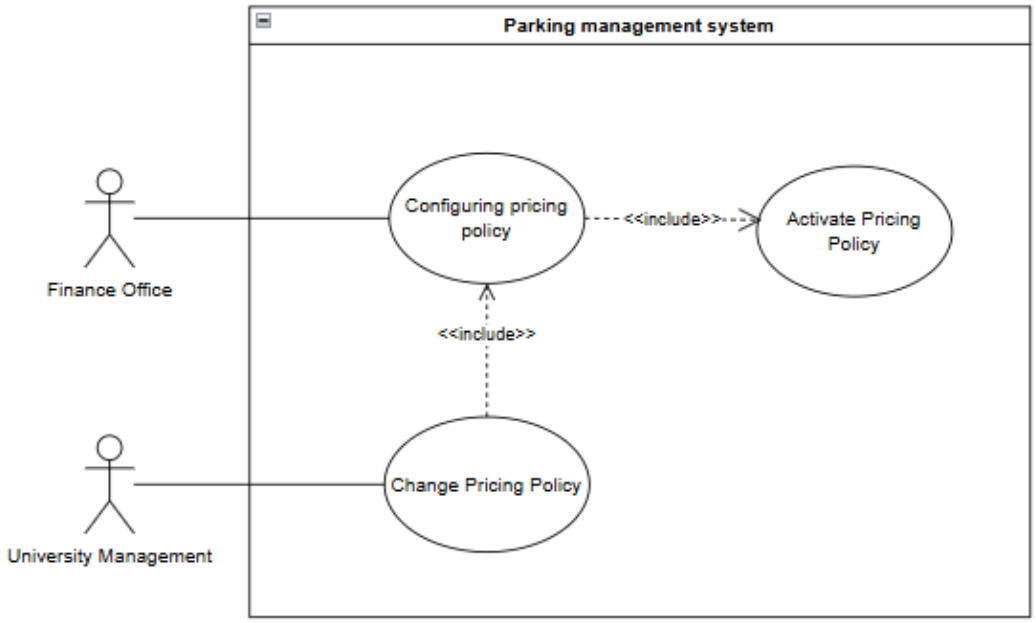

Figure 2.8: UC-07: Configure Pricing Policy

b. Description

Name	Configure Pricing Policy
ID	UC-07
Actors	Finance Office, University Management
Description	Sets up the financial rules for the system including hourly rates, subscription costs, and visitor multipliers.
Trigger	Annual review or policy change by University Management.
Preconditions	Finance Officer is authenticated.
Postconditions	New pricing logic is applied globally to fee calculations.
Normal Flow	1. Actor logs into the Finance Module. 2. Actor defines rules: Base fee, Hourly rate, Free-time grace period, and Visitor multipliers. 3. Actor sets subscription prices for Students vs. Faculty. 4. System saves and activates the policy globally.
Alternative Flow	None.
Exceptions	**Step 4a (Database Error):** - The system fails to save the new policy due to a database connection error. - System notifies the Actor that the update failed and logs the error.

Table 8: Configure Pricing Policy

2.2.8 UC-08: Issue Refund

a. Diagram

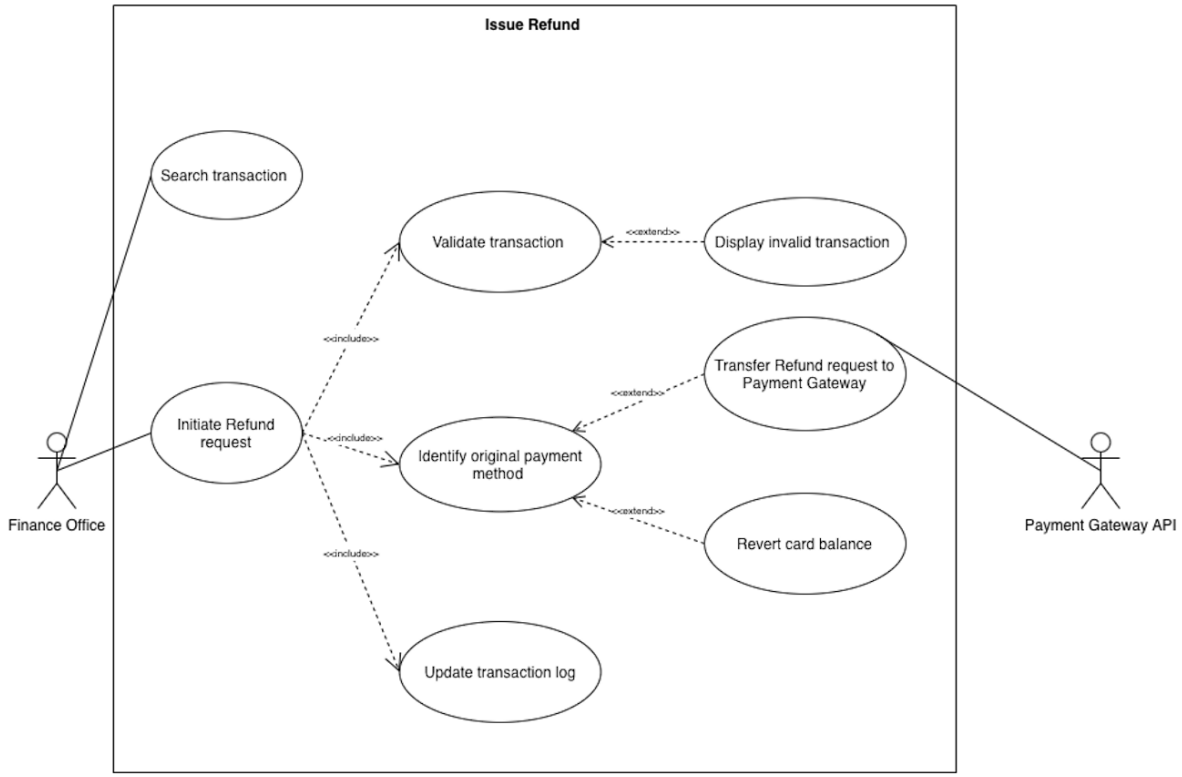

Figure 2.9: UC-08: Issue Refund

b. Description

Name	Issue Refund
ID	UC-08
Actors	Finance Office, BKPay
Description	This use case describes the process where the Finance Office processes a refund for a previously completed parking transaction due to disputes, double charges, or system errors.
Trigger	A refund request is initiated by the Finance Office after a dispute or system error is identified.
Preconditions	- The Finance Officer is authenticated. - The transaction exists in the system database. - The transaction status is "Paid" and has not been refunded before.
Postconditions	- The transaction status is updated to "Refunded". - The refunded amount is returned to the original payment source. - An audit log entry is created with actor ID, timestamp, old status, and new status.
Normal Flow	1. The Finance Officer searches for a transaction using Transaction ID, User ID, or License Plate. 2. The system displays transaction details. 3. The Finance Officer selects "Initiate Refund". 4. The Finance Officer provides a mandatory refund reason and confirms the request. 5. The system validates the transaction eligibility. 6. The system determines the original payment method. 7. If the payment was external, the system sends a refund request to the BKPay. 8. If the payment was internal, the system restores the deducted card balance. 9. The system updates the transaction status and logs the refund operation.
Exceptions	**Step 5e (Invalid Transaction):** - If the transaction does not exist or is already refunded, the system rejects the request and displays an error message.

Table 9: Issue Refund

2.2.9 UC-09: Manage User Roles and Permissions

a. Diagram

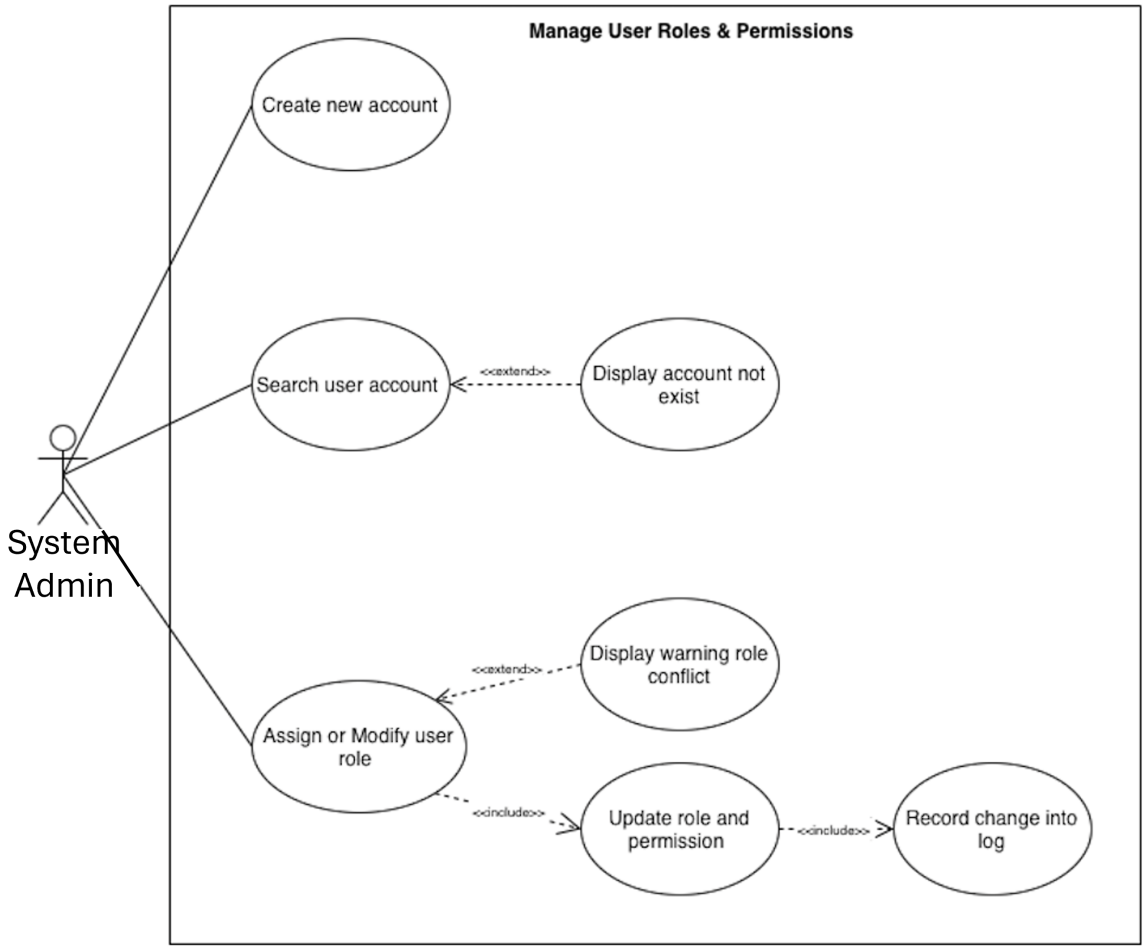

Figure 2.10: UC-09: Manage User Roles and Permissions

b. Description

Name	Manage User Roles & Permissions
ID	UC-09
Actors	System Administrator
Description	This use case describes the process where IT Management assigns, modifies, or revokes system roles and permissions for internal staff members based on their job responsibilities.
Trigger	A new staff member joins the organization or an existing staff member changes responsibilities.
Preconditions	- The System Administrator is authenticated. - The administrator has "Super Admin" privileges.
Postconditions	- The user role and access permissions are updated in the system. - The access control matrix reflects the new role. - An audit log entry is created recording the change.
Normal Flow	1. The System Administrator searches for a user account using Employee ID or Email. 2. The system displays the current role and permission details. 3. The System Administrator selects "Assign Role" or "Modify Role". 4. The System Administrator selects the appropriate role (e.g., Finance Officer, Parking Officer, System Admin). 5. The system updates the access control matrix accordingly. 6. The system records the change in the audit log with old role, new role, actor ID, and timestamp. 7. The system confirms successful update.
Alternative Flow	**Step 1e (User Not Found):** - If the user account does not exist, the system displays an error message. The actor can create new user account and search again.
Exceptions	**Step 4a (Role Conflict):** - If the selected role conflicts with existing policies, the system displays a warning.

Table 10: Manage User Roles & Permissions

2.2.10 UC-10: Integrate External API / IoT Data Ingestion

a. Diagram

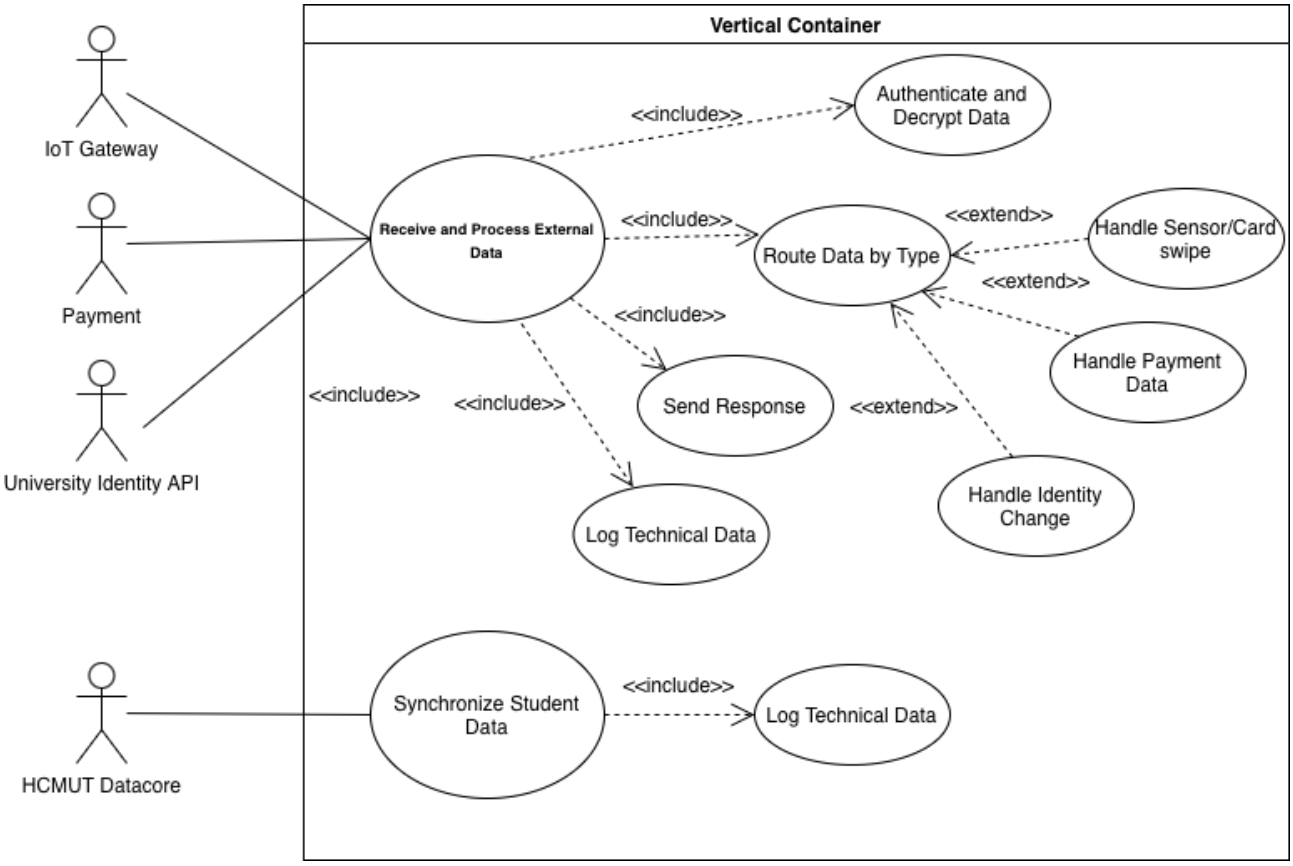

Figure 2.11: UC-10: Integrate External API / IoT Data Ingestion

b. Description

Name	Integrate External API / IoT Data Ingestion
ID	UC-10
Actors	IoT Gateway, BKPay, `HCMUT_SSO`, `HCMUT_DATACORE`
Description	This use case describes the automated background process of securely receiving, authenticating, validating, and routing data from IoT devices and financial services. The system integrates `HCMUT_SSO` for authentication and performs periodic synchronization with `HCMUT_DATACORE` to maintain authoritative user and vehicle information in the internal parking database.
Trigger	- A hardware event (sensor/card swipe). - A payment confirmation from BKPay. - A scheduled batch synchronization event with `HCMUT_DATACORE` (every 24 hours).

Table 11: Integrate External API / IoT Data Ingestion

Preconditions	- All external API endpoints (`HCMUT_SSO`, `HCMUT_DATACORE`, BKPay) are active. - Security certificates and OAuth/API keys are valid. - Secure HTTPS communication channels are established.
Postconditions	- User authentication is verified through `HCMUT_SSO`. - Parking system database contains updated user and vehicle information synchronized from `HCMUT_DATACORE`. - Relevant business logic (entry authorization, billing update, privilege validation) is executed using locally stored data.
Normal Flow	1. **Data Ingestion:** An external actor sends an encrypted JSON packet. 2. **Authentication via SSO:** The system validates the API key/token with `HCMUT_SSO`. If valid, the identity token is verified and decrypted. 3. **Local User Data Lookup:** The system retrieves the user profile, role, and registered vehicle information from the internal parking database. 4. **Payment Handling:** If the event originates from BKPay, the system validates the transaction and updates the parking session or subscription status in the Finance database. 5. **Business Logic Execution:** The system triggers appropriate actions (grant/deny entry, update session, adjust slot count). 6. **Response:** The system sends an ACK (Acknowledgement) or requested response data to the originating service. 7. **Logging:** All transactions and API communications are logged in Technical Logs for audit and administrative review. 8. **Batch Synchronization with DATACORE:** Every 24 hours, the system initiates a background synchronization with `HCMUT_DATACORE` to update student and faculty profiles. This process updates access privileges, revokes access for graduated students, updates newly registered faculty license plates, and refreshes subscription or eligibility status in the internal parking database.
Alternative Flow	**Step 2a (SSO Authentication Failure):** - The token/API key is invalid or expired. - The system rejects the request and returns an authentication error response. - No further processing occurs. **Step 4a (Payment Verification Failure):** - Transaction ID is invalid or payment not confirmed. - The session status is not updated and an error is logged.
Exceptions	- **SSO Timeout:** If the authentication service is unreachable, the system returns a NACK (Negative Acknowledgement) and logs the technical failure. - **DATACORE Synchronization Failure:** If the scheduled synchronization with `HCMUT_DATACORE` fails, the system logs the error and retries based on the synchronization policy while continuing to operate using the last successfully synchronized dataset. - **Decryption Failure:** If payload decryption fails, the packet is discarded and an alert is generated in Technical Logs.

Table 11 – Continue from the previous page

2.2.11 UC-11: Configure System Parameters and Business Rules

a. Diagram

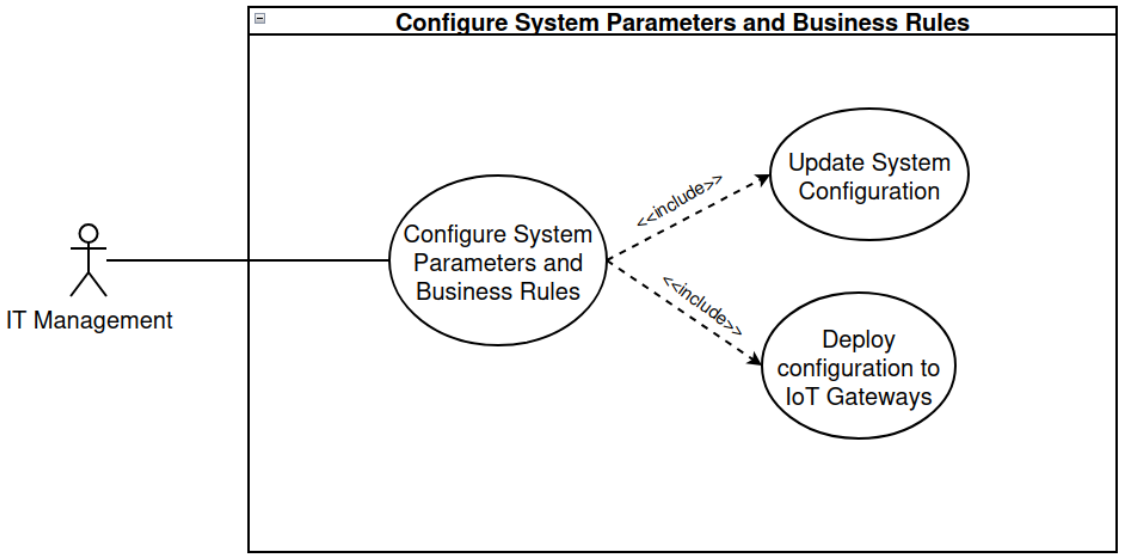

Figure 2.12: UC-11: Configure System Parameters and Business Rules

b. Description

Name	Configure System Parameters & Business Rules
ID	UC-11
Actors	IT Management
Description	Sets global technical variables like sensor sensitivity and gate timeouts.
Trigger	Hardware maintenance or operational optimization.
Preconditions	- IT Management is authenticated.
Postconditions	- IoT hardware behavior is updated.
Normal Flow	1. IT Management enters "System Settings" in the portal. 2. IT Management adjusts variables: Grace Period (e.g., 15m), Gate Timeout, and ALPR confidence thresholds. 3. System pushes configuration to IoT Gateways. 4. System confirms successful deployment.
Alternative Flow	**Step 2a (Invalid Value Entered):** - IT Management enters a value outside the accepted range. - The system displays a validation error and prompts the IT Management to correct the input.
Exceptions	- **IoT Gateway Unreachable:** If one or more IoT Gateways do not acknowledge the configuration push, the system logs the failure and alerts the IT Management to retry.

Table 12: Configure System Parameters & Business Rules

2.2.12 UC-12: Monitor and Review Operational Logs

a. Diagram

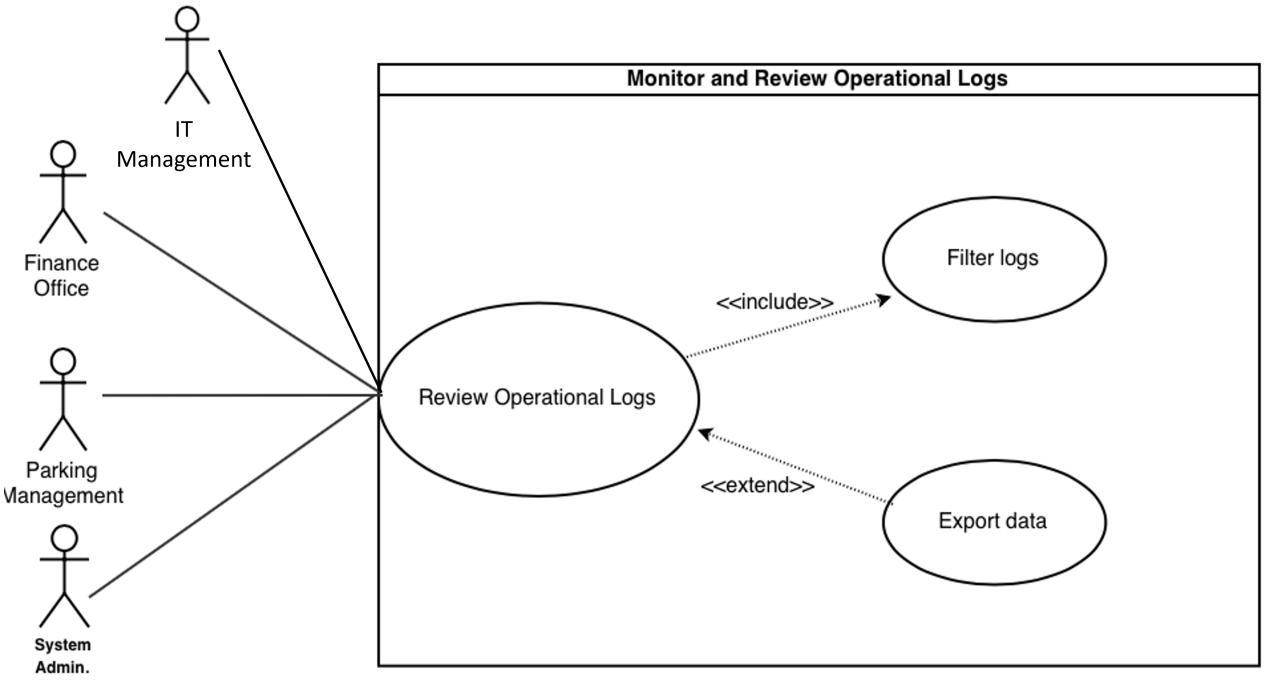

Figure 2.13: UC-12: Monitor and Review Operational Logs

b. Description

Name	Monitor and Review Operational Logs
ID	UC-12
Actors	Finance Office, Parking Management, System Admin, IT Management
Description	This use case describes the process of administrative staff accessing, reviewing, and exporting operational logs, which are automatically filtered based on their specific roles.
Trigger	An actor needs to conduct a routine audit, generate a report, or investigate an incident.
Preconditions	- The system is operational and actively generating/storing log data. - The actor is successfully authenticated and logged into the System Dashboard.
Postconditions	- The actor successfully views the requested operational logs. - (Optional) The log data is exported and downloaded to the actor's device.
Normal Flow	1. The actor accesses the log monitoring section on the Dashboard. 2. The system identifies the actor's specific role. 3. The system displays logs filtered by the actor's permissions (Finance: Payment/Refund; Parking: Entry/Exit/Occupancy; Admin: All logs). 4. The actor inputs specific criteria to filter the data (e.g., by Date, Time, or Transaction/Vehicle ID). 5. The system fetches and displays the matching results. 6. The actor selects the option to export the data. 7. The system generates and downloads the report (PDF/CSV).
Alternative Flow	**Step 6a (No export needed):** - The actor views the necessary logs on the screen and exits the module without exporting. **Step 5a (No data found):** - The system finds no logs matching the applied Date/ID filters. - The system displays a "No records found" message.
Exceptions	- **Database Timeout:** The system fails to retrieve logs due to a database connection error; an error message is displayed. - **Access Violation:** If an actor attempts to view logs outside their designated role, the system blocks the action and displays an "Access Denied" message.

Table 13: Monitor and Review Operational Logs

2.2.13 UC-13: View Personal Parking & Payment History

a. Diagram

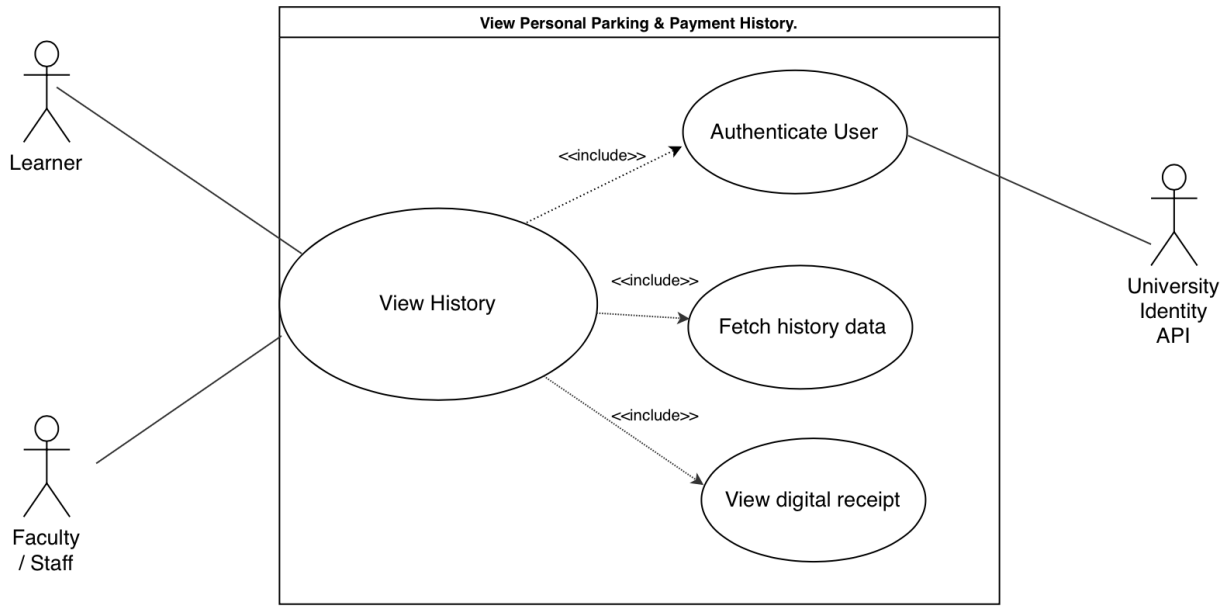

Figure 2.14: UC-13: View Personal Parking & Payment History

b. Description

Name	View Personal Parking & Payment History
ID	UC-13
Actors	Learner, Faculty, Staff, HCMUT_SSO
Description	This use case allows registered internal users to access and view their own parking activity, durations, and financial deductions through a mobile or web portal.
Trigger	The user wants to check their internal balance, verify a specific charge, or review recent parking sessions.
Preconditions	- The user is logged into the mobile/web self-service portal. - The system can communicate with the HCMUT_SSO.
Postconditions	- The user is successfully informed of their parking and payment history.
Normal Flow	1. The user navigates to the "History" section on the portal. 2. The system authenticates the user's session via the HCMUT_SSO. 3. The system fetches the data based on the user's ID. 4. The system displays the records including Date, Duration, Fee, and Payment method (e.g., "Deducted from Balance"). 5. The user selects a specific session to view the digital receipt.
Alternative Flow	**Step 2a (Authentication Failed):** - The API rejects the token/session. - The system prompts the user to log in again. **Step 4a (No History Available):** - The user has not parked recently. - The system displays a "No parking records found" message.
Exceptions	- **API Timeout:** If HCMUT_SSO is unreachable, the system displays a "Service temporarily unavailable" message and logs the technical error.

Table 14: View Personal Parking & Payment History

2.3 Non-interactive functional requirements

Automatic Session Timeout & Cleanup

- **Description:** The system shall automatically identify and flag "Ghost Sessions" where a vehicle entered the lot but no exit was recorded within 48 hours.
- **Trigger:** A background "cleanup" script that runs every 6 hours.
- **Purpose:** Alerts the Parking Management Office to potential security risks (e.g., stolen vehicles, lost cards, or hardware failures where a vehicle exited without a swipe).

Automated Monthly Reporting & Data Archiving

- **Description:** On the first day of every month at 12:00 AM, the system shall automatically generate and distribute two comprehensive PDF/CSV reports: the Financial Revenue Report and the Parking Utilization Report.
- **Trigger:** System internal clock (Cron job).
- **Purpose:** Ensures consistent oversight of revenue and space efficiency without requiring manual data extraction.

Comprehensive User Activity Logging

- **Description:** The system shall automatically record and maintain a complete audit trail of every user interaction and parking-related activity, from card authentication to parking session closure and payment completion.
- **Details:** The activity log must include, but is not limited to:
  - User ID / Card ID
  - User Role (Student/Staff/Teacher/Visitor)
  - Timestamp of each event
  - Event Type (Card Swipe, Authentication Result, Access Granted/Denied, Entry Gate Opened, Exit Gate Opened, Payment Initiated, Payment Confirmed, Session Closed)
  - Gate Location (Entry/Exit point ID)
  - Parking Area Type (General / Privileged)
  - Payment Amount (if applicable)
  - Payment Transaction ID (if applicable)
  - System Response Status (Success/Failure)
  - IP Address or Device ID (for API-triggered events)
- **Purpose:** To provide a complete end-to-end trace ("black box") of each parking session and system interaction, enabling University Management to audit user behavior, investigate security incidents, resolve financial discrepancies, and ensure accountability.

Background Audit Logging of Admin Actions

- **Description:** The system shall automatically record every configuration change made by a System Admin or Finance Office user into a read-only Audit Log.
- **Details:** The log must include the User ID, Timestamp, IP Address, Old Value, and New Value (e.g., "Base fee changed from $2 to $3").
- **Purpose:** Provides a "black box" for University Management to review if a security breach or financial discrepancy occurs.

Automated Occupancy Calculation

- **Description:** The system shall automatically update the available parking count for each specific lot (Learner, Faculty, or Visitor) every time a vehicle successfully passes through an entry or exit barrier.
- **Trigger:** Hardware signal from the induction loop sensor (post-barrier).
- **Purpose:** Ensures the "Lot Full" status is accurate in real-time for the Parking Management Office dashboard and entry kiosks without manual counting.

3 Non-functional requirements

Performance and Scalability

- **Latency:** The system shall process an ID card swipe and trigger the barrier gate (including ALPR license plate matching) in less than 2 seconds under normal load.
- **Throughput:** The IoT Gateway must be capable of handling up to 50 concurrent entry/exit events across multiple gates without data loss.
- **Scalability:** The system architecture shall support an increase of up to 20% in total users (Learners/Staff) per year without requiring a hardware overhaul of the central server.

Availability and Reliability

- **Uptime:** The system shall maintain 99.9% availability (excluding scheduled maintenance), ensuring the parking lot remains accessible 24/7.
- **Fault Tolerance:** In the event of a HCMUT_SSO outage, the system shall switch to a "Local Cache Mode" to allow pre-verified users (Staff/Faculty) entry based on the last synchronized data.
- **Data Integrity:** The system must ensure 100% accuracy in financial transactions; no "Parking Session" shall be closed without a corresponding log in the Finance Office database.

Security and Privacy

- **Authentication:** All administrative access (System Admin, Finance, Parking Office) must require Multi-Factor Authentication (MFA).
- **Data Encryption:** All sensitive data, including user license plates and payment tokens from the BKPay, must be encrypted at rest and in transit using AES-256 and TLS 1.3.
- **Privacy Compliance:** The system shall automatically mask personally identifiable information (PII) in general logs, showing only User_ID instead of full names to unauthorized staff.

Usability and Accessibility

- **Kiosk Interface:** The Automated Visitor Kiosk UI shall be designed for high-glare outdoor environments and must be operable by users in less than 45 seconds (from arrival to card issuance).
- **Multi-language Support:** The user-facing portals and kiosks shall support at least two languages (e.g., English and the local primary language).
- **Responsiveness:** The User Self-Service Portal must be mobile-responsive, allowing Students/Staff to check their history or balance on any smartphone browser.

Maintainability and Auditability

- **Audit Trail:** The system shall maintain a non-modifiable Audit Log for a minimum of 5 years to comply with university financial regulations.
- **Modular Design:** The system shall use a microservices-based approach so that the BKPay can be swapped or updated without taking the IoT Gateway (entry/exit gates) offline.
- **Logging:** Technical logs must be generated for every API handshake failure, with automated alerts sent to the System Admin within 60 seconds of a critical failure.

Robustness and Disaster Recovery

- **Backup Frequency:** The system shall perform an automated, encrypted backup of the entire database (Transactions, User Roles, and Logs) every 4 hours to an off-site cloud storage location.
- **Recovery Time Objective (RTO):** In the event of a total server failure, the IT Management team shall be able to restore the system to full operational status within 4 hours.

- **Power Resilience:** The IoT Gateway and physical barrier components must be connected to an Uninterruptible Power Supply (UPS) capable of maintaining gate operations for at least 30 minutes during a local power outage.

Interoperability and Standards

- **API Standardization:** All communication between the parking system and external actors (Payment Gateway, University Database) shall follow RESTful API standards using JSON for data exchange.
- **Hardware Agnostic:** The software shall be designed to be compatible with any ALPR camera or sensor that supports the ONVIF or MQTT protocols, preventing "vendor lock-in."
- **Standard Time Protocol:** All system components (Gates, Servers, Kiosks) must synchronize their internal clocks via NTP (Network Time Protocol) to ensure entry/exit timestamps are accurate to within 100 milliseconds.

Capacity and Resource Constraints

- **Storage Longevity:** The system server shall have sufficient storage capacity to retain high-resolution license plate images for 90 days and text-based transaction logs for 7 years.
- **Bandwidth Efficiency:** The IoT Gateway shall be optimized to ensure that image transmission from ALPR cameras does not consume more than 5 Mbps of the university's local area network (LAN) per gate.

Portability and Environment

- **Hardware Durability:** All outdoor IoT components (Sensors, Kiosks, Barrier Actuators) must have an IP65 rating or higher to ensure functionality in extreme weather conditions (heavy rain, dust, and high heat).
- **Browser Compatibility:** The administrative and user portals shall maintain full functionality across the latest versions of Chrome, Safari, and Edge.

4 Sequence Diagram

4.1 UC-01: Enter Parking Lot

This diagram outlines the "Enter Parking Lot" usecase. When a driver swipes their ID at the IoT Gateway, the system validates the credentials via HCMUT_SSO and checks for available slots in the Parking Database. If the ID is valid and space is available, the database logs the session and the gateway opens the gate for entry. If the ID is invalid or the parking lot is full, the system denies entry, keeps the gate closed, and displays an error message.

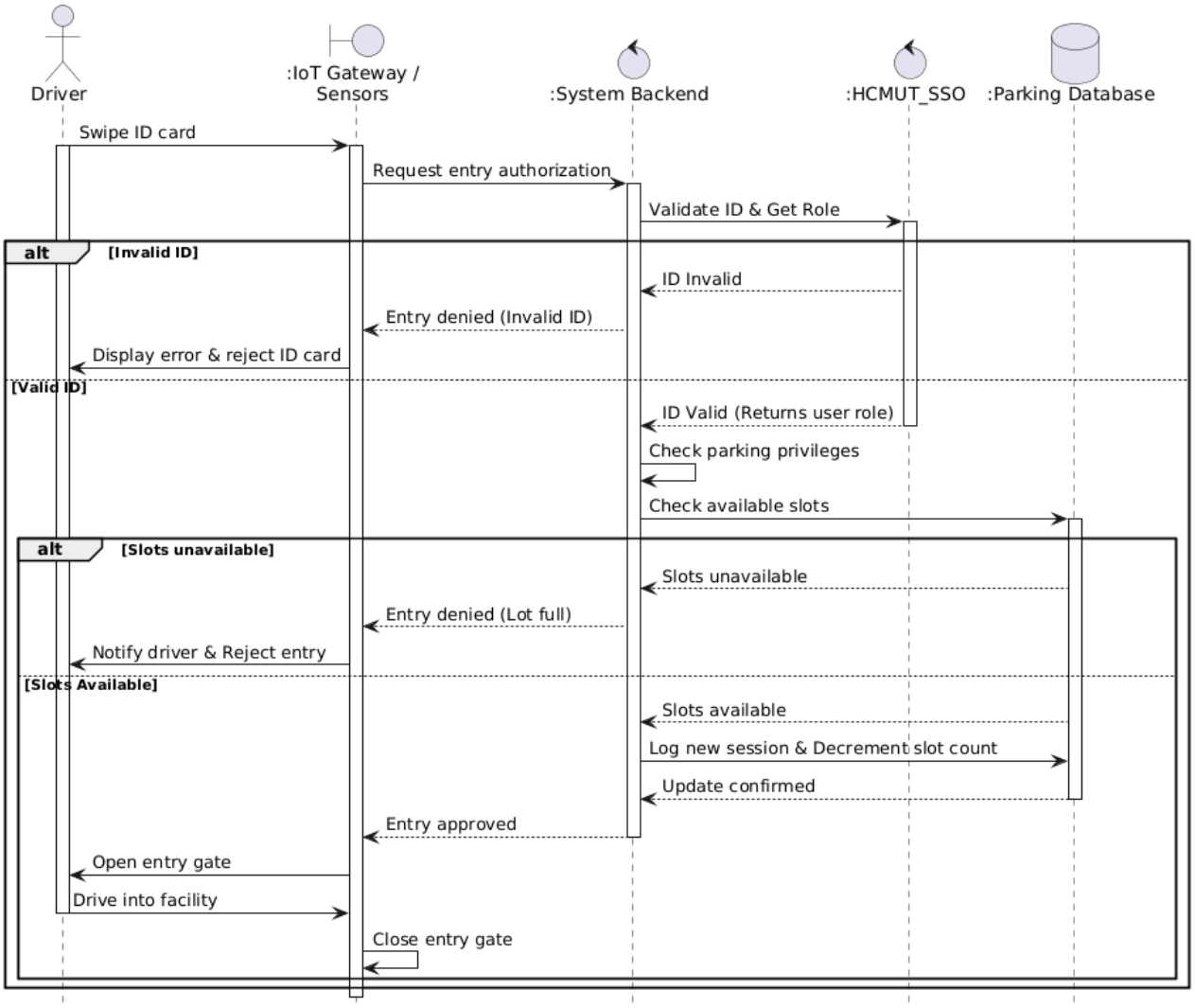

Figure 4.1: Enter Parking Lot

4.2 UC-02: Exit Parking Lot

This diagram covers the "Exit Parking Lot" usecase. When a driver swipes their ID, the IoT Gateway validates it via HCMUT_SSO. The flow branches: standard users are authorized immediately, visitors must first pay a calculated fee, and invalid IDs are explicitly denied. Upon successful exit authorization, the Parking Database logs the session's end and frees a parking slot, prompting the gateway to open the exit gate.

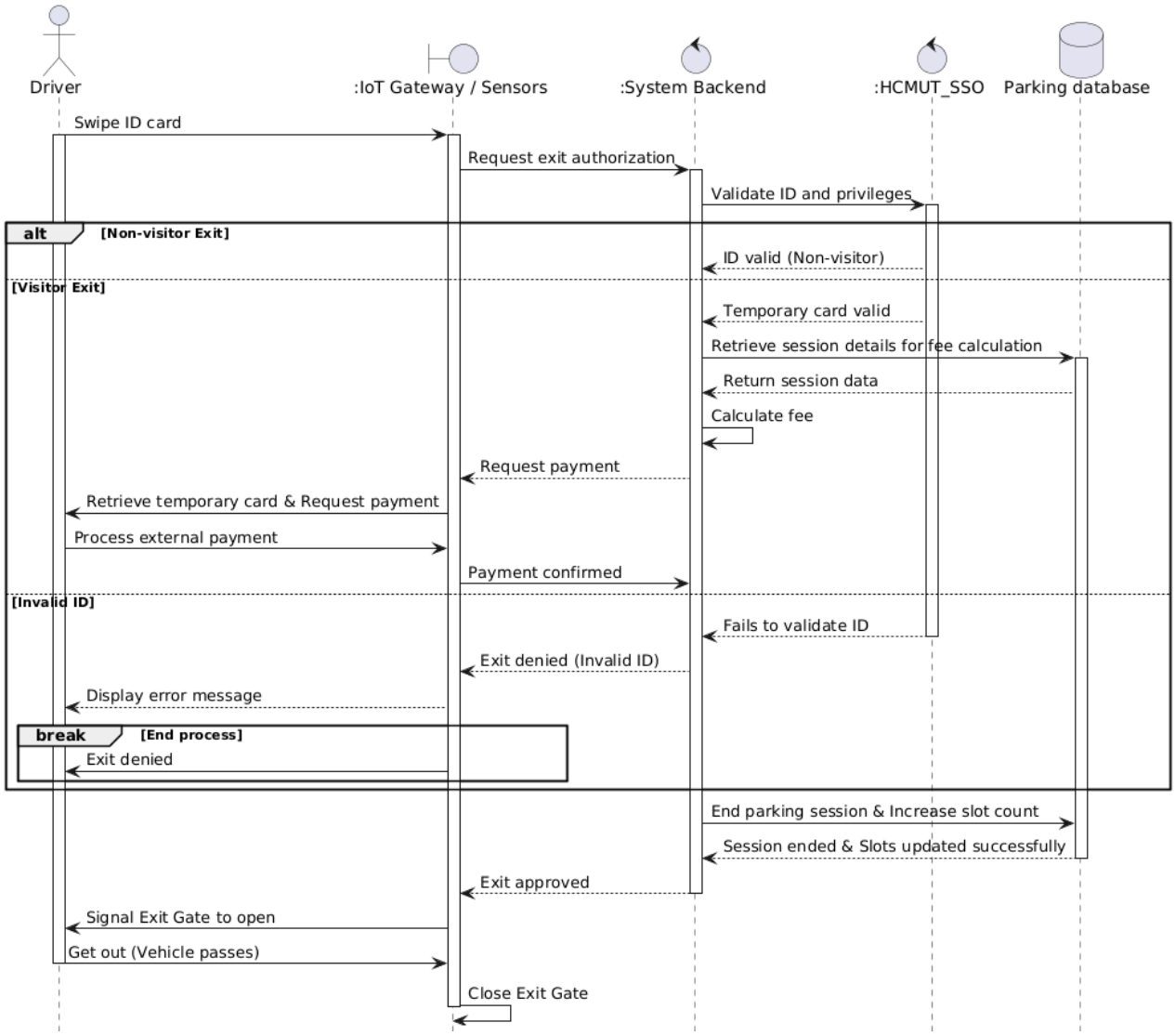

Figure 4.2: Exit Parking Lot

4.3 UC-03: Process Payment

This diagram outlines the "Process Payment" use case, which covers two methods. Billing Period Payments calculate the total fee and validate funds via BKPay, resulting in a "Paid" status or a recorded debt for insufficient balances. Conversely, External Payments at Exit calculate the session fee for a cash payment, immediately updating the database to "Paid."

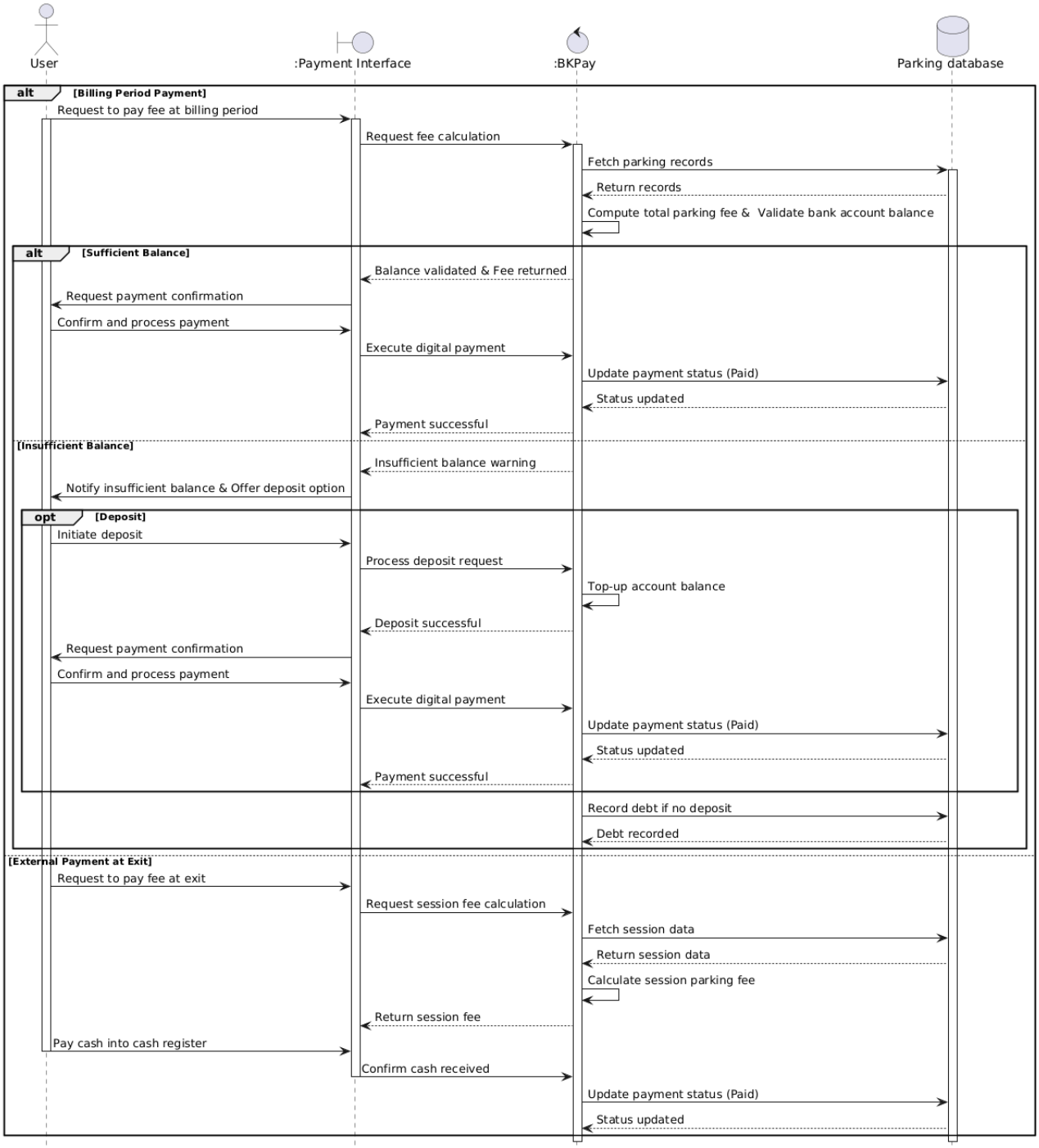

Figure 4.3: Process Payment

4.4 UC-04: Issue Temporary Card

The "Issue temporary card" use case begins with the IoT Gateway verifying parking capacity upon a visitor's arrival, grant a temporary card for visitor, denying entry if the lot is full. The empty dispenser case is also included.

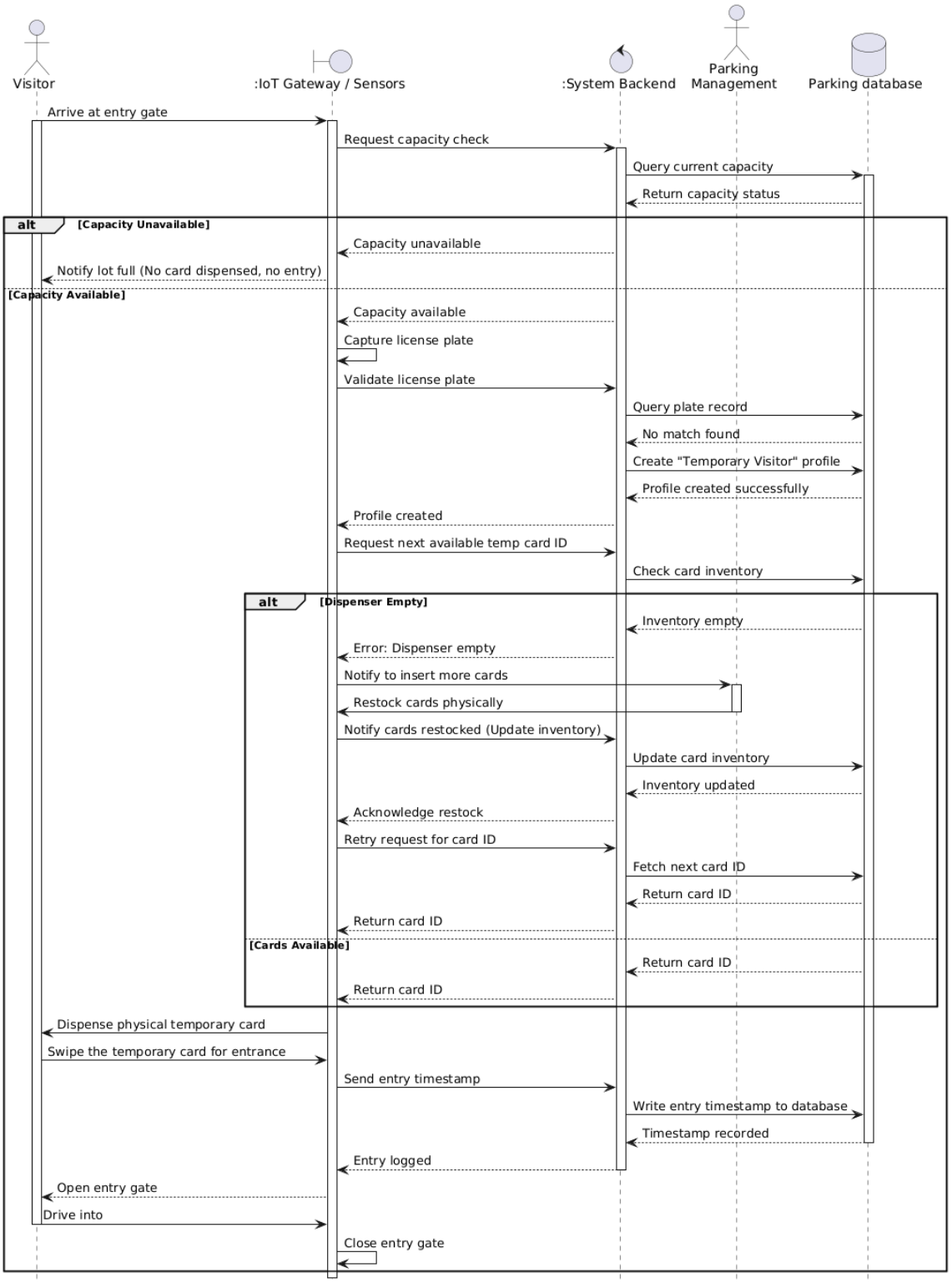

Figure 4.4: Issue Temporary Card

4.5 UC-05: Monitor Parking Dashboard

This sequence diagram illustrates the "Monitor Parking Dashboard" usecase. Upon initial access, the portal requests data through the Dashboard Backend, which retrieves real-time occupancy and hardware health statuses from the Parking Database—data that is continuously updated in the background by the IoT Gateway. The portal then renders this comprehensive view for the user. Additionally, an optional flow allows the manager to select a specific parking zone, prompting the backend to query and the portal to display targeted occupancy and status data for that specific area.

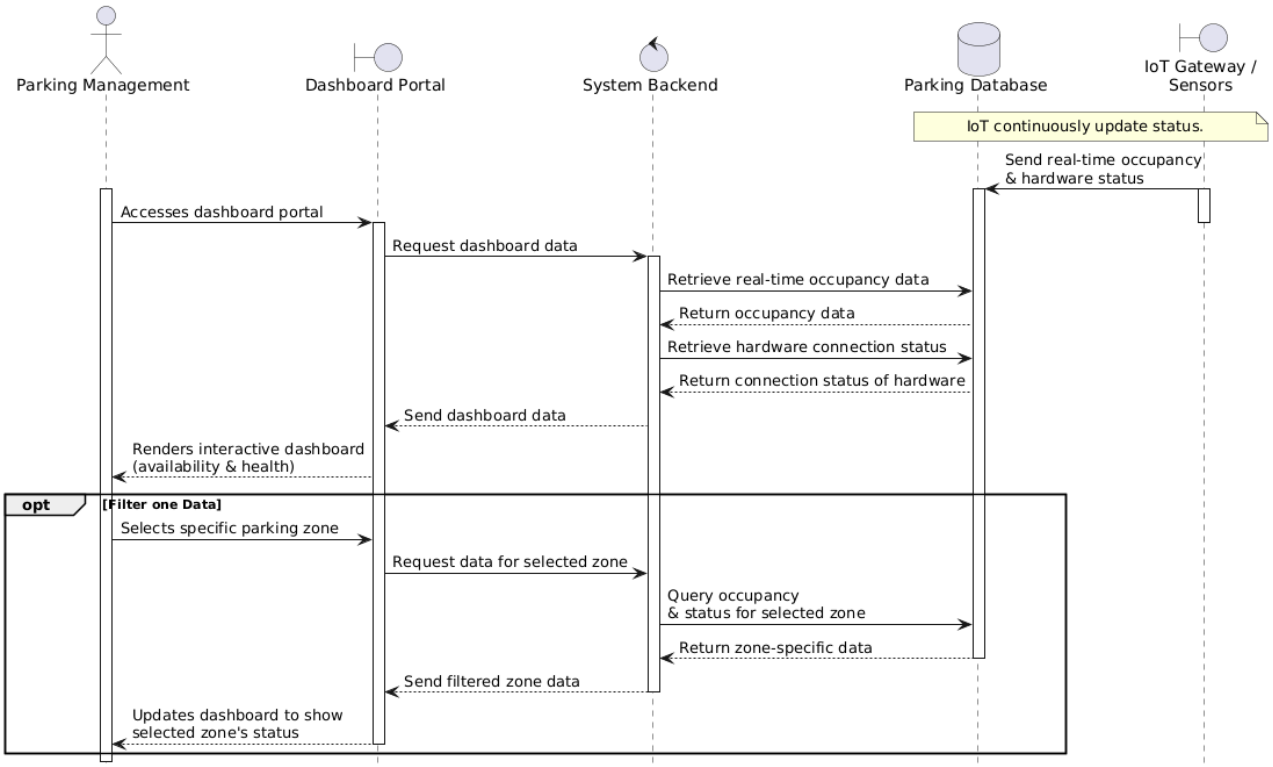

Figure 4.5: Monitor Parking Dashboard

4.6 UC-06: Handle Exception Cases

This diagram outlines the "Handle Exception Cases" usecase. Staff respond to alerts by dispatching technicians for hardware failures, restocking empty card dispensers, or using live footage to manually allow or deny access during plate mismatches. All interventions are securely logged in the database.

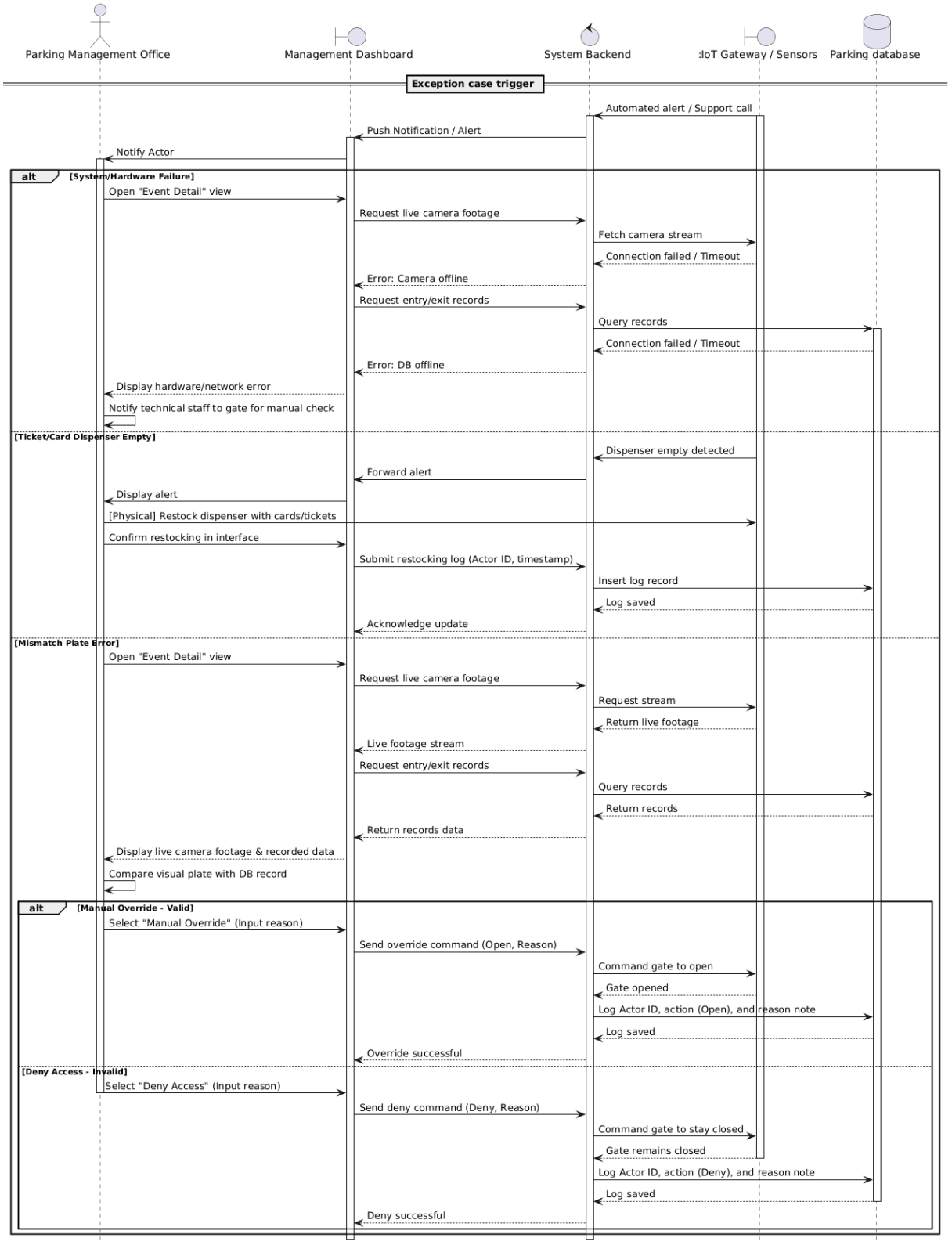

Figure 4.6: Handle Exception Cases

4.7 UC-07: Configure Pricing Policy

This diagram outlines the "Configure Pricing Policy" use case. The Finance Office updates parking pricing rules when requested by University Management during annual reviews or policy changes. The actor opens the Finance Module, reviews the current pricing configuration, and submits updated values such as base fee, hourly rate, grace period, visitor multiplier, and subscription prices. The system validates the rules, stores the policy in the database, and activates it for global use. If invalid logic is detected or the database fails to save the new policy, the system reports the issue to the actor.

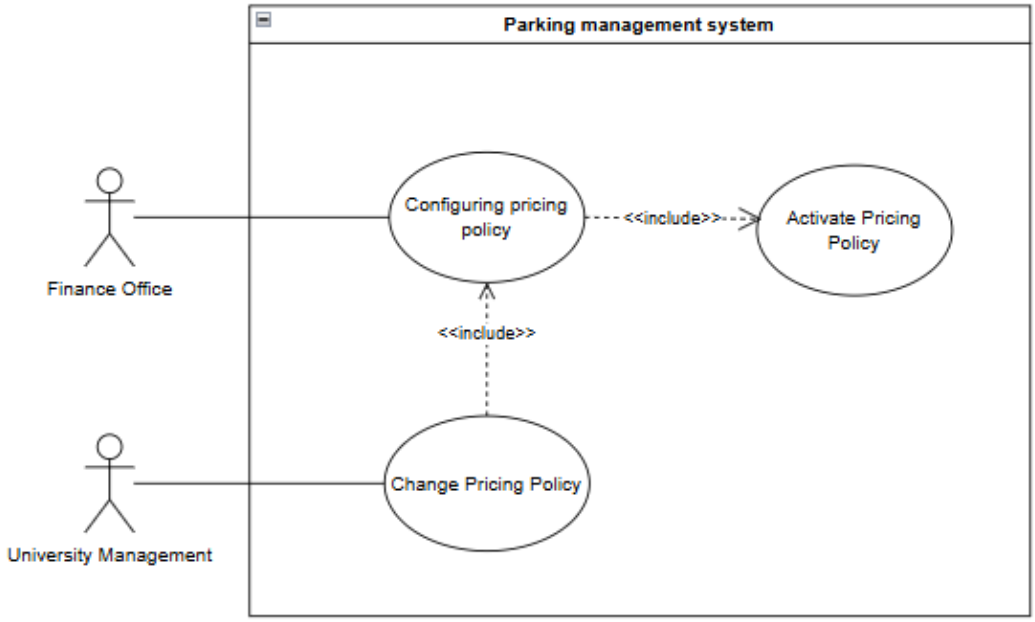

Figure 4.7: Configure Pricing Policy

4.8 UC-08: Issue Refund

This diagram outlines the "Issue Refund" use case. The Finance Office searches for a completed parking transaction and reviews its details before initiating a refund request with a required reason. The system validates whether the transaction is eligible for refund and identifies the original payment method. If the payment was made through an external gateway, the system sends a refund request to BKPay; if the payment was internal, the deducted card balance is restored. All successful refund operations are recorded in the audit log, while invalid or already refunded transactions are rejected and reported back to the actor.

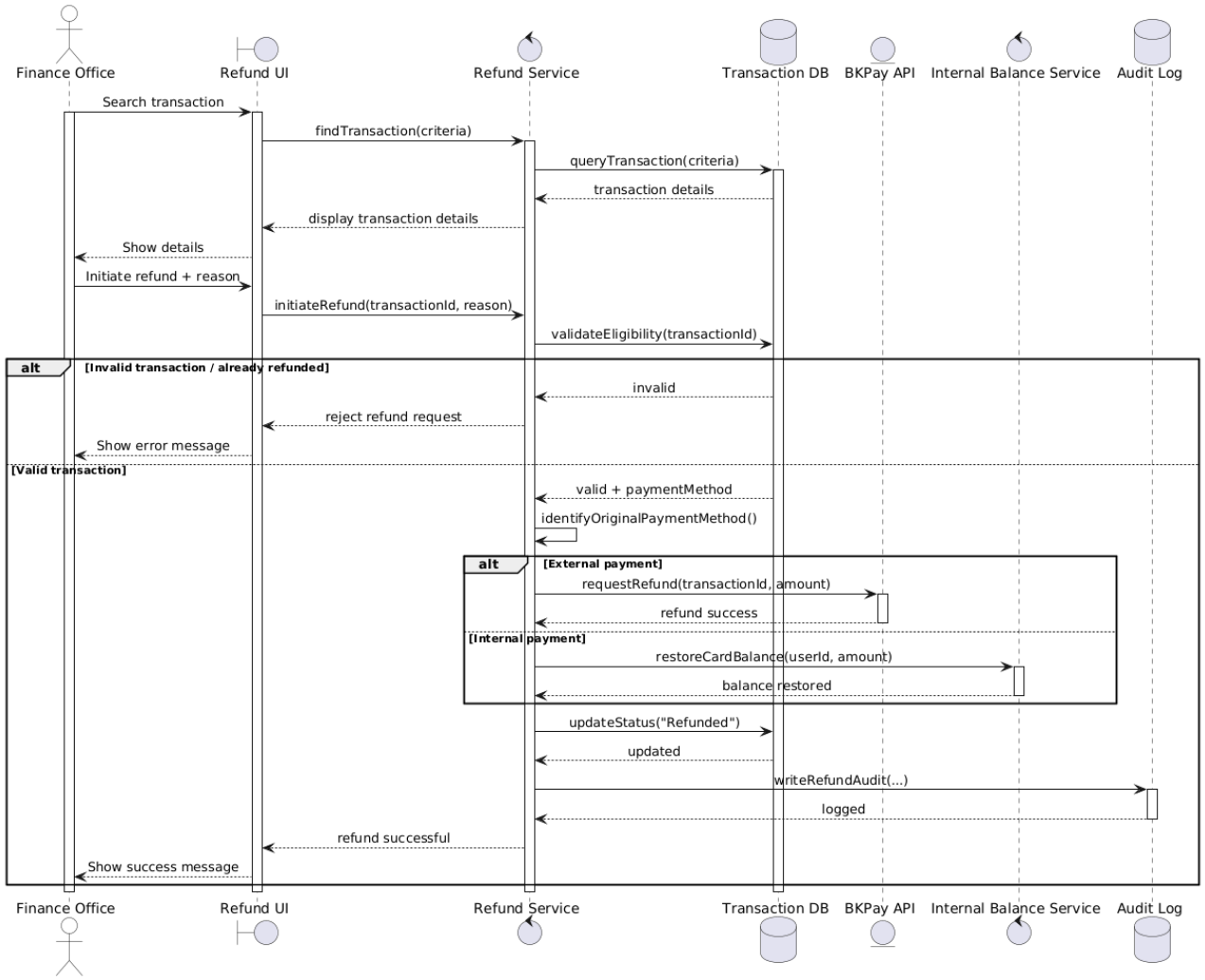

Figure 4.8: Issue Refund

4.9 UC-09: Manage User Roles and Permissions

This diagram outlines the "Manage User Roles and Permissions" use case. The System Administrator searches for a staff account using identifying information such as employee ID or email, then reviews the user's current role and permission details. The administrator can assign or modify roles according to job responsibilities, and the system validates whether the selected role conflicts with existing policies. If the role is valid, the access control matrix is updated, the new role is saved in the database, and the change is recorded in the audit log. If the user cannot be found or a role conflict occurs, the system displays the appropriate warning or error message.

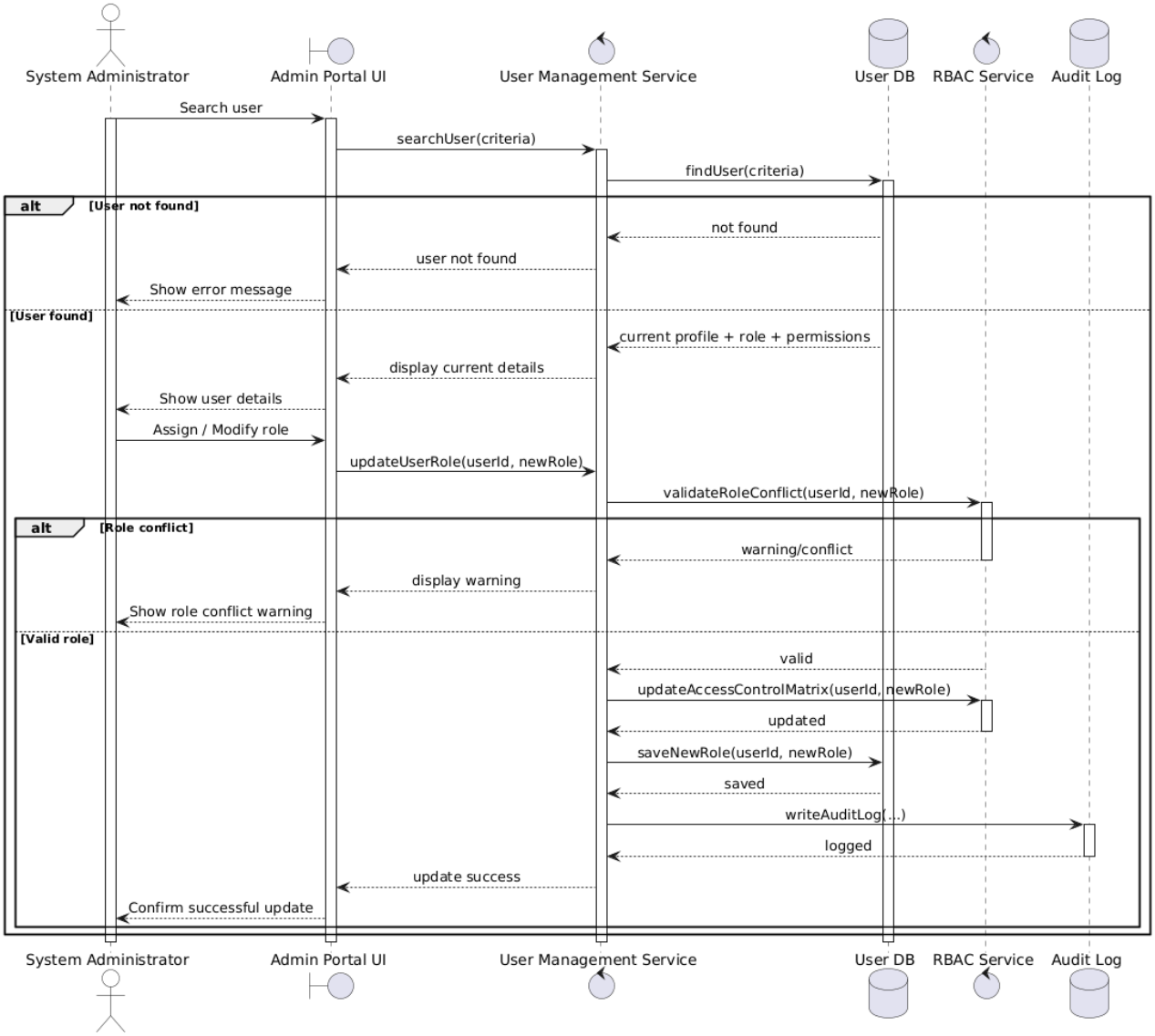

Figure 4.9: Manage User Roles and Permissions

4.10 UC-10: Integrate External API / IoT Data Ingestion

This subsection presents the "Integrate External API / IoT Data Ingestion" use case through two related sequence diagrams. In the real-time ingestion flow, events from IoT Gateway devices or BKPay are received as encrypted data packets, authenticated through HCMUT_SSO, decrypted, matched with local parking data, and processed by the business logic service. Depending on the event type, the system updates entry/exit sessions, validates payments, or returns acknowledgment and error responses while recording all activities in the technical log. In the batch synchronization flow, the system periodically connects to HCMUT_DATACORE to refresh user profiles, vehicle information, privileges, and eligibility data in the parking database. Synchronization failures are logged and retried according to the system policy.

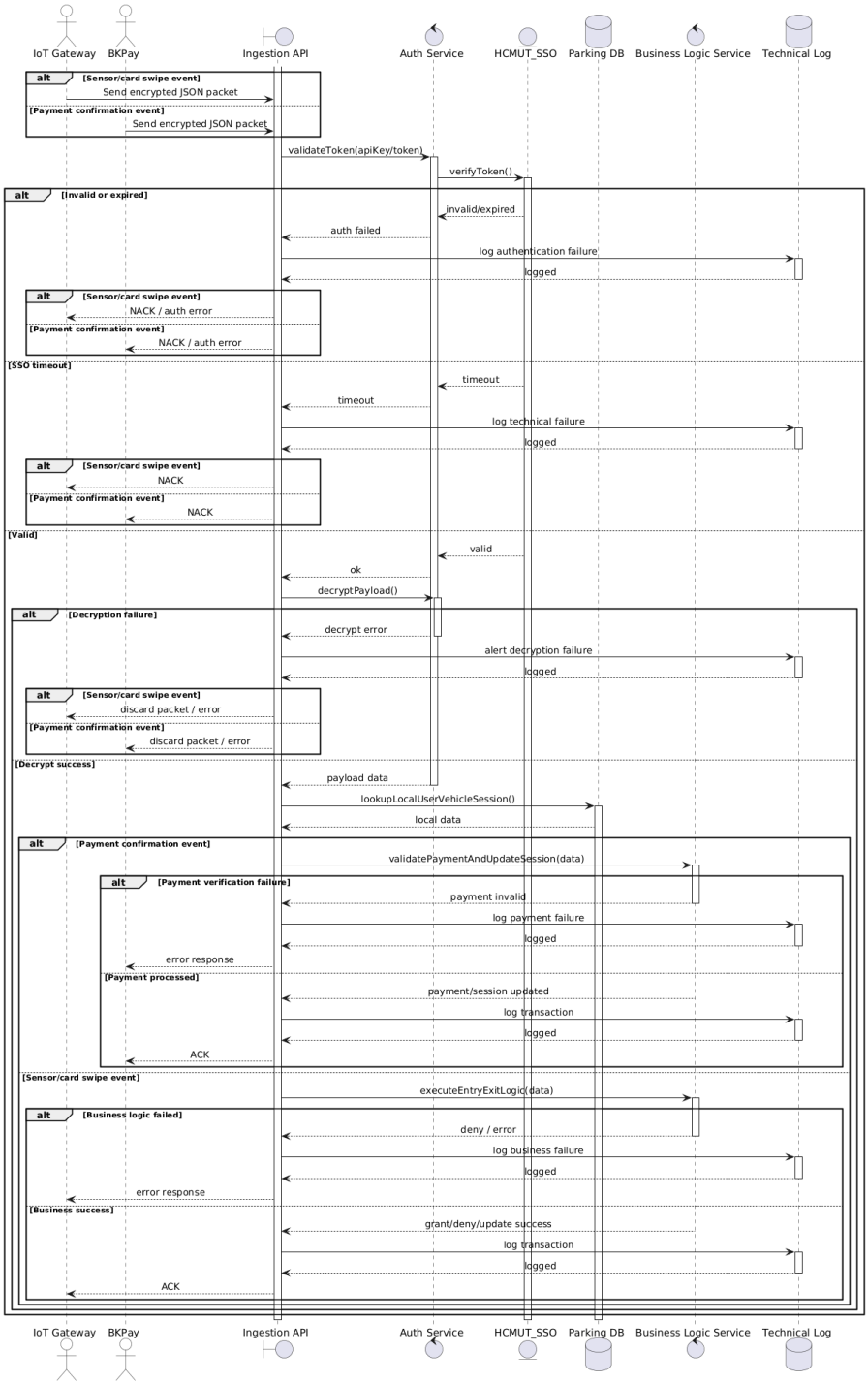

Figure 4.10: Integrate External API / IoT Data Ingestion – Real-time External Event Ingestion

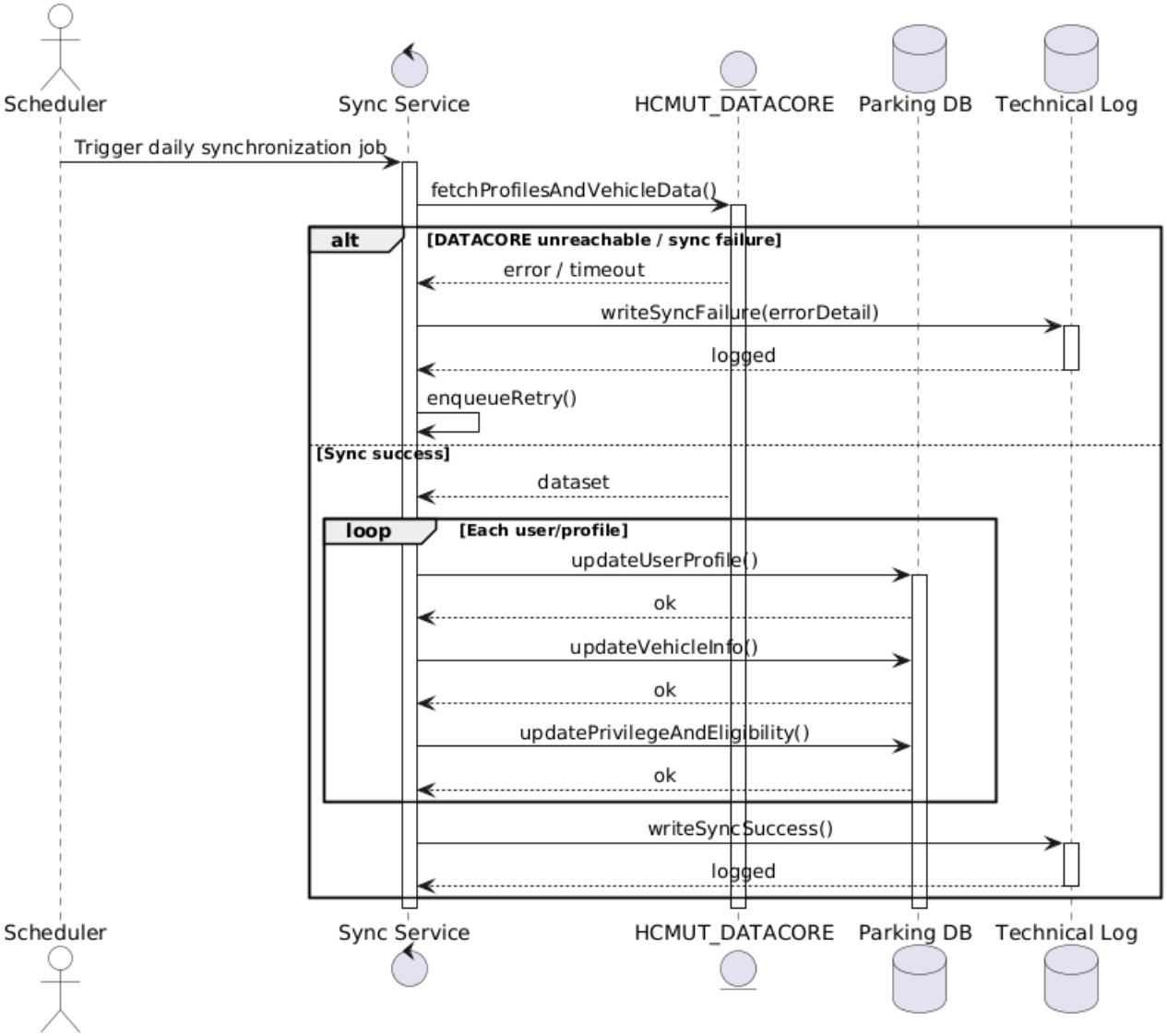

Figure 4.11: Integrate External API / IoT Data Ingestion – Batch Synchronization with DATACORE

4.11 UC-11: Configure System Parameters and Business Rules

This diagram outlines the "Configure System Parameters and Business Rules" use case. IT Management accesses the System Settings module to adjust global technical parameters such as grace period, gate timeout, and ALPR confidence threshold. After the new values are submitted, the system validates them against accepted ranges, saves the approved configuration, and deploys it to the IoT Gateway infrastructure. If an invalid value is entered, the system prompts the actor to correct it. If one or more IoT Gateways fail to acknowledge the update, the system logs the deployment failure and alerts IT Management to retry the operation.

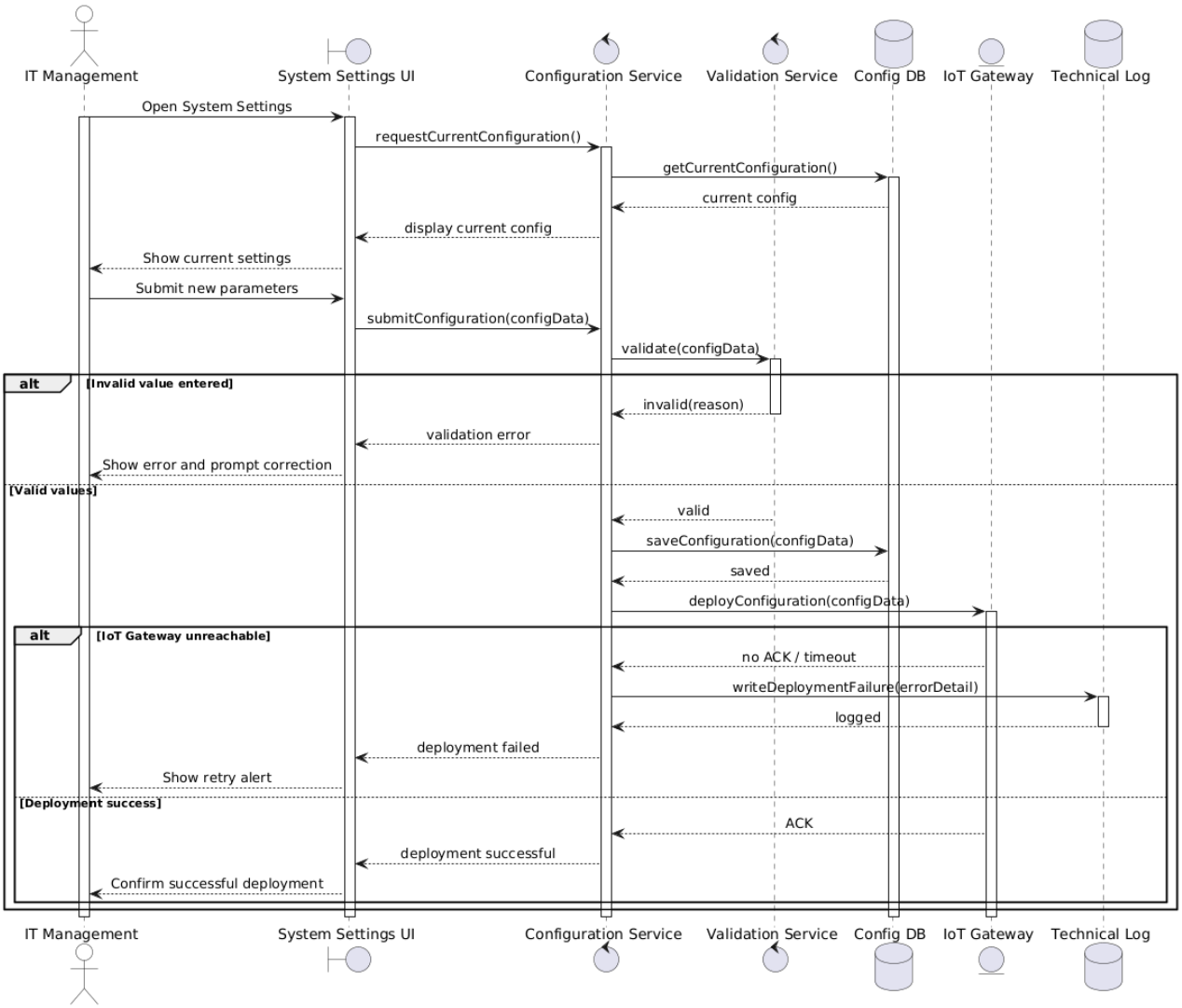

Figure 4.12: Configure System Parameters and Business Rules

4.12 UC-12: Monitor and Review Operational Logs

This diagram outlines the "Monitor and Review Operational Logs" use case. Administrative actors such as Finance Office, Parking Management, System Admin, and IT Management access the log dashboard to review operational records according to their assigned permissions. The system identifies the actor's role, applies role-based filtering, and retrieves matching entries based on criteria such as date, time, transaction ID, or vehicle ID. The actor may review the results directly on screen or export them as PDF or CSV reports for auditing or reporting purposes. The system also handles special situations such as access violations, empty query results, and database timeouts.

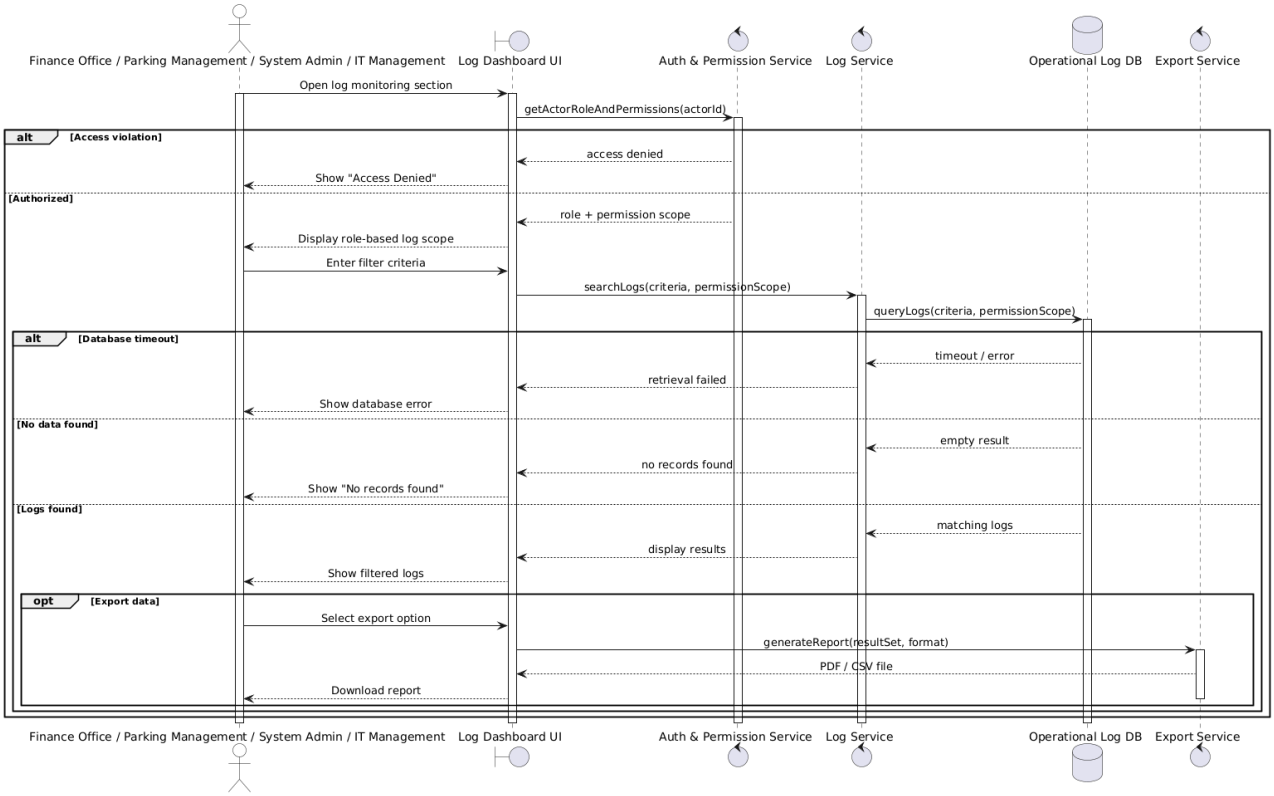

Figure 4.13: Monitor and Review Operational Logs

4.13 UC-13: View Personal Parking & Payment History

This diagram outlines the "View Personal Parking & Payment History" use case. Learners, faculty members, and staff use the self-service portal to review their own parking sessions and payment records. After the user opens the History section, the system authenticates the session through HCMUT_SSO and retrieves the corresponding parking and payment history from the database. The portal displays details such as date, duration, fee, and payment method, and the user may select a specific session to view the digital receipt. If authentication fails, no history is available, or the SSO service times out, the system provides the appropriate notification and logs technical errors when necessary.

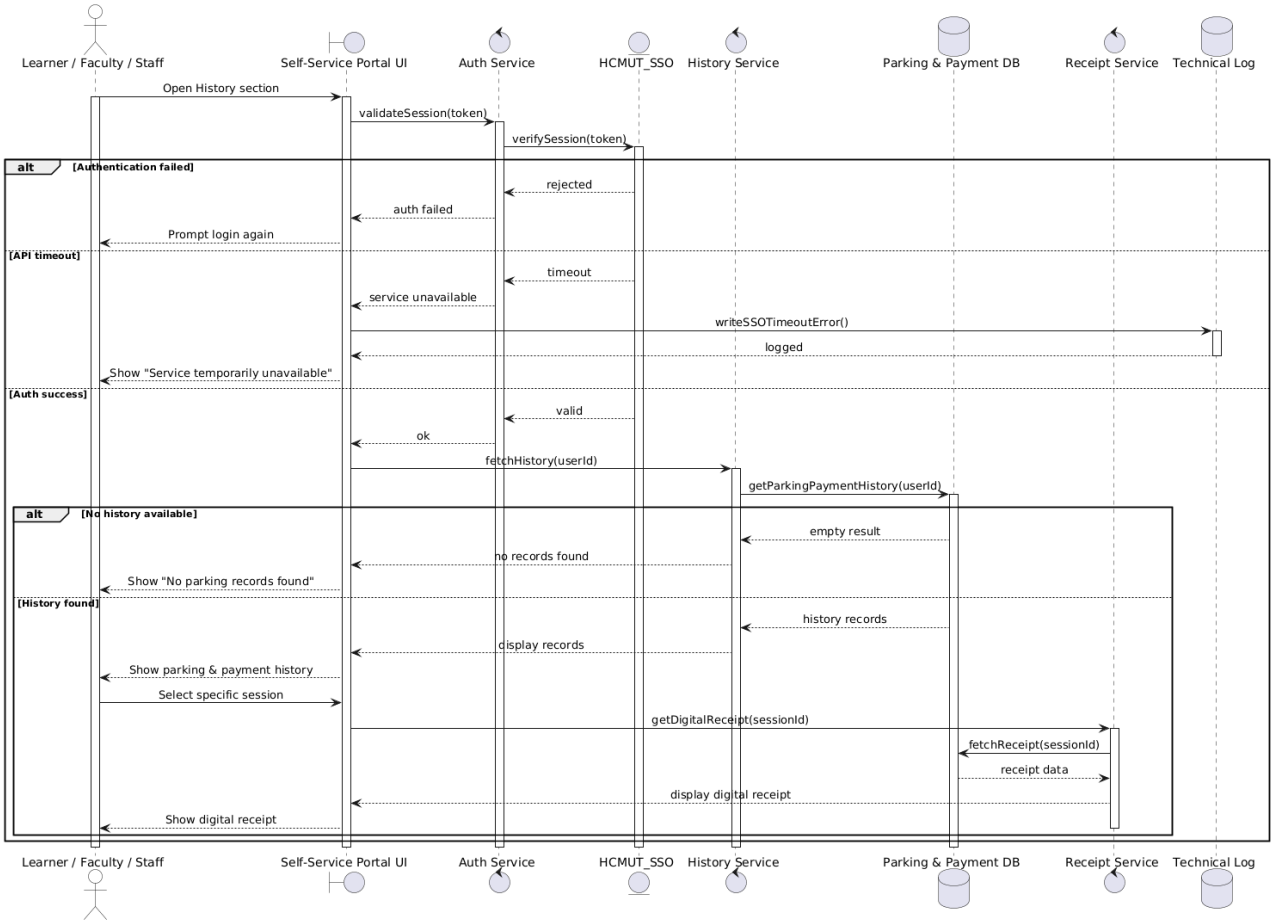

Figure 4.14: View Personal Parking & Payment History

5 Activity Diagram

5.1 UC-01: Enter Parking Lot

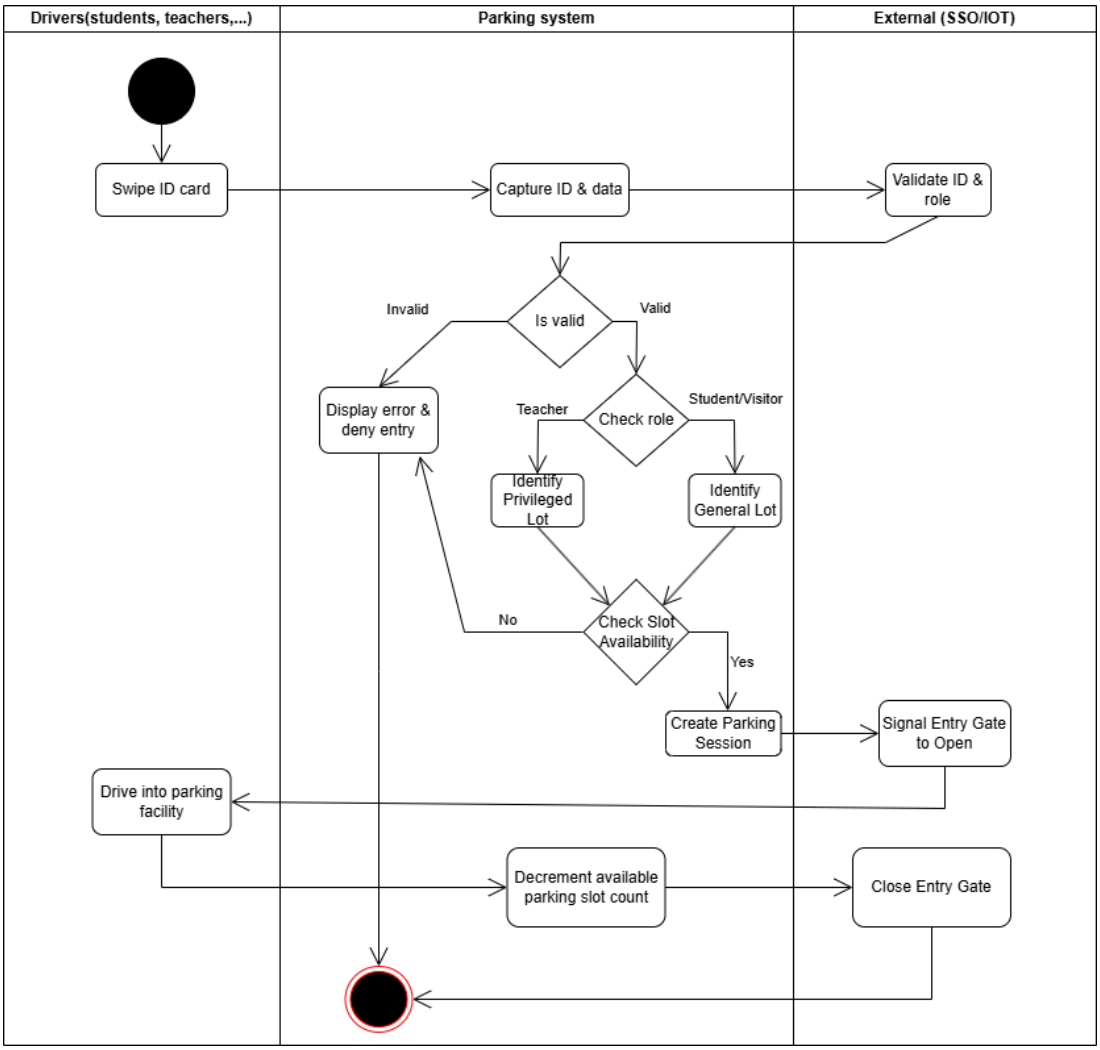

Figure 5.1: Activity Diagram for UC-01: Enter Parking Lot

Description: This diagram illustrates the process of a user entering the parking facility. It highlights the interaction between the Driver, Parking System, and External SSO/IoT services, featuring the validation logic for privileged versus general parking zones and the exception flow for handling invalid ID cards or unauthorized access.

5.2 UC-02: Exit Parking Lot

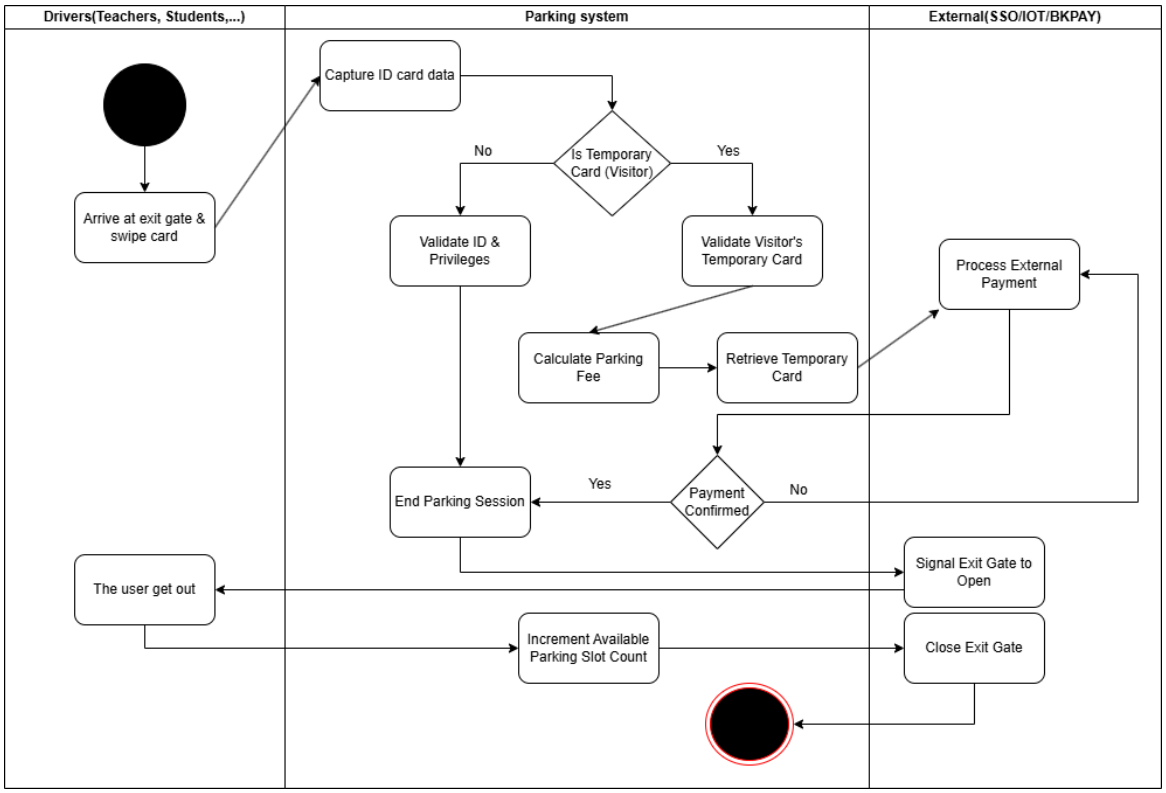

Figure 5.2: Activity Diagram for UC-02: Exit Parking Lot

Description: This diagram demonstrates the vehicle exit process. It details the branching logic for handling internal IDs versus visitor temporary cards, including the steps for fee calculation, temporary card retrieval, and successful payment confirmation before signaling the IoT Gateway to open the exit gate.

5.3 UC-03: Process Payment

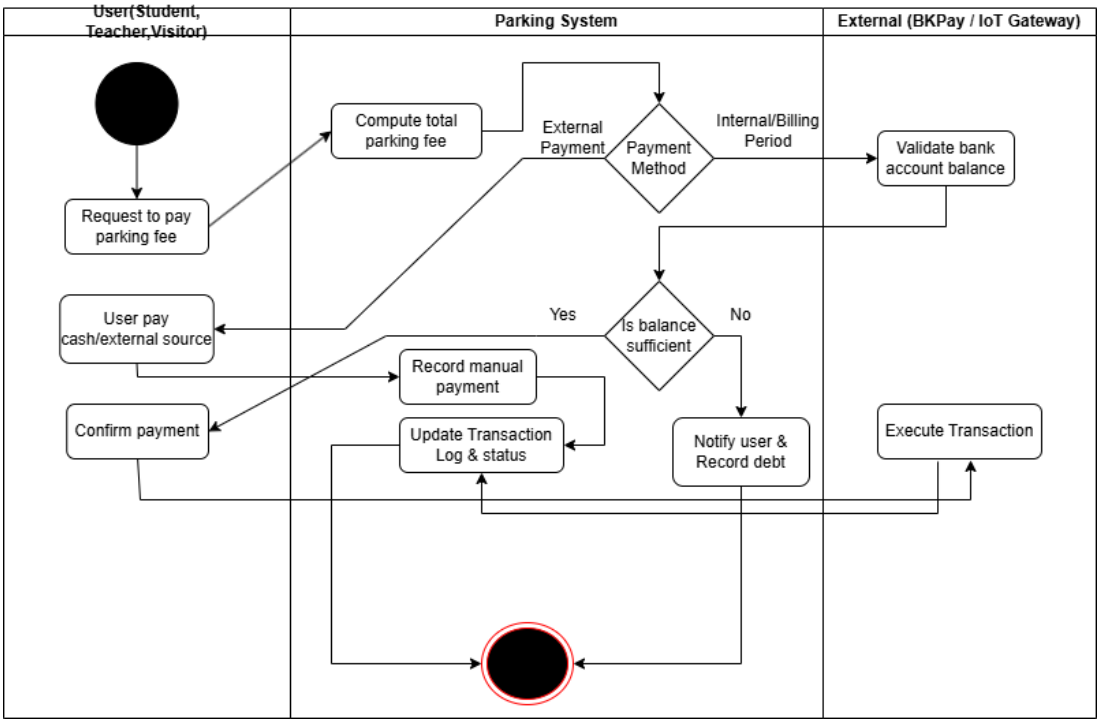

Figure 5.3: Activity Diagram for UC-03: Process Payment

Description: This diagram outlines the payment processing workflow. It involves three swimlanes (User, Parking System, and BKPay) and illustrates the divergent paths for internal billing periods versus external payments, including a crucial exception flow for handling insufficient bank balances by recording a debt before updating the transaction log.

5.4 UC-04: Issue Temporary Card

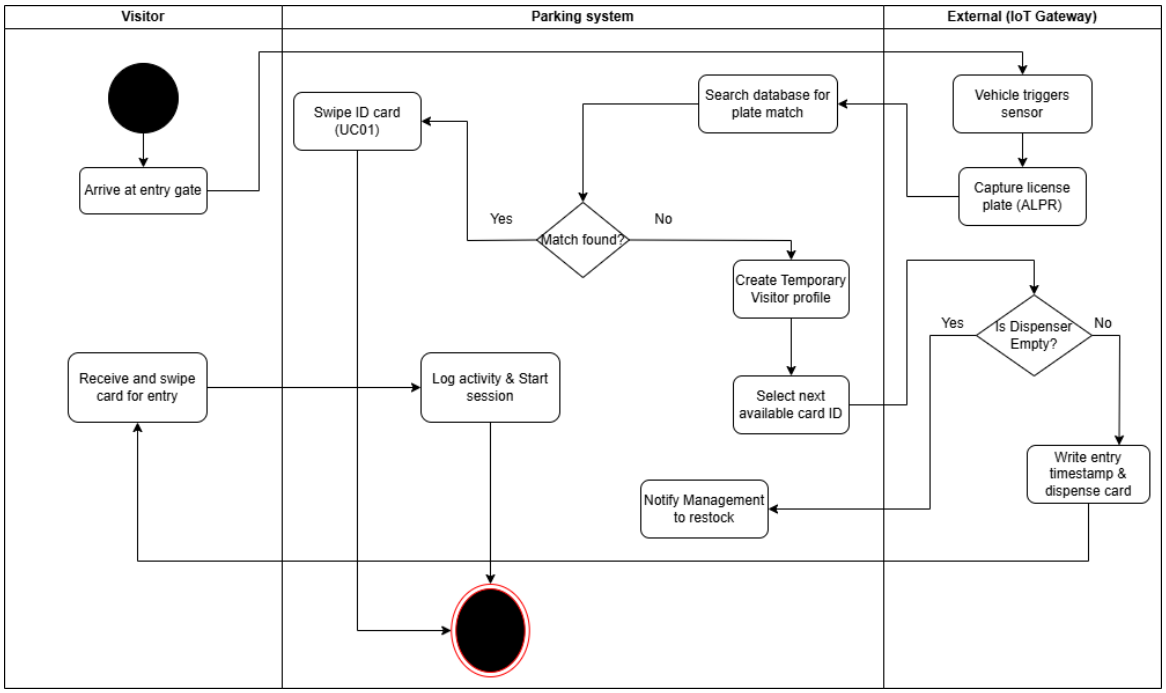

Figure 5.4: Activity Diagram for UC-04: Issue Temporary Card

Description: This diagram maps the automated temporary card issuance process for visitors. It showcases the integration with the IoT Gateway for Automated License Plate Recognition (ALPR) and details the exception handling when the card dispenser is empty, triggering an alert to the management office to restock the physical cards.

5.5 UC-05: Monitor Parking Dashboard

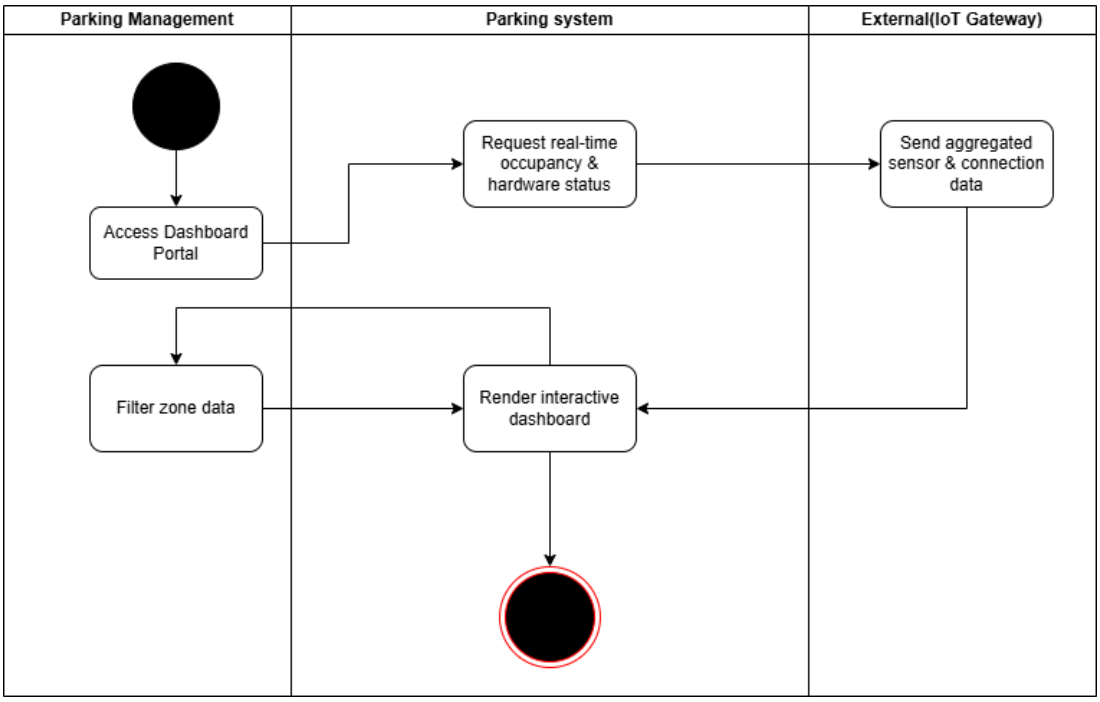

Figure 5.5: Activity Diagram for UC-05: Monitor Parking Dashboard

Description: This diagram depicts the real-time monitoring of the parking system. It visualizes the data retrieval flow from IoT Gateways to the core System and highlights the alternative flow where the Parking Management can apply filters to the interactive dashboard to isolate and analyze specific zone data.

5.6 UC-06: Handle Exception Case

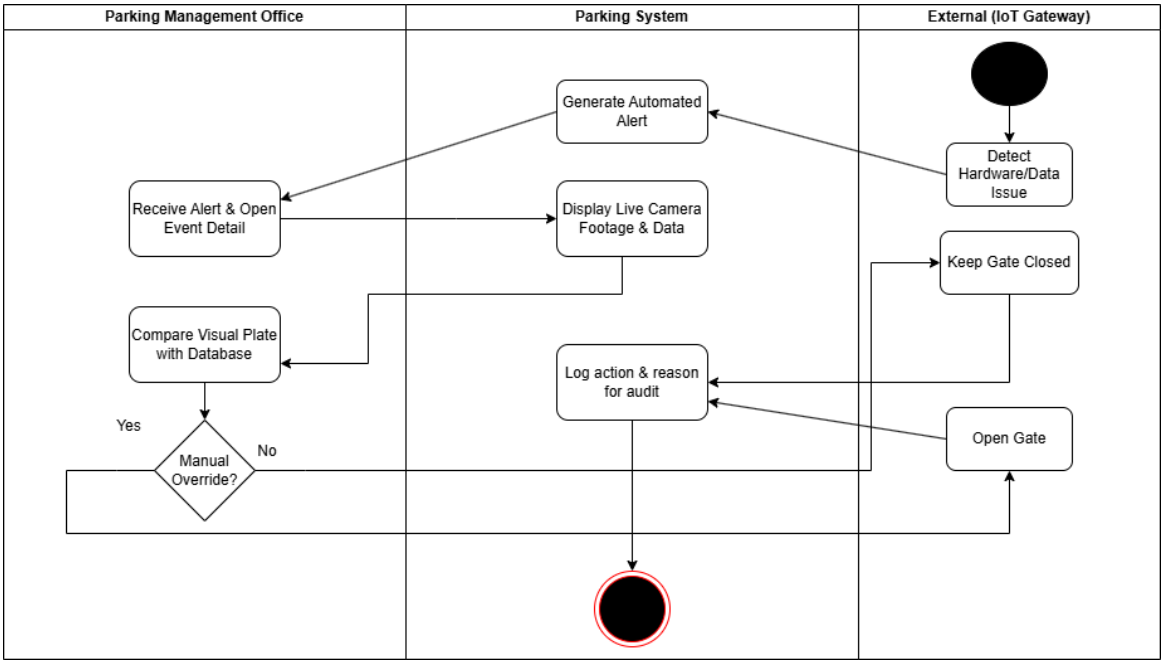

Figure 5.6: Activity Diagram for UC-06: Handle Exception Case

Description: This diagram visualizes the manual exception handling process initiated by hardware or data issues. It captures the automated alert generation, the display of live camera footage for visual cross-referencing, and the critical decision loop for a manual override or access denial, concluding with strict audit logging to ensure system security.

5.7 UC-07: Configure Pricing Policy

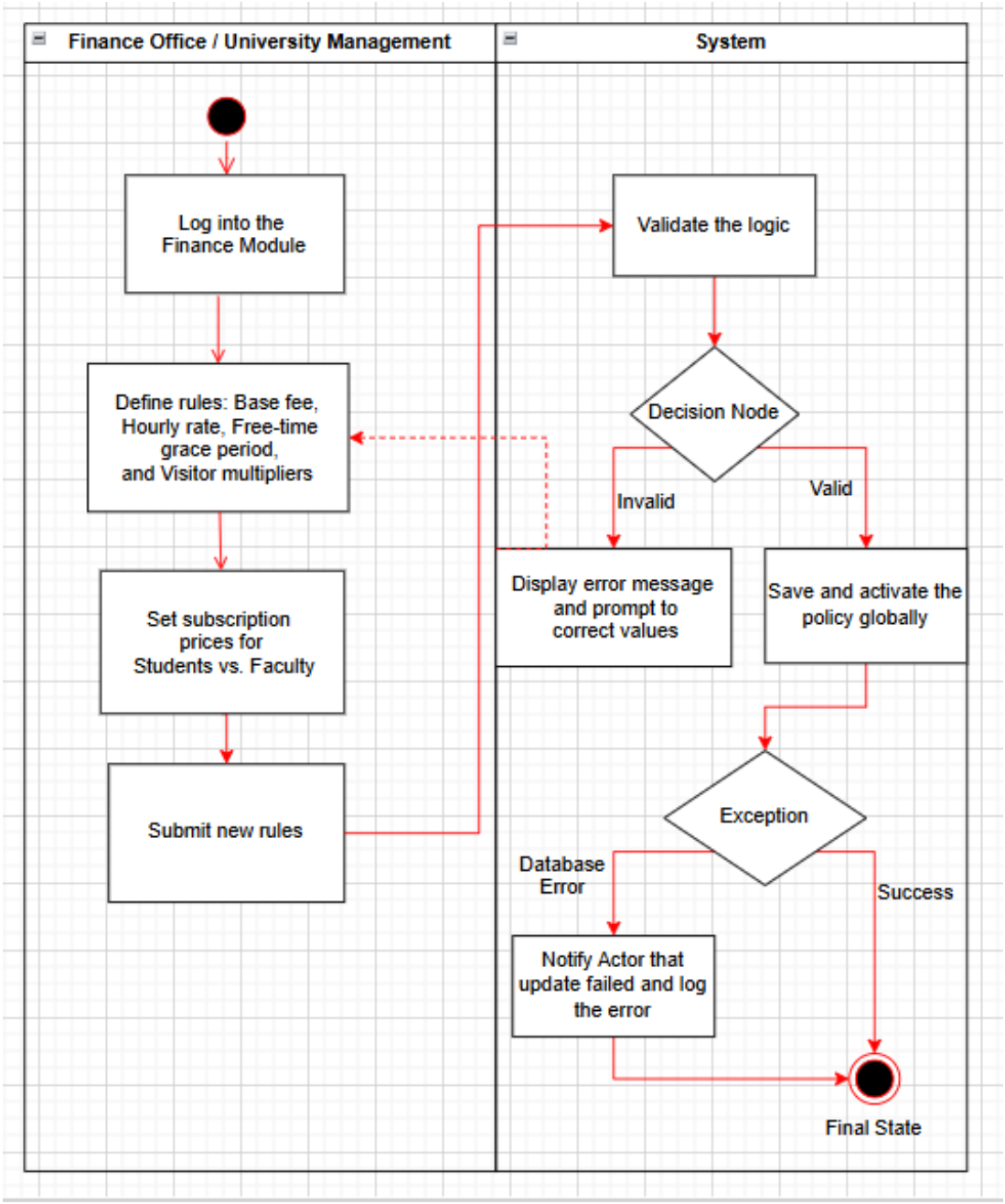

Figure 5.7: Activity Diagram for UC-07: Configure Pricing Policy

Description: This diagram illustrates the flow for configuring the pricing policy. It highlights the interaction between the Finance Office and the System, including the logic validation loop for inputs and the exception handling for database connection errors during the update process.

5.8 UC-08: Issue Refund

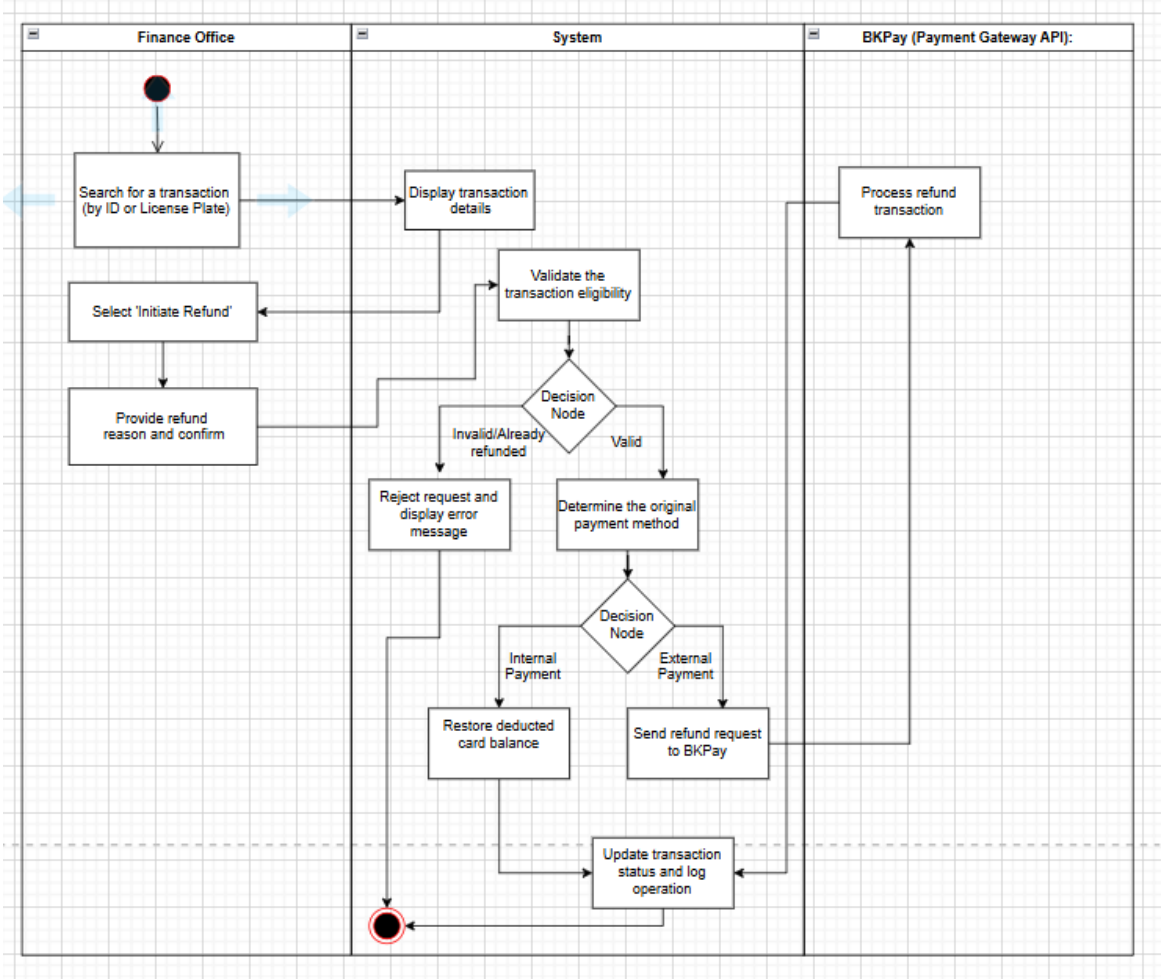

Figure 5.8: Activity Diagram for UC-08: Issue Refund

Description: This diagram demonstrates the refund issuance process. It involves three swimlanes (Finance Office, System, and BKPay) and details the branching logic for handling invalid transactions, as well as the divergent processing paths for internal versus external (BKPay) payment methods before converging to update the final transaction log.

5.9 UC-09: Manage User Roles and Permissions

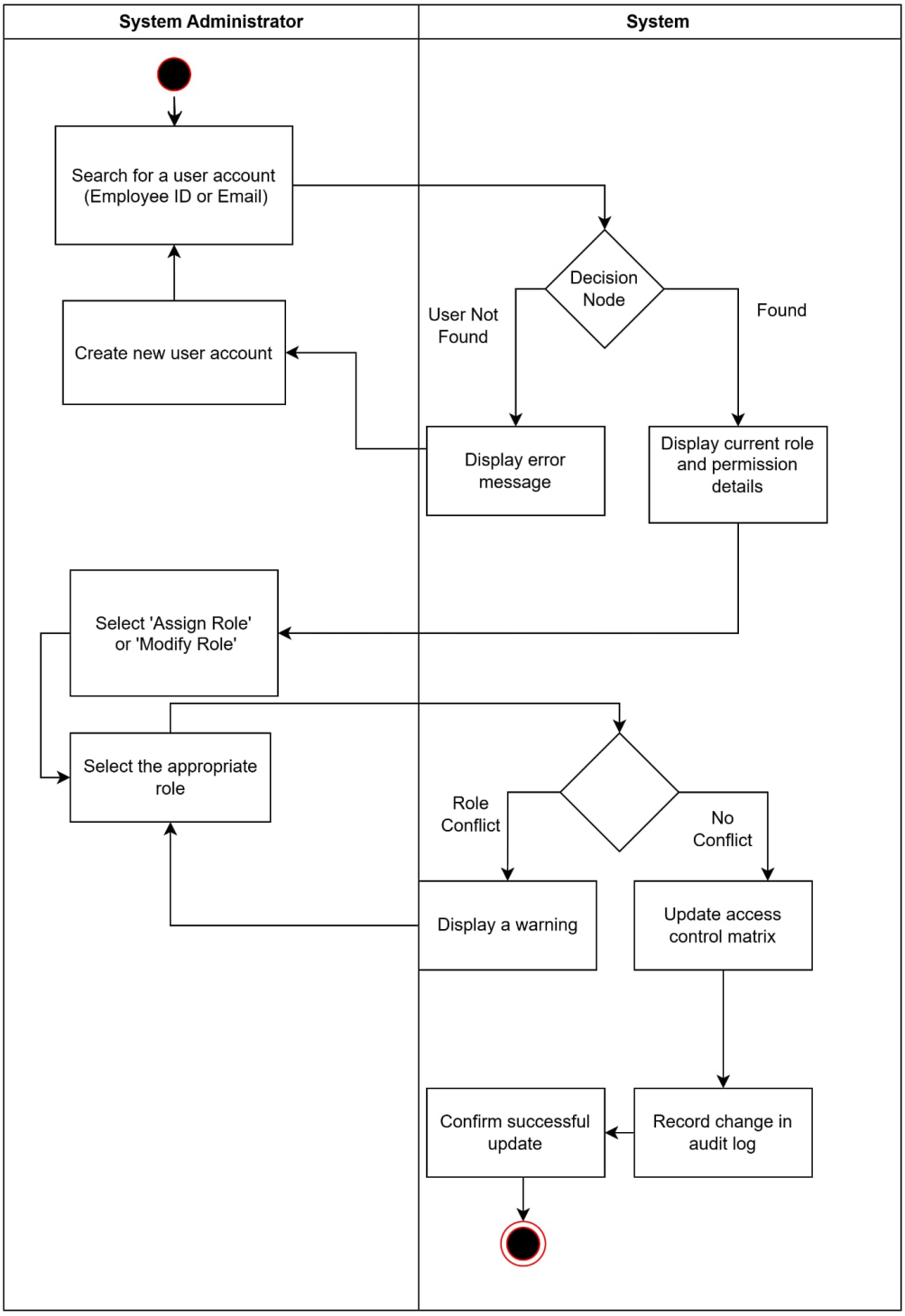

Figure 5.9: Activity Diagram for UC-09: Manage User Roles and Permissions

Description: This diagram outlines the workflow for managing user roles. It features two primary loops: an alternative flow prompting the creation of a new account if the user is not found, and an exception flow that alerts the System Administrator to resolve role conflicts before successfully updating the access control matrix.

5.10 UC-10: Integrate External API / IoT Data Ingestion

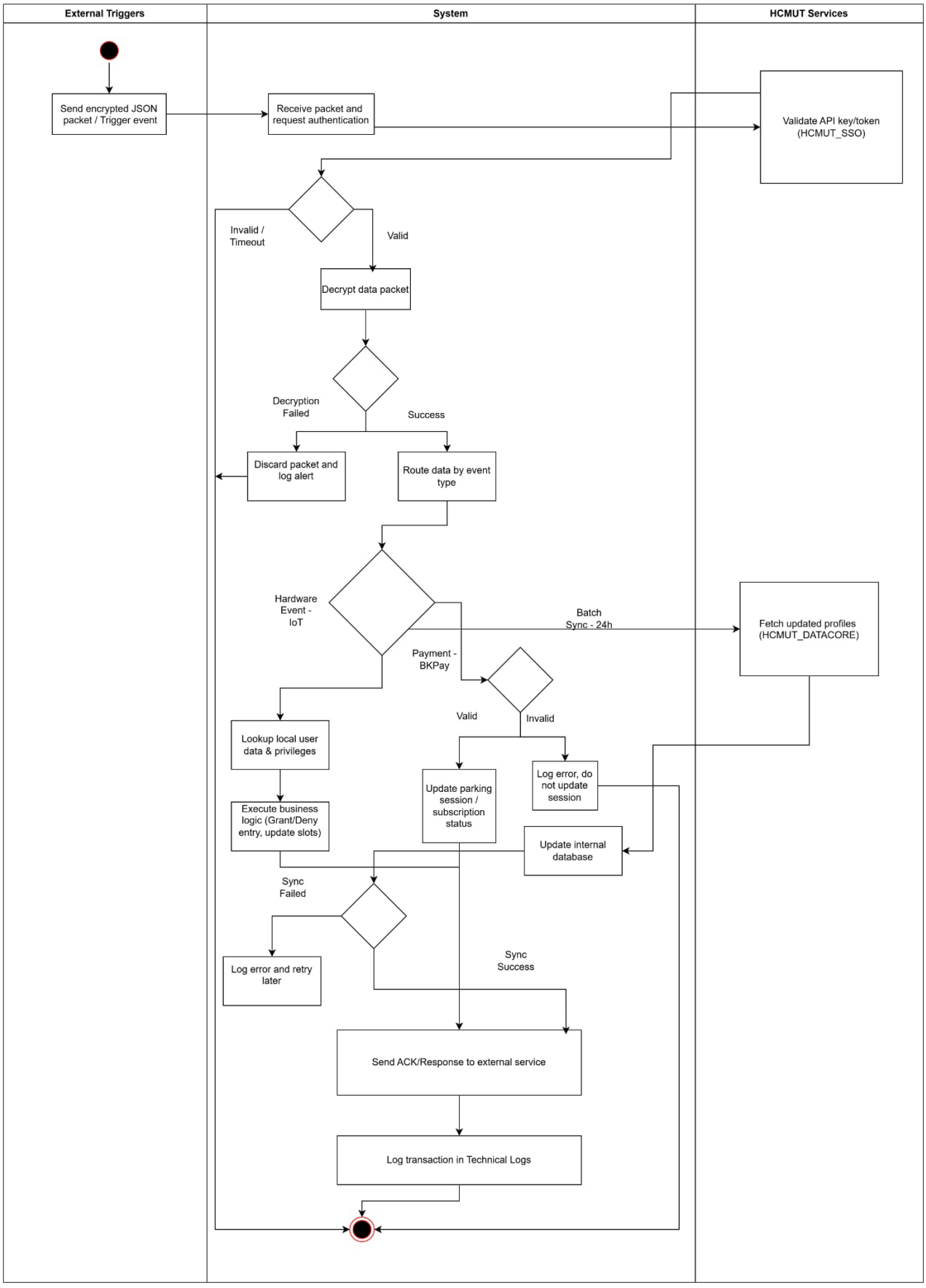

Figure 5.10: Activity Diagram for UC-10: Integrate External API / IoT Data Ingestion

Description: This complex diagram maps the background data ingestion process involving External Triggers, the core System, and HCMUT Services. It details the SSO authentication phase, decryption, and the subsequent routing of data into three distinct processing branches: hardware IoT events, BKPay payment updates, and 24-hour batch synchronizations.

5.11 UC-11: Configure System Parameters and Business Rules

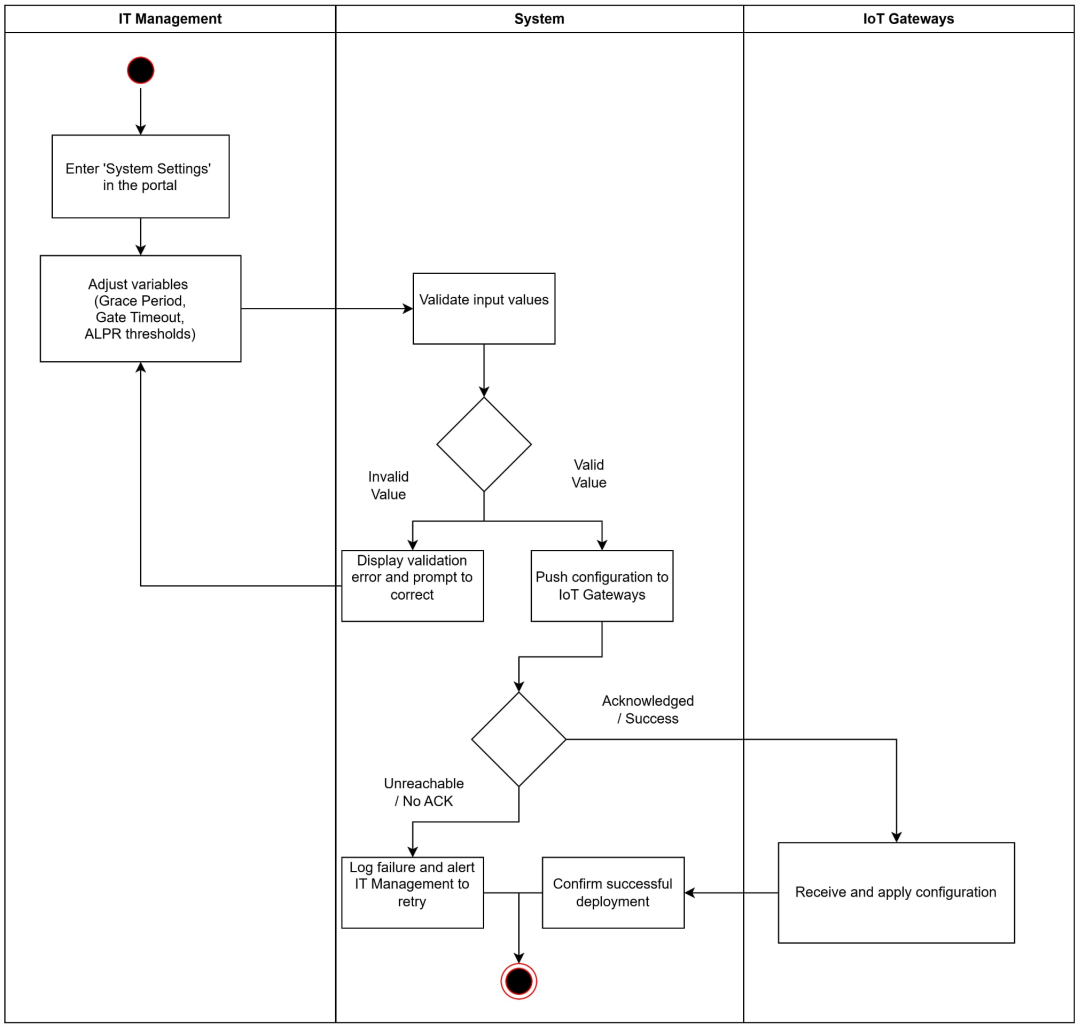

Figure 5.11: Activity Diagram for UC-11: Configure System Parameters and Business Rules

Description: This diagram depicts the configuration of hardware parameters. It showcases the validation of input variables by the IT Management and the deployment flow to the IoT Gateways, including a dedicated exception branch to handle instances where the gateway is unreachable or fails to acknowledge the configuration.

5.12 UC-12: Monitor and Review Operational Logs

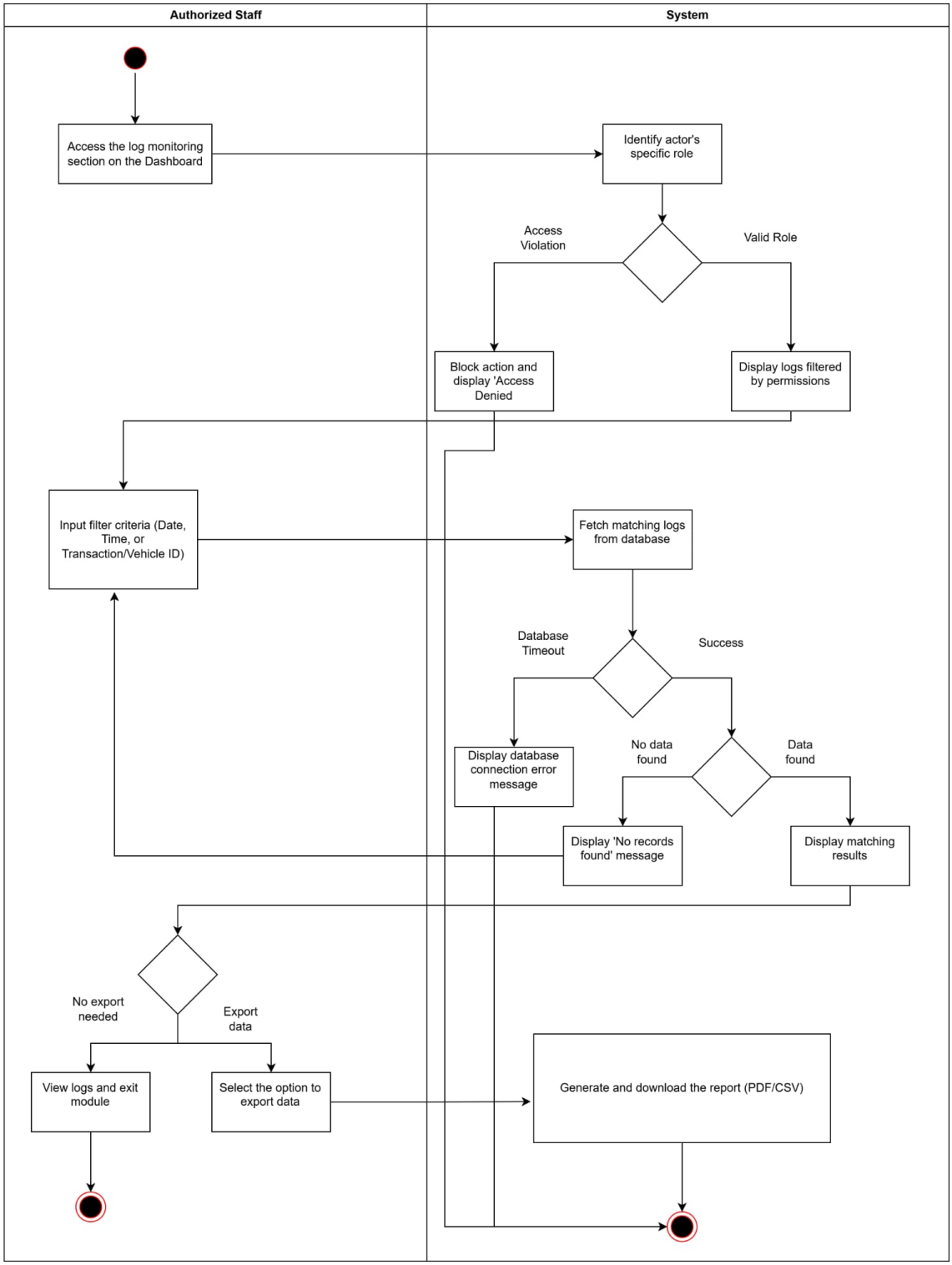

Figure 5.12: Activity Diagram for UC-12: Monitor and Review Operational Logs

Description: This diagram visualizes the operational log monitoring process. It captures the initial role-based access validation, the data fetching logic including handling database timeouts or empty result sets, and the final user decision branch to either view the logs directly or export them to a PDF/CSV file.

5.13 UC-13: View Personal Parking & Payment History

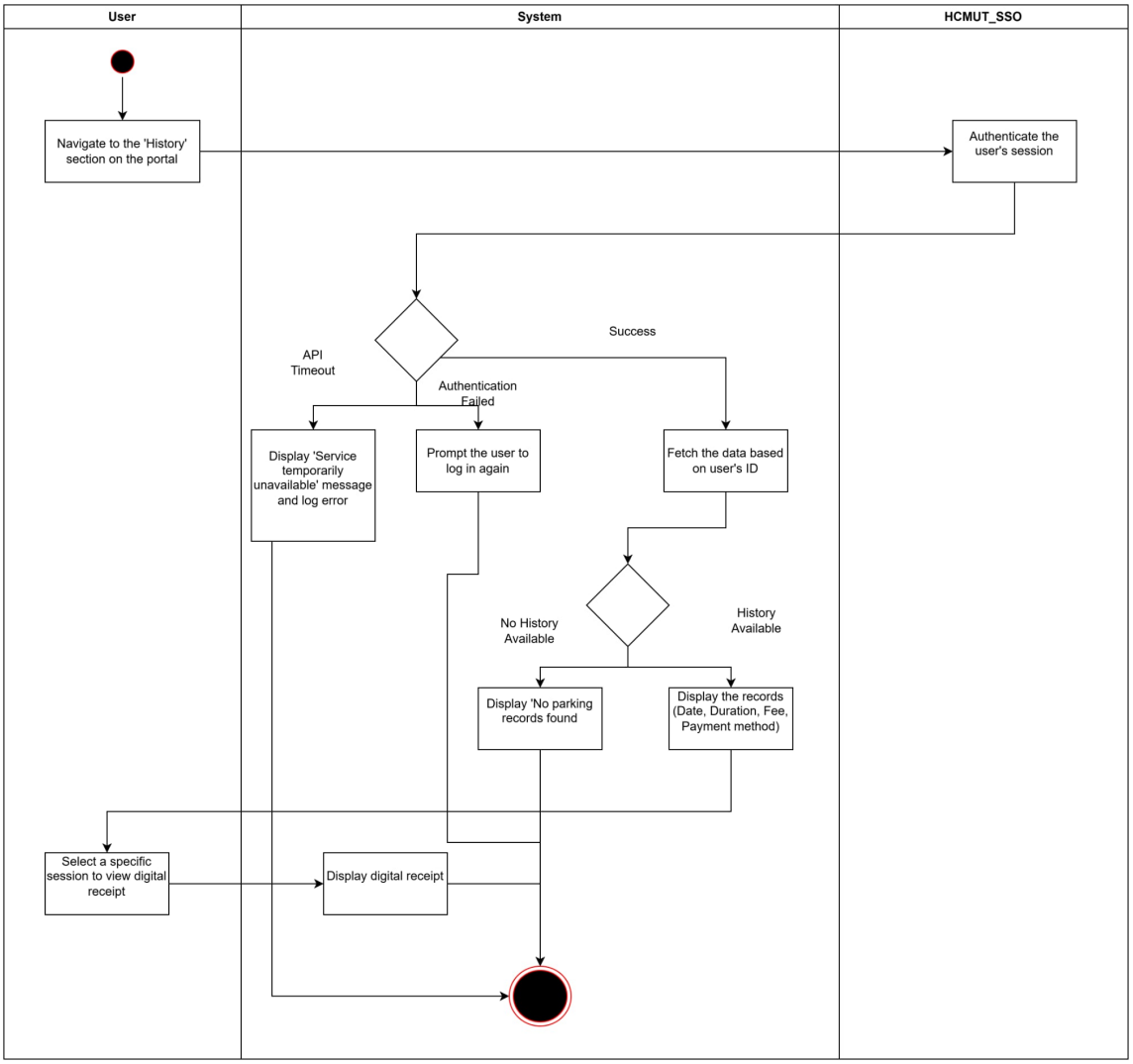

Figure 5.13: Activity Diagram for UC-13: View Personal Parking & Payment History

Description: This diagram shows the process for users to view their parking history. It highlights the integration with the HCMUT_SSO for session authentication, handles exceptions such as API timeouts or failed logins, and illustrates the path for retrieving and displaying both the overall history and specific digital receipts.

6 State-chart Diagram

6.1 Session

The session state represents the lifecycle of a user interaction from entry to exit. It includes active, and pending payment.

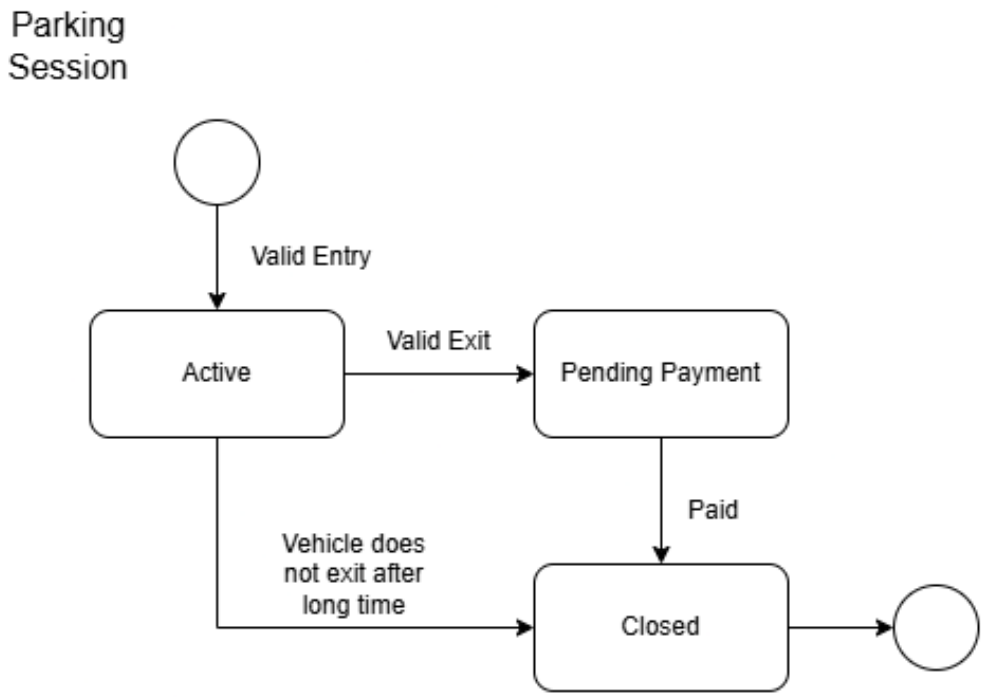

Figure 6.1: Session State-chart Diagram

6.2 Parking Slot

This state describes the availability and occupancy of parking slots. It transitions between states empty, and occupied based on vehicle interactions.

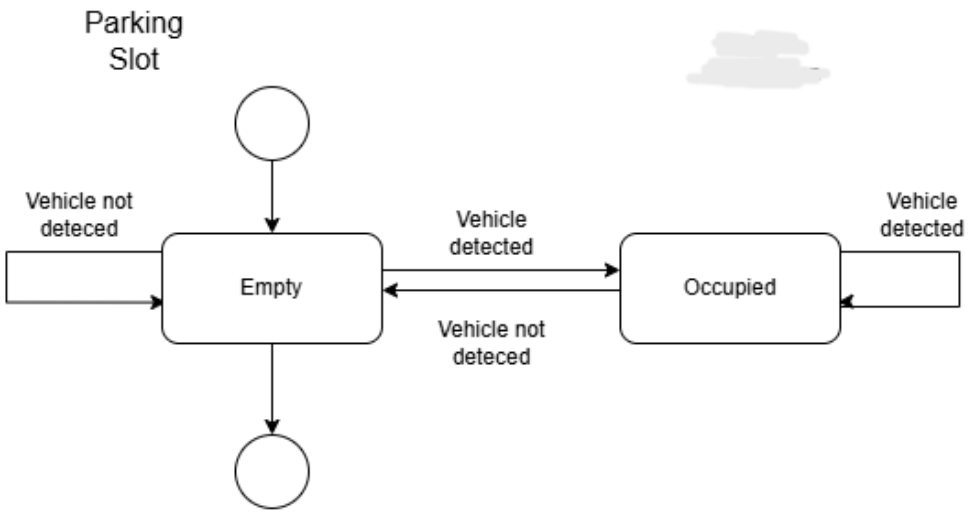

Figure 6.2: Parking Slot State-chart Diagram

6.3 Device

The device state models the behavior of hardware components in the system. It includes states inactive, active, and broken to ensure reliable operation.

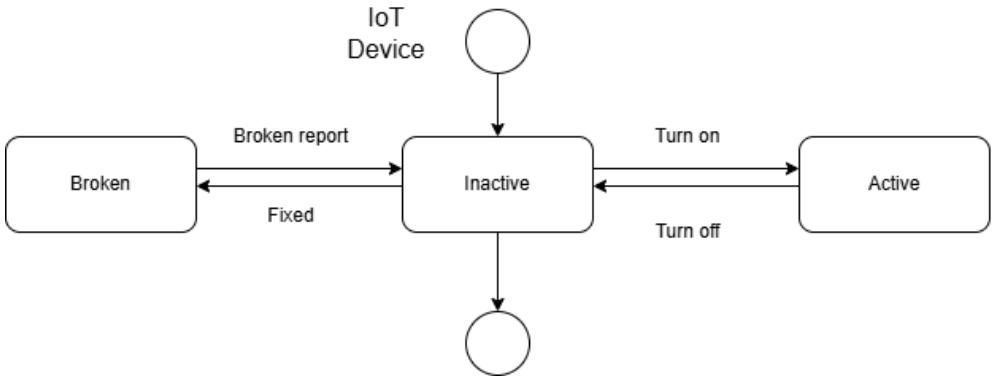

Figure 6.3: Device State-chart Diagram

6.4 Dashboard

The dashboard state represents the system interface used for monitoring and control. It reflects transitions between active, displaying, and closed.

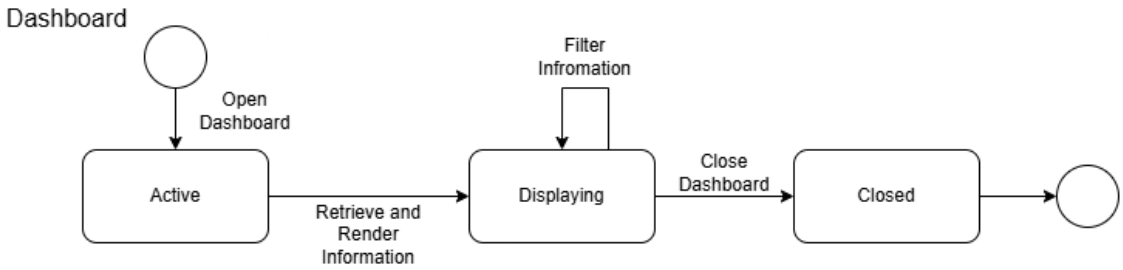

Figure 6.4: Dashboard State-chart Diagram

6.5 Card

This state captures the lifecycle of access cards, including active and in inventory. It ensures secure authentication and authorization.

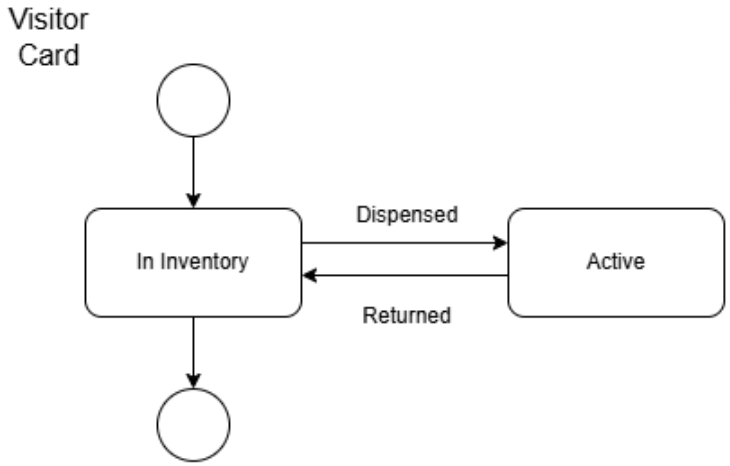

Figure 6.5: Card State-chart Diagram

6.6 Barrier

The barrier state models the opening and closing mechanism of entry/exit gates. It includes transitions closed, opening, open, and closing based on access conditions.

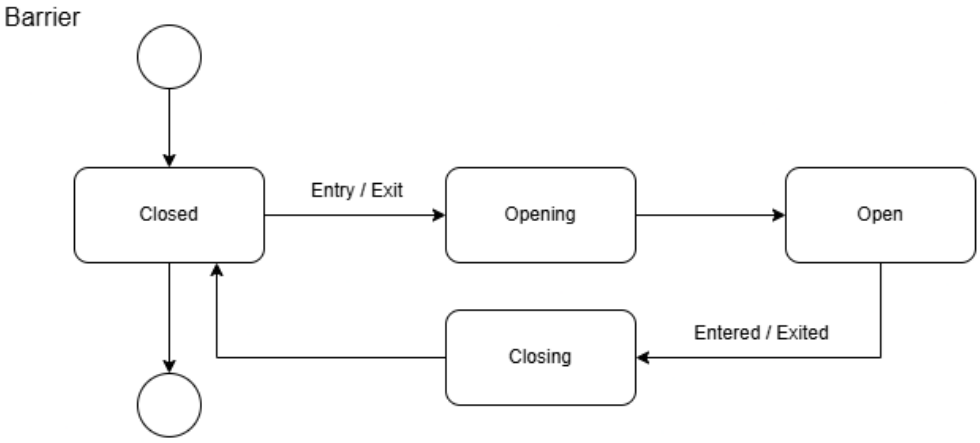

Figure 6.6: Barrier State-chart Diagram

7 UI Design: Mock Up

7.1 Login Page

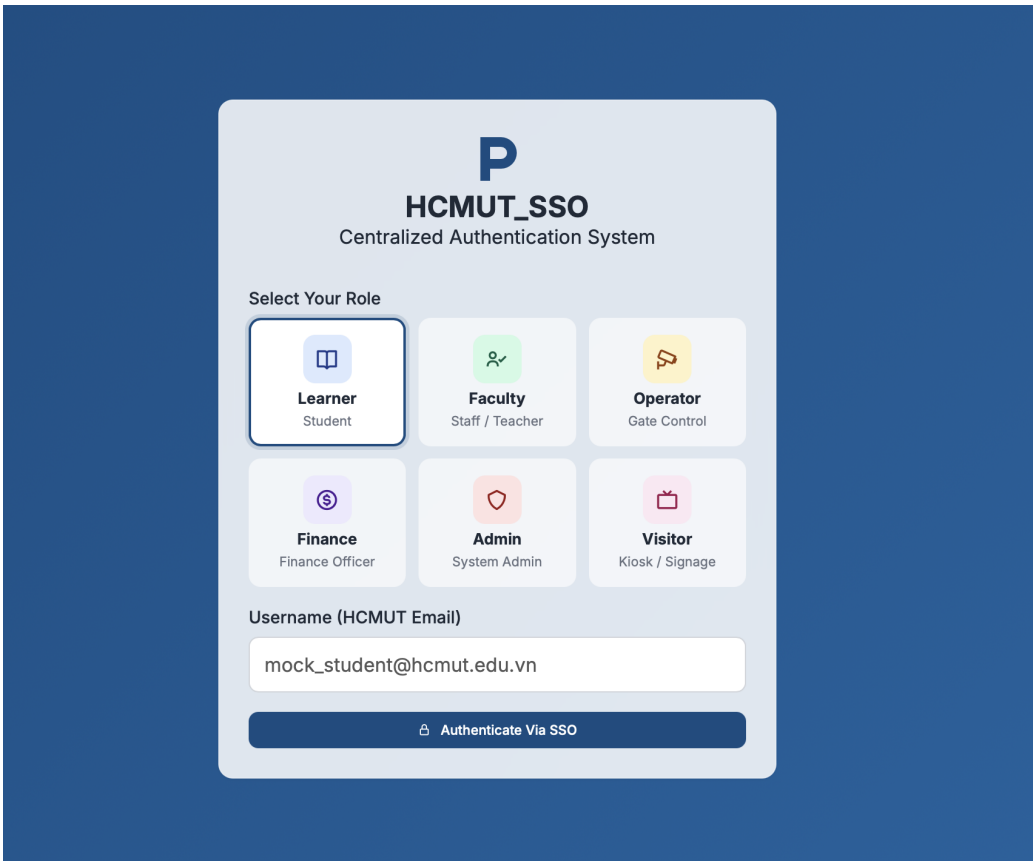

Figure 7.1: UI for login screen

7.1.1 Description

The Login page is the entry point to the system, built on **HCMUT_SSO** (Single Sign-On) — the university's centralized authentication infrastructure. The interface features a card-based layout with a role selection grid of six tiles:

- **Learner** (Student), **Faculty** (Staff / Teacher), **Operator** (Gate Control)
- **Finance** (Finance Officer), **Admin** (System Admin), **Visitor** (Kiosk / Signage)

Below the grid, users enter their HCMUT institutional email and click **"Authenticate Via SSO"** to proceed.

7.1.2 Functionality

- **Role Selection:** The selected role determines the user's dashboard and access privileges after login.
- **SSO Authentication:** Authentication is delegated to HCMUT_SSO, so no separate password is managed by the parking system.
- **Role-Based Access Control:** The authenticated identity combined with the selected role routes each user to the appropriate interface.

7.2 Learner Dashboard

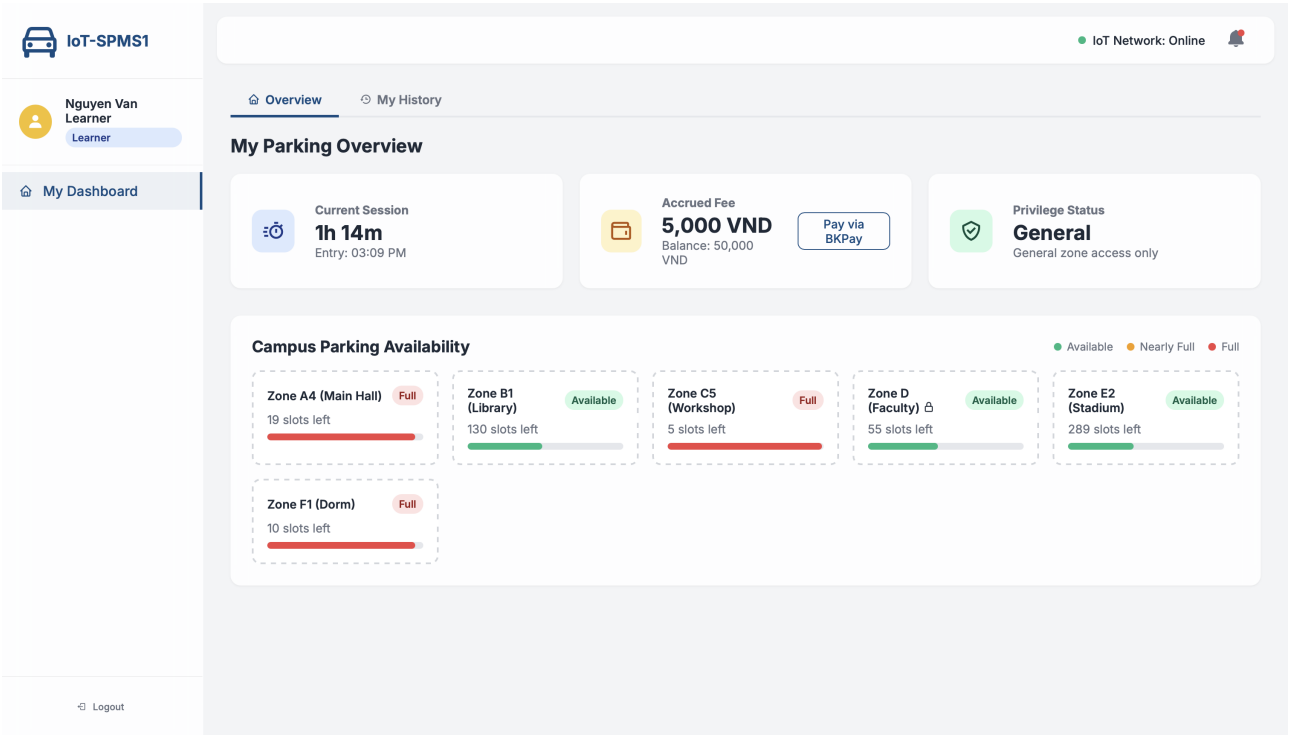

Figure 7.2: UI for student view

7.2.1 Description

The Learner Dashboard provides a real-time overview of the student's parking activity. The top bar displays the system name (**IoT-SPMS1**) and IoT network status. The main content is split into two sections:

- **My Parking Overview:** Three summary cards showing the current session duration and entry time, accrued fee with BKPay payment option, and the user's privilege status (e.g., General — general zone access only).
- **Campus Parking Availability:** A grid of zone cards (A4, B1, C5, D, E2, F1) each showing remaining slots and a color-coded fill bar. Zones are tagged *Available*, *Nearly Full*, or *Full*. Restricted zones (e.g., Zone D — Faculty) display a lock icon.

7.2.2 Functionality

- **Live Session Tracking:** Displays elapsed parking time and entry timestamp, updated in real time via IoT sensors.
- **Fee Monitoring & Payment:** Shows the accrued fee against the user's BKPay balance, with a direct *Pay via BKPay* button for in-session payment.
- **Zone Availability:** Slot counts and progress bars update live, helping students identify available zones before moving their vehicle.
- **Access Control Awareness:** Restricted zones are visually marked, reflecting the user's privilege level and preventing unauthorized entry attempts.

7.3 Zone Spot Map

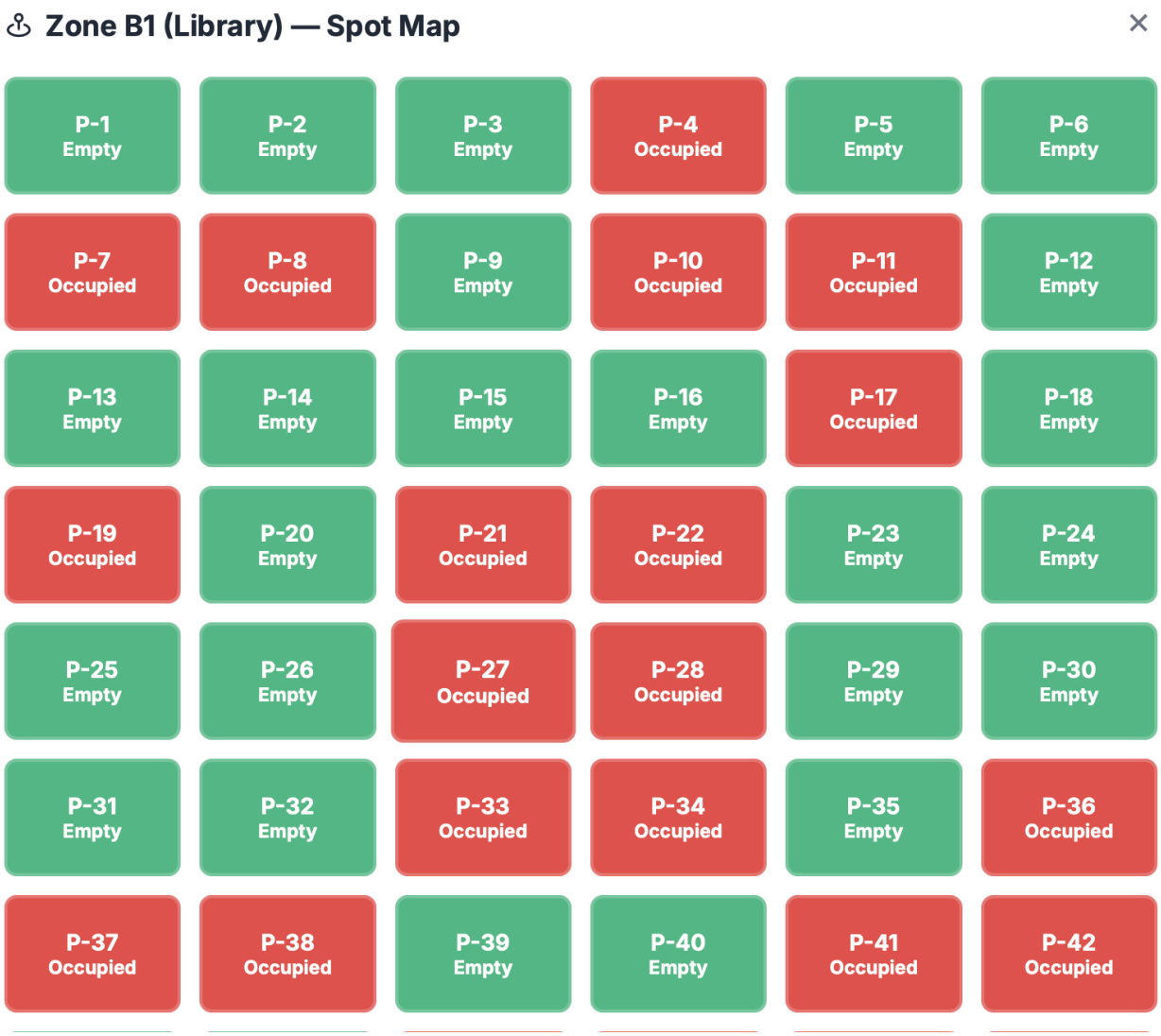

Figure 7.3: UI for spotmap view

7.3.1 Description

A modal overlay showing the individual spot-level layout of a selected zone (e.g., **Zone B1 — Library**). Each spot is displayed as a color-coded tile: green for Empty and red for Occupied, labeled with its spot ID (P-1 through P-42).

7.3.2 Functionality

- **Real-Time Spot Status:** Tile colors update live via IoT sensors, allowing users to locate an available spot before entering the zone.
- **Spot Identification:** Each tile displays a unique ID, enabling precise navigation within the parking area.

7.4 Parking & Payment History

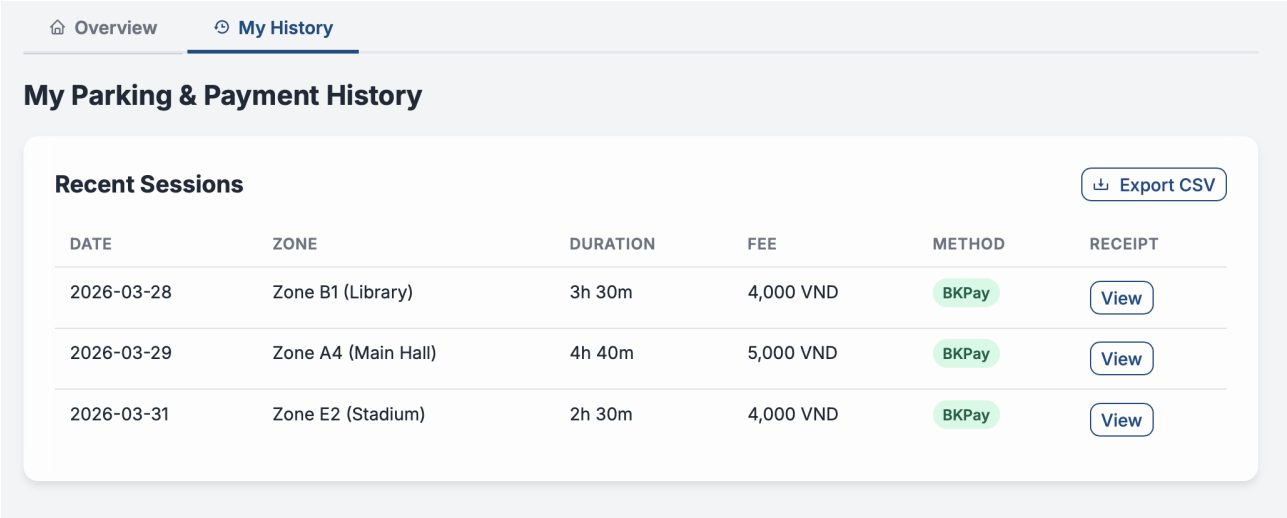

Figure 7.4: UI for payment history of student view

7.4.1 Description

A table listing the user's recent parking sessions, with columns for Date, Zone, Duration, Fee, Payment Method, and a Receipt link. Sessions can be exported via **Export CSV**.

7.4.2 Functionality

- **Session Log:** Displays per-session records pulled from the system database.
- **Receipt Access:** Each row has a *View* button to open the individual payment receipt.
- **CSV Export:** Allows users to download their full history for personal records.

7.5 Faculty/Staff Dashboard

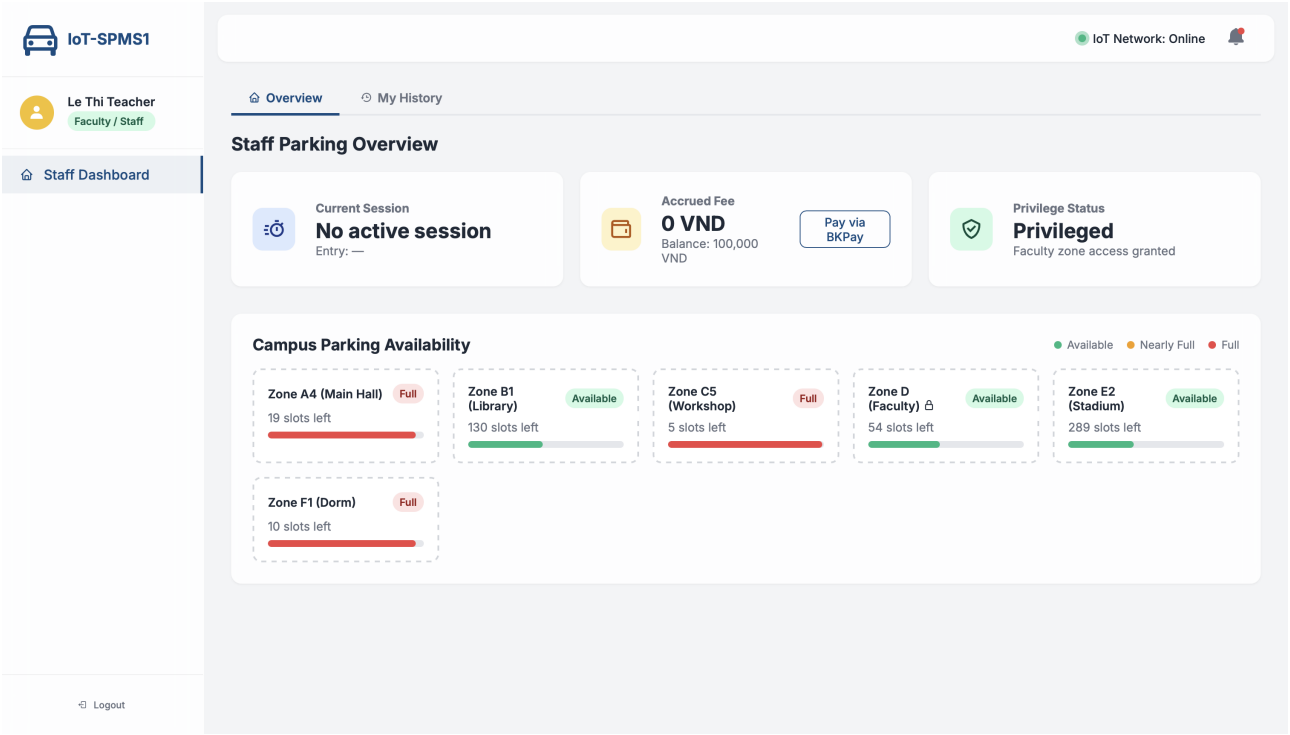

Figure 7.5: UI for faculty/staff view

7.5.1 Description

Identical in layout to the Learner Dashboard, but reflects Faculty/Staff privileges. Key differences: the privilege card shows **Privileged** status with *Faculty zone access granted*, and Zone D (Faculty) is accessible without a restriction indicator.

7.5.2 Functionality

Same as the Learner Dashboard, with one distinction:

- **Elevated Access:** Faculty/Staff users can access restricted zones (e.g., Zone D), which are locked for Learners.

7.6 Operator (Gate Control) Dashboard

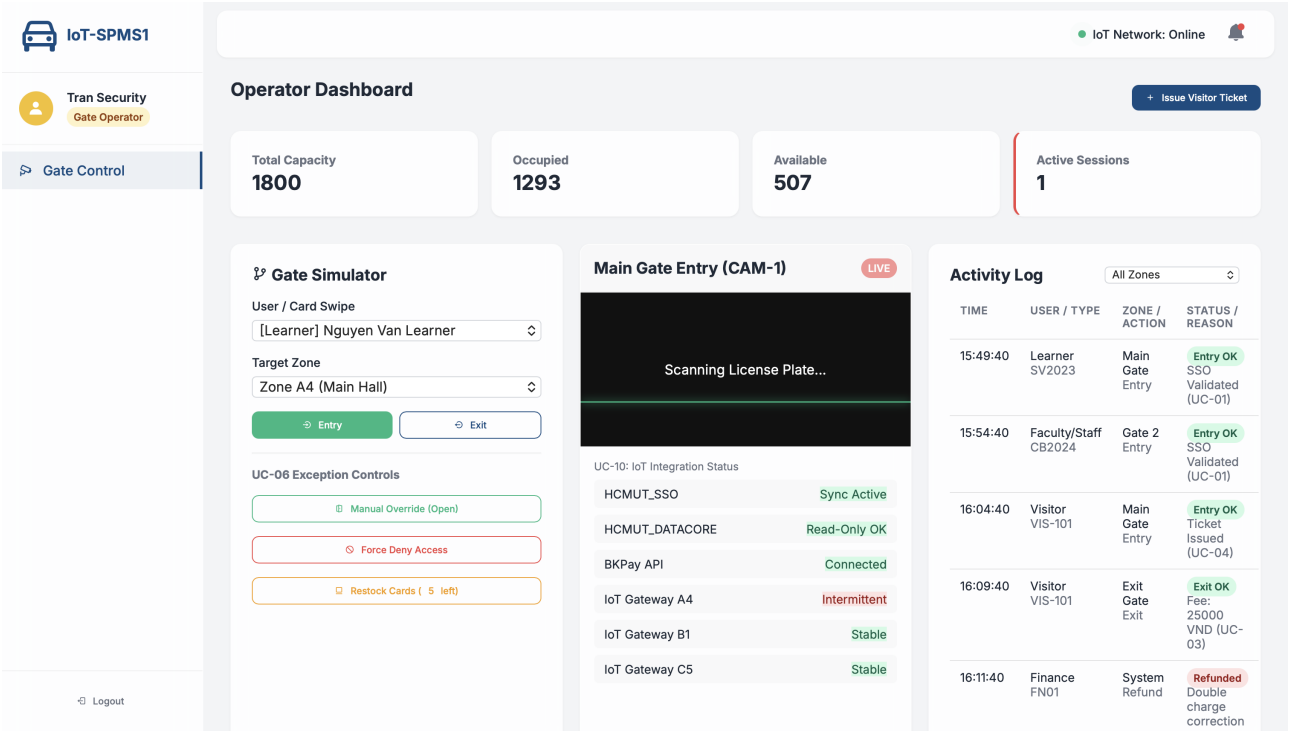

Figure 7.6: UI for gate operator view

7.6.1 Description

The Operator Dashboard provides gate management tools across three panels: a capacity summary bar (Total/Occupied/Available/Active Sessions), a **Gate Simulator** for manual entry/exit control, a live **CAM-1** feed with IoT integration status, and an **Activity Log** of recent gate events.

7.6.2 Functionality

- **Gate Simulator:** Operators select a user and target zone to manually trigger Entry or Exit, simulating card swipe events.
- **Exception Controls:** Buttons for Manual Override (open), Force Deny Access, and Restock Cards handle edge cases and hardware issues.
- **Live Camera & IoT Status:** CAM-1 streams live license plate scanning; the panel also shows connectivity status of HCMUT_SSO, BKPay API, and per-zone IoT gateways.
- **Activity Log:** A real-time log of all gate events (entry, exit, refund) filterable by zone, showing user, action, and status reason.
- **Visitor Ticket:** The *Issue Visitor Ticket* button generates a temporary access ticket for non-SSO users.

7.7 Finance Office Dashboard

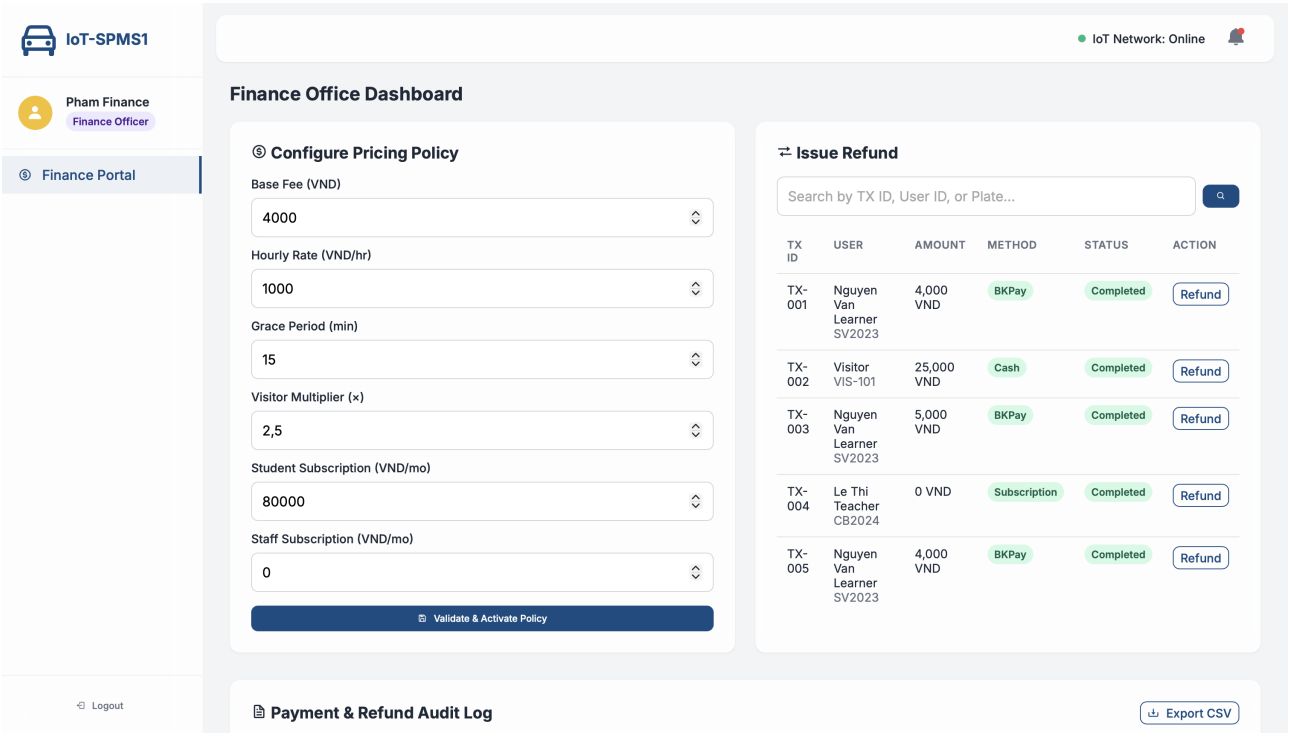

Figure 7.7: UI for finance office view

7.7.1 Description

Split into two panels: **Configure Pricing Policy** (base fee, hourly rate, grace period, visitor multiplier, and subscription rates) and **Issue Refund** (a searchable transaction table with TX ID, user, amount, method, and status). A **Payment & Refund Audit Log** with CSV export sits below.

7.7.2 Functionality

- **Pricing Configuration:** Finance officers set and activate system-wide fee parameters via *Validate & Activate Policy*.
- **Refund Management:** Transactions can be searched by TX ID, User ID, or license plate, and refunded individually.
- **Audit Log:** Full payment and refund history is available for review and export.

7.8 System Administration Dashboard

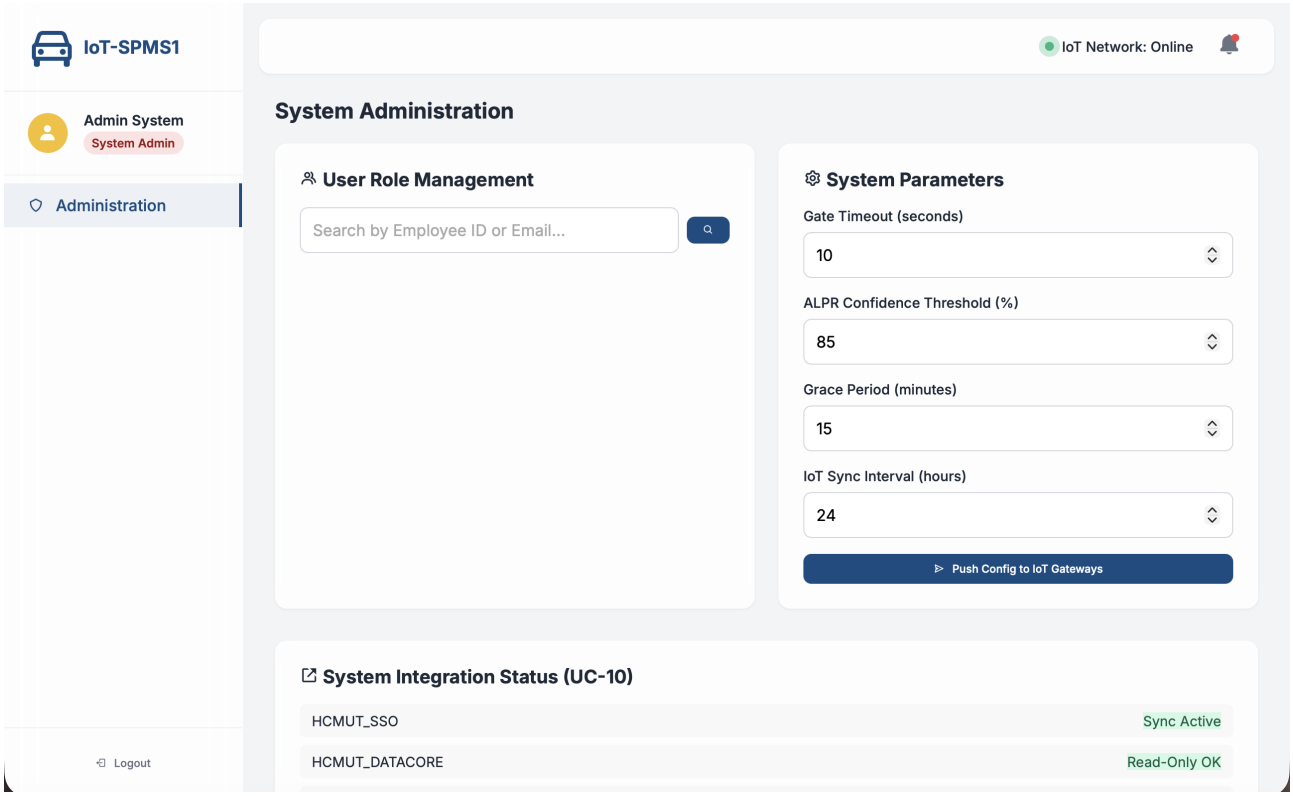

Figure 7.8: UI for administrator dashboard view

7.8.1 Description

Two panels: **User Role Management** (search users by Employee ID or email) and **System Parameters** (Gate Timeout, ALPR Confidence Threshold, Grace Period, IoT Sync Interval). A **System Integration Status** section below shows live connectivity of external services (HCMUT_SSO, HCMUT_DATACORE, etc.).

7.8.2 Functionality

- **Role Management:** Admins can look up and modify user roles and permissions system-wide.
- **Parameter Configuration:** Core system parameters are adjustable and pushed to all IoT gateways via *Push Config to IoT Gateways*.
- **Integration Monitoring:** Real-time status of all external service connections for system health oversight.

7.9 Administrator Operational Overview

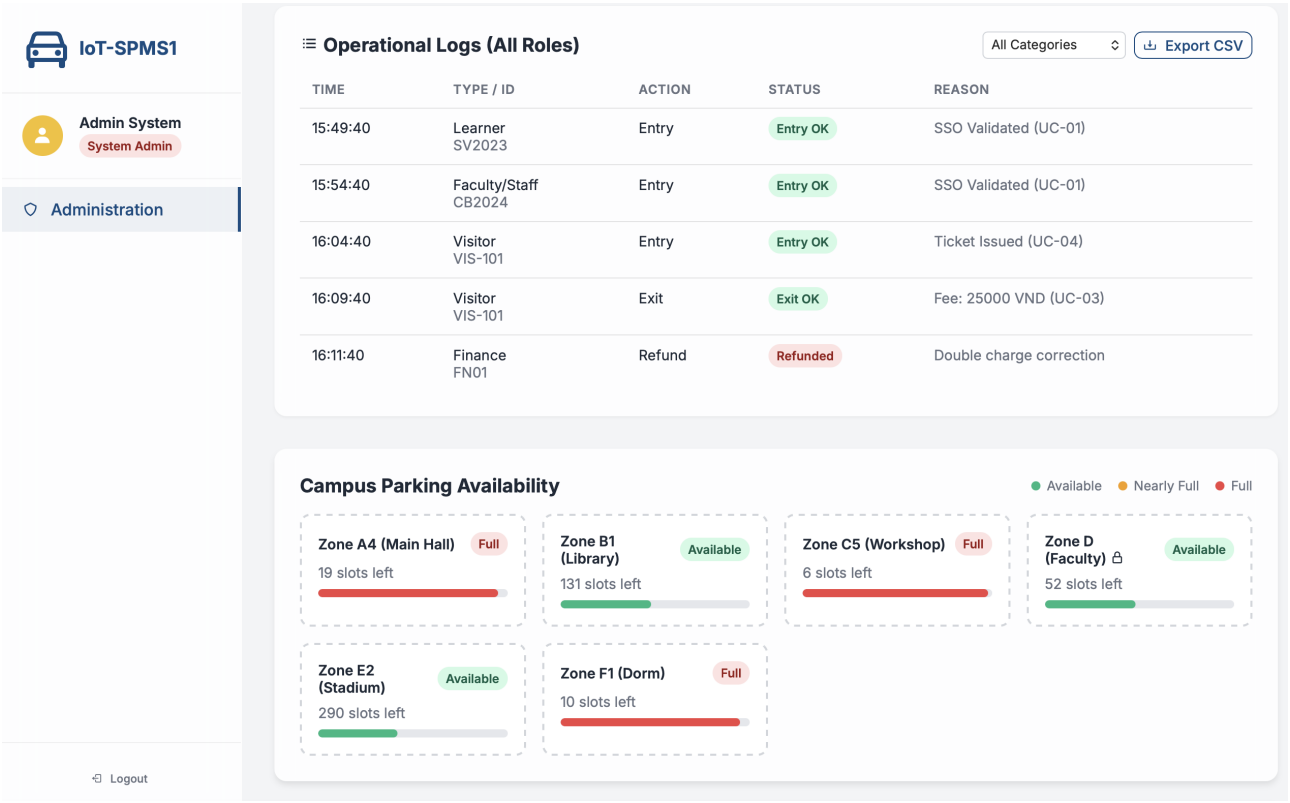

Figure 7.9: UI for admininistrator operational overview

7.9.1 Description

A scrollable admin view combining an **Operational Logs** table (Time, Type/ID, Action, Status, Reason) filterable by category with CSV export, and the full **Campus Parking Availability** grid below.

7.9.2 Functionality

- **System-Wide Logs:** Aggregates all role actions (entry, exit, refund) across the entire system for audit purposes.
- **Parking Overview:** Same live zone availability as user dashboards, giving admins a unified operational picture.

7.10 Visitor Kiosk Display

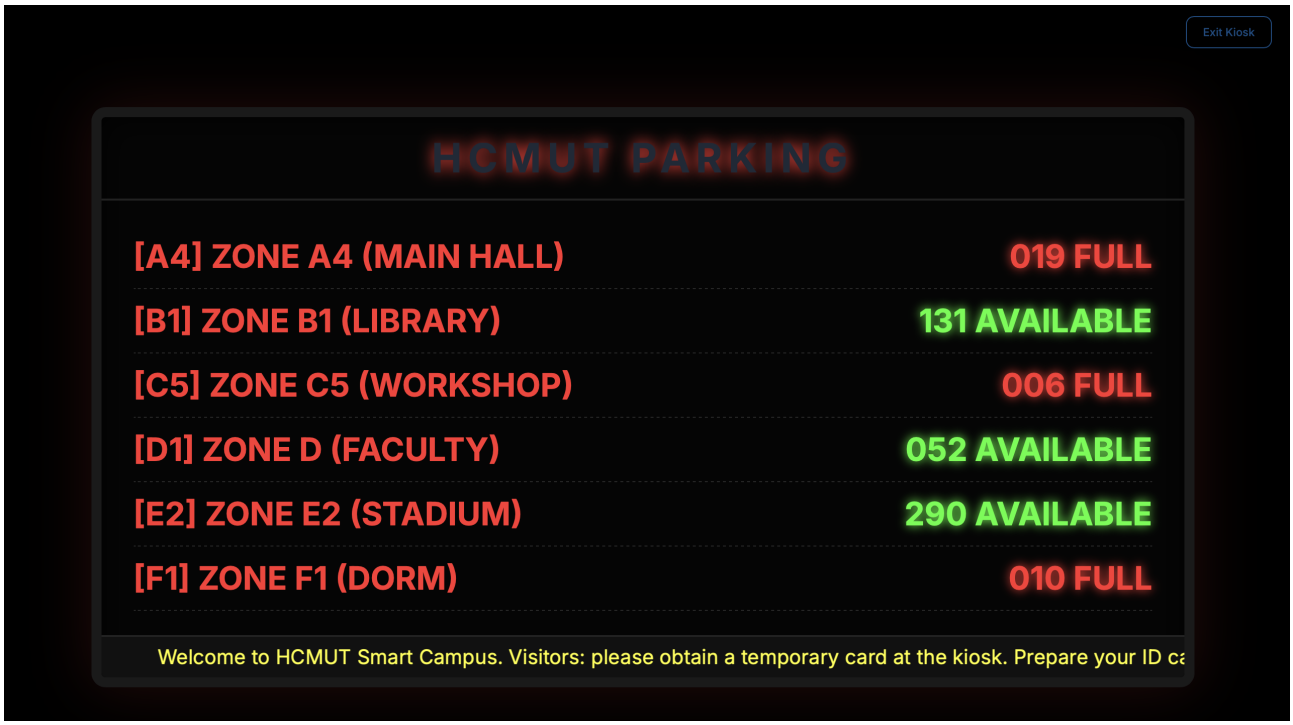

Figure 7.10: UI for visitor kiosk display

7.10.1 Description

A fullscreen signage display with a dark background and neon-style typography, listing all parking zones with their remaining slot counts in **green** (Available) or **red** (Full). A scrolling ticker at the bottom instructs visitors to obtain a temporary card at the kiosk.

7.10.2 Functionality

- **Public Availability Display:** Shows real-time slot counts for all zones, intended for physical signage at campus entrances.
- **Visitor Guidance:** The ticker directs unregistered visitors to the kiosk for temporary card issuance.
- **Kiosk Exit:** An *Exit Kiosk* button in the corner allows the operator to return to the admin interface.

8 Deployment View

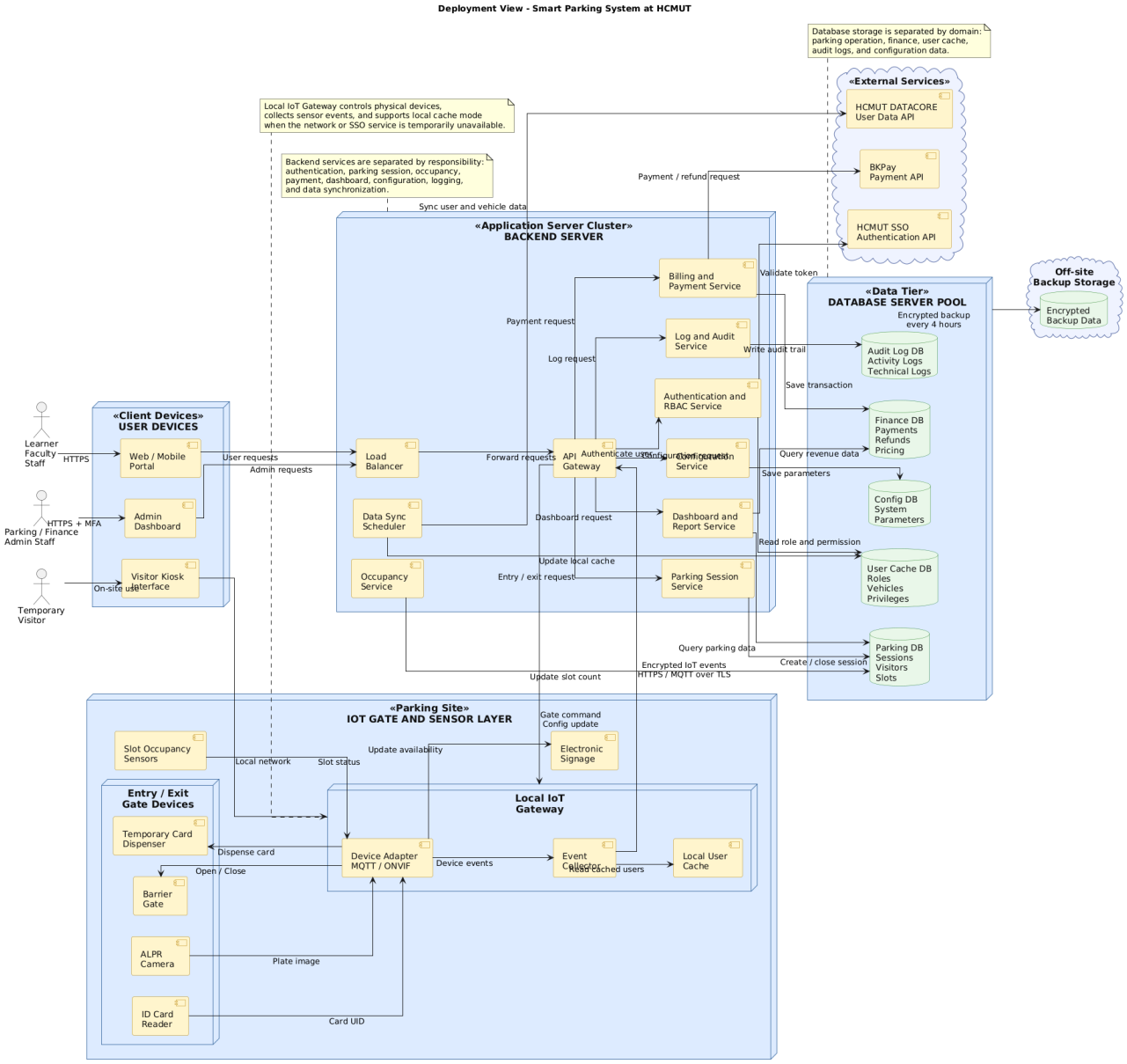

Figure 8.1: Deployment View Diagram of Smart Parking System

8.1 Overview

The Deployment View illustrates a robust, modular, and validated architecture for the Smart Parking System at HCMUT. The system follows a layered deployment structure consisting of Client Devices, an IoT Gate and Sensor Layer, an Application Server Cluster, a Data Tier, and integrated External Services. This architecture supports real-time parking operations, automated gate control, secure authentication, payment processing, operational monitoring, and reliable data persistence.

The design emphasizes separation of responsibilities by assigning each deployment node a clear role. Client devices handle user interaction, IoT gateways manage communication with physical parking devices, backend services execute business logic, database servers maintain persistent data, and external services provide authentication, payment, and university data synchronization.

8.2 Key Architectural Components and Alignment

The system is compartmentalized into several logical deployment environments: Client Devices, IoT Gate and Sensor Layer, Application Server Cluster, Data Tier, External Services, and Backup Storage.

1. **Client Devices:**
    - **User Access:** Internal users such as learners, faculty, and staff access the system through the `Web/Mobile Portal` to view parking sessions, payment history, and parking availability.
    - **Administrative Access:** Parking Management, Finance Office, System Admin, and IT Management access the `Admin Dashboard` to monitor parking operations, configure policies, review logs, and generate reports.
    - **Visitor Interaction:** Temporary visitors interact with the `Visitor Kiosk UI` at the parking gate to request a temporary card and perform entry-related actions.
2. **IoT Gate and Sensor Layer:**
    - **Physical Gate Control:** The `Entry/Exit Gate Devices` include ID card readers, ALPR cameras, barrier gates, and temporary card dispensers. These devices support automated vehicle entry and exit processing.
    - **Local IoT Gateway:** The `Local IoT Gateway` collects data from card readers, cameras, slot sensors, and gate devices. It forwards encrypted IoT events to the backend and receives commands such as opening or closing the barrier gate.
    - **Local Cache Mode:** The gateway maintains a `Local Cache` of valid user and vehicle data, allowing limited gate operation during temporary network or SSO service interruptions.
    - **Real-time Occupancy Monitoring:** Slot sensors and electronic signage provide live parking availability information for dashboards, kiosks, and user-facing interfaces.
3. **Application Server Cluster (Backend):**
    - **High Availability:** The inclusion of a `Load Balancer` and `API Gateway` supports scalable and reliable communication between client devices, IoT gateways, and backend services.
    - **Modular Service Design:** Backend logic is separated into cohesive services, each responsible for a specific business domain:
        - `Authentication&RBAC_Service`: handles authentication, authorization, and role-based access control.
        - `ParkingSession_Service`: manages vehicle entry, exit, and parking session lifecycle.
        - `Occupancy_Service`: updates parking slot availability and real-time occupancy status.
        - `Billing&Payment_Service`: calculates parking fees and coordinates payment processing.
        - `Dashboard&Report_Service`: provides monitoring dashboards and operational reports.
        - `Configuration_Service`: manages system parameters and IoT gateway configuration.
        - `Log&Audit_Service`: records activity logs, technical logs, and audit trails.
        - `DataSync_Scheduler`: synchronizes user, role, and vehicle data from external university services.
    - **Parking Operation Flow:** The `ParkingSession_Service` handles vehicle entry and exit sessions, validates parking privileges, updates parking status, and coordinates with the billing service when payment is required.
    - **Monitoring and Reporting:** The `Dashboard&Report_Service` retrieves parking, financial, and audit data to support operational dashboards, revenue reports, and administrative review.
    - **System Configuration:** The `Configuration_Service` manages technical parameters such as gate timeout, ALPR confidence threshold, and grace period, then deploys configuration updates to the IoT Gateway.

4. **Data Tier (Database Server Pool):**
   - **Decoupled Storage:** The database tier is divided into specialized databases, including `Parking_DB`, `Finance_DB`, `UserCache_DB`, `AuditLog_DB`, and `Config_DB`. This separation improves maintainability and enforces clear data ownership.
   - **Parking Data Persistence:** The `Parking_DB` stores active sessions, historical sessions, visitor card data, parking slot status, and occupancy records.
   - **Financial Data Management:** The `Finance_DB` stores transactions, refunds, debts, and pricing policies used by the billing and payment services.
   - **Auditability:** The `AuditLog_DB` maintains activity logs, technical logs, and administrative action logs, supporting incident investigation and operational transparency.
5. **External Systems Integration:**
   - **Authentication Integration:** The backend integrates with `HCMUT_SSO` to validate user sessions, authenticate institutional accounts, and enforce secure access control.
   - **Data Synchronization:** The `DataSync_Scheduler` periodically synchronizes user profiles, roles, vehicle information, and access privileges from `HCMUT_DATACORE` into the local `UserCache_DB`.
   - **Payment Processing:** The `Billing&Payment_Service` communicates with `BKPay` to process parking payments, verify payment confirmations, and support refund operations.
6. **Backup and Reliability Support:**
   - **Off-site Backup:** Core databases are backed up to `Off-site Backup Storage` to protect parking, financial, user cache, audit, and configuration data.
   - **Recovery Support:** The backup mechanism improves disaster recovery capability and supports restoration of critical system data in case of server failure.

9 Development View - Component Diagram

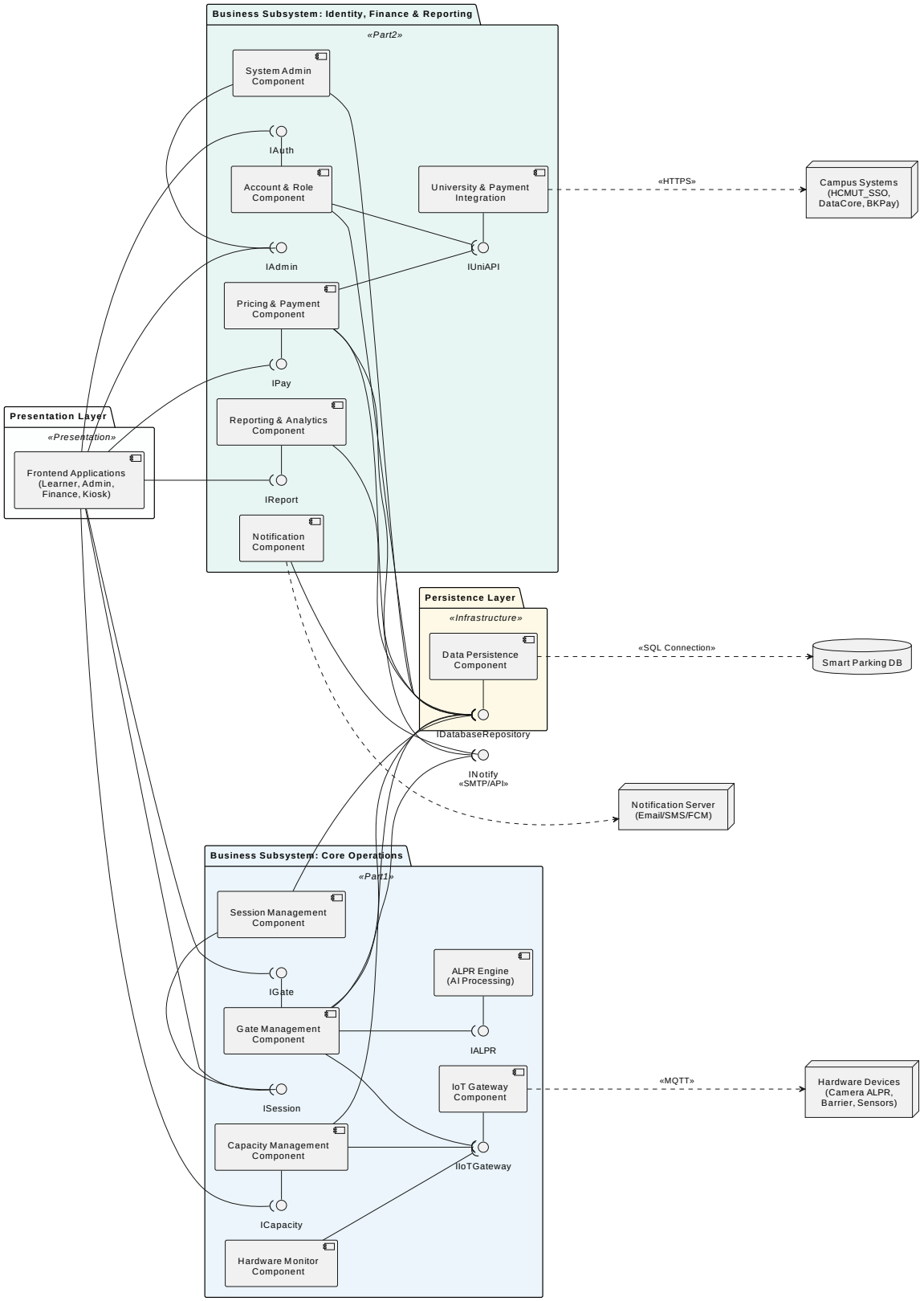

Figure 9.1: Component Diagram of Smart Parking System

9.1 Presentation Layer and Frontend Interaction

The **Presentation Layer** represents all user-facing entry points of the system through the **Frontend Applications** component, which supports learners, administrators, finance staff, and kiosk interfaces. This layer does not directly access the database, hardware devices, or external campus systems. Instead, it depends only on service interfaces exposed by backend components. :contentReference[oaicite:0]index=0

- **Provides:**
  - None.
- **Requires:**
  - `IAuth` for authentication and account-related operations.
  - `IAdmin` for administrative and system configuration functions.
  - `IPay` for pricing and payment-related operations.
  - `IReport` for reporting and analytics features.
  - `IGate` for gate control and access operations.
  - `ISession` for parking session management.
  - `ICapacity` for parking lot capacity and occupancy monitoring.

Each required interface from the frontend is connected to a corresponding backend component. This ensures that presentation logic remains separated from business processing and infrastructure concerns.

9.2 Identity, Finance & Reporting Subsystem

The **Identity, Finance & Reporting** subsystem groups components related to authentication, administration, pricing, reporting, notification, and external university/payment integration. It contains the **System Admin Component**, **Account & Role Component**, **Pricing & Payment Component**, **Reporting & Analytics Component**, **Notification Component**, and **University & Payment Integration**. :contentReference[oaicite:1]index=1

9.2.1 System Admin Component

The **System Admin Component** is responsible for higher-level administrative and system configuration functions.

- **Provides:**
  - `IAuth` for selected authentication and administrative control operations.
- **Requires:**
  - `IDatabaseRepository` to store and retrieve system-level configuration data and administrative records.

By isolating system-level configuration into a dedicated component, the architecture avoids mixing administrative responsibilities with operational parking workflows.

9.2.2 Account & Role Component

The **Account & Role Component** encapsulates authentication, identity handling, and authorization logic for users of the Smart Parking System. It supports login, account validation, and role-based access control, while delegating communication with institutional services to the integration layer. :contentReference[oaicite:2]index=2

- **Provides:**
  - `IAuth` for authentication and account-related operations used by the frontend.
- **Requires:**

  - `IUniAPI` from **University & Payment Integration** to communicate with HCMUT SSO and DataCore.
  - `IDatabaseRepository` to store account records, roles, and authorization-related data.

This design ensures that identity-related business logic remains independent from low-level external API communication and persistence details.

9.2.3 Pricing & Payment Component

The **Pricing & Payment Component** is responsible for parking fee calculation, pricing policy management, and payment-related business rules. It exposes payment functionality to the frontend while relying on the integration layer for communication with campus payment services. :contentReference[oaicite:3]index=3

- **Provides:**
  - `IPay` for pricing and payment-related operations.
- **Requires:**
  - `IUniAPI` from **University & Payment Integration** to access services such as BKPay.
  - `IDatabaseRepository` to store pricing rules, payment transactions, and policy history.

By separating pricing logic from integration details, the system becomes easier to extend when payment methods or pricing policies change.

9.2.4 Reporting & Analytics Component

The **Reporting & Analytics Component** aggregates operational and financial data in order to generate reports and analytical summaries for administrators and finance staff. It provides reporting services to the frontend while relying on the persistence layer as the main source of stored data. :contentReference[oaicite:4]index=4

- **Provides:**
  - `IReport` for reporting and analytics functions used by the frontend.
- **Requires:**
  - `IDatabaseRepository` as the single point to query stored parking, financial, and operational data for reporting purposes.

In this way, reporting logic remains encapsulated in its own component and avoids introducing separate or duplicated data-access paths.

9.2.5 Notification Component

The **Notification Component** manages outgoing notifications such as parking alerts, payment confirmations, and system notices. It isolates message-delivery concerns from the rest of the business logic. :contentReference[oaicite:5]index=5

- **Provides:**
  - `INotify` for notification dispatching and message delivery.
- **Requires:**
  - `IDatabaseRepository` to retrieve notification-related data and delivery records.
  - External **Notification Server** through SMTP/API for sending email, SMS, or push notifications.

By isolating notification logic into a dedicated component, the system can evolve its communication channels independently of core parking operations.

9.2.6 University & Payment Integration

The **University & Payment Integration** component acts as a façade and adapter to external campus systems. It centralizes communication with HCMUT SSO, DataCore, and BKPay, thereby shielding business components from protocol and endpoint details. :contentReference[oaicite:6]index=6

- **Provides:**
  - `IUniAPI` for authentication, university data access, and payment platform communication.
- **Requires:**
  - External **Campus Systems** (`HCMUT_SSO`, `DataCore`, `BKPay`) through HTTPS.

This design ensures that business components do not directly depend on low-level HTTP communication or external platform specifics.

9.3 Core Operations Subsystem

The **Core Operations** subsystem captures the main parking workflow and operational logic of the system. It consists of the **Session Management Component**, **Gate Management Component**, **Capacity Management Component**, **ALPR Engine**, **IoT Gateway Component**, and **Hardware Monitor Component**. Together, these components manage session lifecycle, gate control, lot occupancy, license plate recognition, hardware communication, and monitoring. :contentReference[oaicite:7]index=7

9.3.1 Session Management Component

The **Session Management Component** manages the lifecycle of parking sessions, including session creation, status tracking, and completion.

- **Provides:**
  - `ISession` for parking session operations used by the frontend and related backend components.
- **Requires:**
  - `IDatabaseRepository` to store session metadata, timestamps, vehicle information, and status history.

This component centralizes all parking session data and ensures that the system manages parking events consistently.

9.3.2 Gate Management Component

The **Gate Management Component** controls the business logic associated with entry and exit barriers. It determines gate actions based on recognized vehicles, authorization, and parking session validity. :contentReference[oaicite:8]index=8

- **Provides:**
  - `IGate` for gate control and access-handling operations.
- **Requires:**
  - `IALPR` from the **ALPR Engine** to obtain recognized license plate information.
  - `IIoTGateway` from the **IoT Gateway Component** to send control commands to physical barriers and devices.
  - `ISession` from the **Session Management Component** to validate or update parking sessions during entry and exit.

In this way, physical gate operations remain coordinated through explicit service interfaces instead of direct device access.

9.3.3 Capacity Management Component

The **Capacity Management Component** is responsible for tracking parking lot occupancy and available capacity. It supports occupancy monitoring and capacity-based decision making within the parking system. :contentReference[oaicite:9]index=9

- **Provides:**
  - `ICapacity` for capacity and occupancy-related operations.
- **Requires:**
  - `IDatabaseRepository` to store and retrieve lot occupancy and capacity data.
  - `IIoTGateway` to receive device-related occupancy signals when needed.

By isolating occupancy logic into a separate component, the architecture makes it easier to extend capacity policies independently from gate or session logic.

9.3.4 ALPR Engine

The **ALPR Engine** is the specialized component responsible for automatic license plate recognition using camera input. It encapsulates recognition logic in a dedicated processing component. :contentReference[oaicite:10]index=10

- **Provides:**
  - `IALPR` for license plate recognition results and related processing services.
- **Requires:**
  - Input streams or image data from parking cameras through the hardware and gateway infrastructure.

Separating ALPR into its own component allows future improvement or replacement of recognition algorithms without affecting the rest of the system.

9.3.5 IoT Gateway Component

The **IoT Gateway Component** acts as the bridge between software components and physical parking devices. It centralizes device communication and abstracts low-level messaging details from business logic. :contentReference[oaicite:11]index=11

- **Provides:**
  - `IIoTGateway` for communication with barriers, cameras, and sensors.
- **Requires:**
  - External **Hardware Devices** (camera ALPR, barriers, sensors) through MQTT.

This abstraction reduces coupling between operational business services and hardware communication protocols.

9.3.6 Hardware Monitor Component

The **Hardware Monitor Component** supervises the health, connectivity, and status of physical parking devices. It improves operational reliability by isolating monitoring concerns from control logic. :contentReference[oaicite:12]index=12

- **Provides:**
  - Device monitoring and hardware status observation services.
- **Requires:**
  - `IIoTGateway` to access hardware state and telemetry information.
  - `IDatabaseRepository` to persist monitoring logs or device status history when needed.

9.4 Persistence Layer

The **Persistence Layer** is represented by the **Data Persistence Component**, which acts as the centralized repository and data-access abstraction of the Smart Parking System. All major business components depend on this shared interface for reading and writing persistent data. :contentReference[oaicite:13]index=13

- **Provides:**
  - `IDatabaseRepository` for all persistent storage and retrieval operations.
- **Requires:**
  - External **Smart Parking DB** through SQL Connection.

This design hides database-specific details from business logic and allows the underlying storage technology to be maintained or scaled independently.

9.5 External Systems and Infrastructure Connections

The component diagram also makes the system's infrastructure and external integrations explicit. External services are not accessed directly by the business components; instead, they are routed through dedicated integration or gateway components. :contentReference[oaicite:14]index=14

- **Campus Systems** (`HCMUT_SSO`, `DataCore`, `BKPay`)
  - Connected through the **University & Payment Integration** component via HTTPS.
- **Smart Parking DB**
  - Connected through the **Data Persistence Component** via SQL Connection.
- **Notification Server** (`Email/SMS/FCM`)
  - Connected through the **Notification Component** via SMTP/API.
- **Hardware Devices** (`Camera ALPR, Barrier, Sensors`)
  - Connected through the **IoT Gateway Component** via MQTT.

These explicit external links show how the architecture cleanly separates business logic from communication protocols and infrastructure-level dependencies.

9.6 Architectural Discussion

Overall, the final component diagram makes several architectural decisions explicit. First, communication is interface-based, meaning that each dependency is shown through a required or provided interface. Second, responsibilities are separated into clear subsystems, including presentation, identity and finance support, core parking operations, and persistence. Third, external services and hardware are isolated behind dedicated adapters and gateway components. This results in a modular and maintainable architecture in which components can evolve independently while the system remains extensible and robust.

10 Class Diagram

10.1 Core Entity Overview

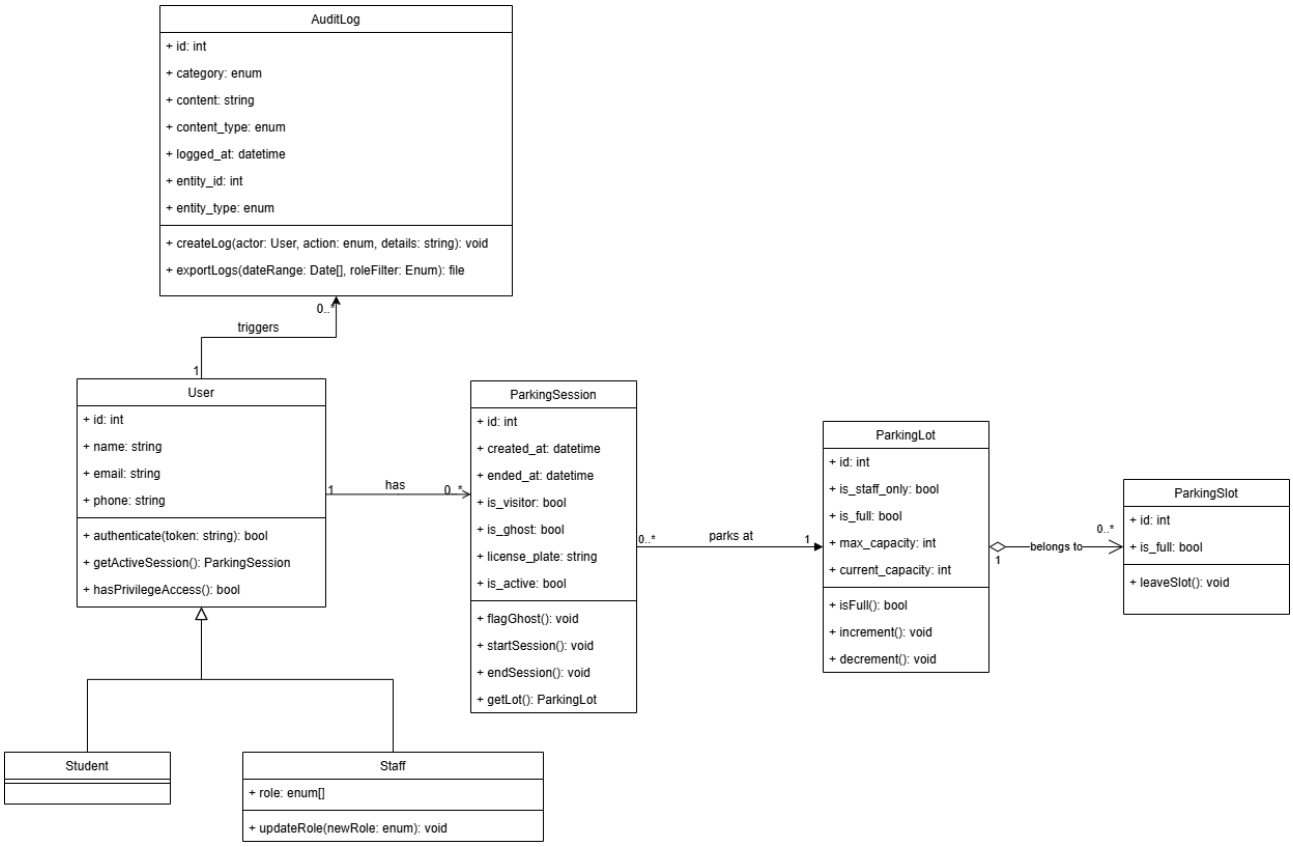

Figure 10.1: Class diagram for core entities.

User

The core entity representing an individual interacting with the system. It serves as a base class for specific user types.

- **Attributes:**
  - `id: int` - Unique identifier for the user.
  - `name: string` - The user's full name.
  - `email: string` - The user's contact email address.
  - `phone: string` - The user's contact phone number.
- **Methods:**
  - `authenticate(token: string): bool` - Validates the user's session or credentials using a token, returning `true` if successful.
  - `getActiveSession(): ParkingSession` - Retrieves the user's currently active parking session, if one exists.
  - `hasPrivilegeAccess(): bool` - Checks if the user has elevated system permissions.

Student

A specific type of `User` representing a student.
Note: This class inherits all attributes and methods from the `User` class but currently has no unique properties or methods defined in the diagram.

Staff

A specific type of `User` representing a staff member, likely with different parking privileges.

- **Attributes:**
  - `role: enum[]` - An array of specific roles assigned to the staff member (e.g., Admin, Faculty, Maintenance).
- **Methods:**
  - `updateRole(newRole: enum): void` - Updates or appends a new role to the staff member's profile.

ParkingSession

Tracks a single instance of a vehicle parking in a lot, from entry to exit.

- **Attributes:**
  - `id: int` - Unique identifier for the parking session.
  - `created_at: datetime` - Timestamp of when the vehicle entered and the session started.
  - `ended_at: datetime` - Timestamp of when the vehicle exited and the session closed.
  - `is_visitor: bool` - Flag indicating if the session belongs to an unregistered guest.
  - `is_ghost: bool` - Flag indicating a system anomaly (e.g., a car is in the lot but didn't scan in, or vice versa).
  - `license_plate: string` - The registered license plate of the vehicle.
  - `is_active: bool` - Indicates whether the session is currently ongoing.
- **Methods:**
  - `flagGhost(): void` - Marks the session as a "ghost" session for administrative review.
  - `startSession(): void` - Initializes the session upon vehicle entry.
  - `endSession(): void` - Finalizes the session upon vehicle exit.
  - `getLot(): ParkingLot` - Retrieves the details of the specific parking lot where this session is taking place.

ParkingLot

Represents a physical parking facility managed by the system.

- **Attributes:**
  - `id: int` - Unique identifier for the parking lot.
  - `is_staff_only: bool` - Flag restricting the lot to Staff users only.
  - `is_full: bool` - A quick-check boolean indicating if maximum capacity has been reached.
  - `max_capacity: int` - The total number of parking spaces available in the lot.
  - `current_capacity: int` - The current number of vehicles parked in the lot.
- **Methods:**
  - `isFull(): bool` - Evaluates current vs. max capacity and returns true if the lot is full.
  - `increment(): void` - Increases the `current_capacity` tally by one (called when a vehicle enters).
  - `decrement(): void` - Decreases the `current_capacity` tally by one (called when a vehicle exits).

ParkingSlot

Represents an individual, distinct parking space within a `ParkingLot`.

- **Attributes:**
  - `id: int` - Unique identifier for the specific parking space.
  - `is_full: bool` - Indicates whether a vehicle is currently occupying this exact slot.
- **Methods:**
  - `leaveSlot(): void` - Updates the slot's status to empty/available when a vehicle leaves.

AuditLog

Represents a system of record for tracking actions and events within the application.

- **Attributes:**
  - `id: int` - Unique identifier for the log entry.
  - `category: enum` - The broader classification of the log (e.g., security, operational, system).
  - `content: string` - A textual description of the logged event.
  - `content_type: enum` - The format or specific type of the content being logged.
  - `logged_at: datetime` - The exact timestamp when the event occurred.
  - `entity_id: int` - The ID of the specific entity (e.g., a User or Session ID) associated with the event.
  - `entity_type: enum` - The type of entity the log refers to.
- **Methods:**
  - `createLog(actor: User, action: enum, details: string): void` - Generates a new log entry triggered by a specific User (`actor`) performing an `action`.
  - `exportLogs(dateRange: Date[], roleFilter: Enum): file` - Compiles and exports a downloadable file of logs based on a specific timeframe and role criteria.

10.2 System management and Telemanagement for officer

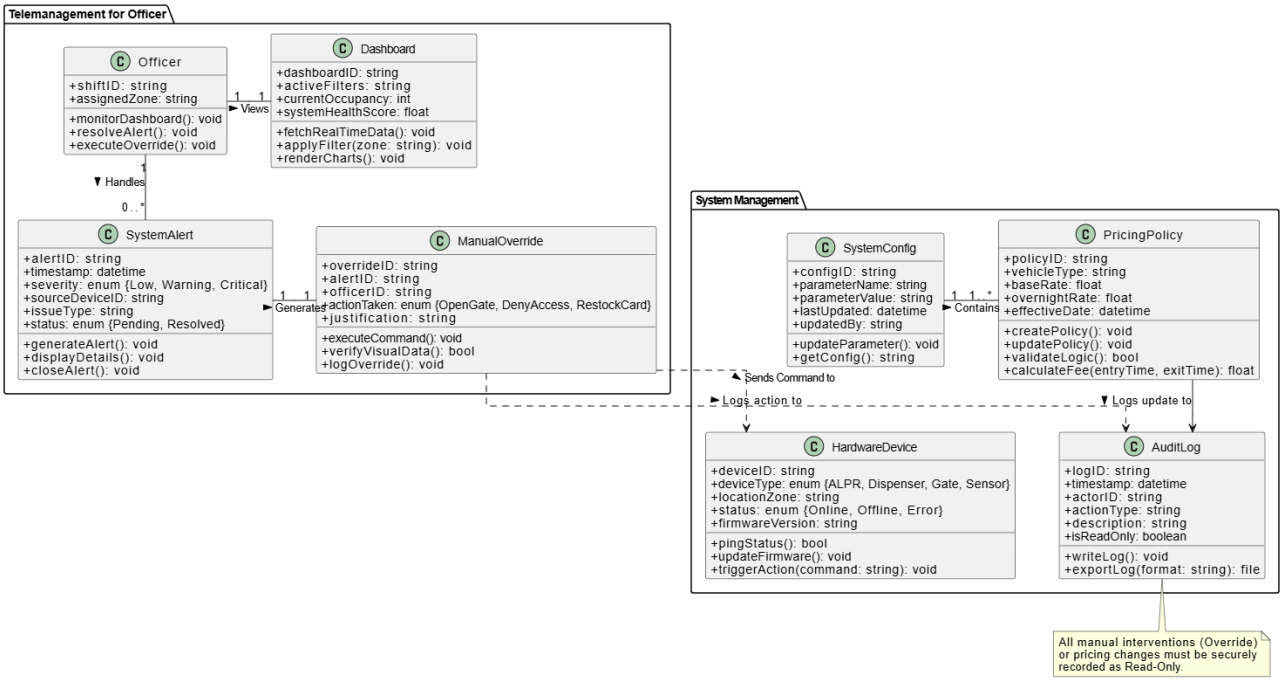

Figure 10.2: Class Diagram for System Management and Telemanagement Subsystems

The System Management and Telemanagement for Officer subsystems are crucial for operational monitoring, configuration, and exception handling. Below are the detailed specifications for the entities within these packages.

SystemConfig
Manages global system parameters and configurations to ensure smooth operation without requiring hard-coded changes.

- **Attributes:**
  - `configID: string`: Unique identifier for the configuration record.
  - `parameterName: string`: The name of the system parameter (e.g., "MaxRetryAttempts").
  - `parameterValue: string`: The assigned value for the parameter.
  - `lastUpdated: datetime`: Timestamp of the most recent modification.
  - `updatedBy: string`: The ID of the administrator who last updated the configuration.
- **Methods:**
  - `updateParameter()`: Modifies an existing system configuration value.
  - `getConfig()`: Retrieves the current value of a specific parameter.

PricingPolicy
Defines the financial rules and rates applied to parking sessions based on vehicle types and timeframes.

- **Attributes:**
  - `policyID: string`: Unique identifier for this pricing configuration.
  - `vehicleType: string`: The category of the vehicle (e.g., Car, Motorcycle).
  - `baseRate: float`: The standard parking fee for daytime or initial block hours.
  - `overnightRate: float`: The specific fee applied for overnight parking.
  - `effectiveDate: datetime`: The date and time when this policy becomes active.
- **Methods:**
  - `createPolicy()`: Initializes a new pricing rule.
  - `updatePolicy()`: Adjusts the rates of an existing policy.
  - `validateLogic()`: Ensures the pricing parameters are mathematically logical (e.g., rates cannot be negative).
  - `calculateFee(entryTime, exitTime)`: Computes the parking cost based on the policy rules.

HardwareDevice
Represents physical IoT equipment within the parking facility, facilitating hardware-software communication.

- **Attributes:**
  - `deviceID: string`: Unique identifier for the physical device.
  - `deviceType: enum`: Categorizes the hardware as ALPR, Dispenser, Gate, or Sensor.
  - `locationZone: string`: The physical area where the device is installed.
  - `status: enum`: Current operational state (Online, Offline, Error).
  - `firmwareVersion: string`: The current software version running on the device.
- **Methods:**
  - `pingStatus()`: Checks the connectivity and health of the device.
  - `updateFirmware()`: Deploys software updates to the physical hardware.

  - `triggerAction(command)`: Sends a direct command to the device (e.g., "OpenBarrier").

Officer
Represents a management staff member actively monitoring the system and responding to real-time events.

- **Attributes:**
  - `shiftID: string`: Identifier for the officer's current working shift.
  - `assignedZone: string`: The specific parking area the officer is monitoring.
- **Methods:**
  - `monitorDashboard()`: Accesses real-time data and visual statistics.
  - `resolveAlert()`: Takes ownership of an active system issue.
  - `executeOverride()`: Initiates a manual intervention process.

Dashboard
The User Interface component that provides aggregated real-time statistics and system health indicators.

- **Attributes:**
  - `dashboardID: string`: Unique session identifier for the active dashboard view.
  - `activeFilters: string`: Current display criteria applied by the officer (e.g., viewing only Zone A).
  - `currentOccupancy: int`: Real-time count of vehicles currently parked.
  - `systemHealthScore: float`: An aggregated metric indicating overall hardware and network stability.
- **Methods:**
  - `fetchRealTimeData()`: Pulls the latest operational metrics from the database.
  - `applyFilter(zone)`: Updates the view to focus on specific data segments.
  - `renderCharts()`: Generates visual graphs for occupancy and revenue.

SystemAlert
Represents an automated warning generated by the system when a hardware failure or data anomaly occurs.

- **Attributes:**
  - `alertID: string`: Unique identifier for the incident.
  - `timestamp: datetime`: The exact time the anomaly was detected.
  - `severity: enum`: Classifies the issue as Low, Warning, or Critical.
  - `sourceDeviceID: string`: The ID of the hardware that triggered the alert.
  - `issueType: string`: Description of the problem (e.g., "ALPR Read Failure").
  - `status: enum`: Tracks if the alert is Pending or Resolved.
- **Methods:**
  - `generateAlert()`: Creates a new alert record upon detecting an issue.
  - `displayDetails()`: Shows associated evidence (e.g., camera footage) to the officer.
  - `closeAlert()`: Marks the issue as resolved after intervention.

ManualOverride
Encapsulates the action taken by an Officer to bypass the automated system, ensuring accountability.

- **Attributes:**
  - `overrideID: string`: Unique identifier for this manual intervention record.
  - `alertID: string`: Links the override to the specific alert that necessitated it.
  - `officerID: string`: The ID of the staff member performing the override.
  - `actionTaken: enum`: The specific command executed (OpenGate, DenyAccess, RestockCard).
  - `justification: string`: Required text input explaining why the override was performed.
- **Methods:**
  - `executeCommand()`: Transmits the override signal to the hardware gateway.
  - `verifyVisualData()`: Cross-references database records with live camera feeds before action.
  - `logOverride()`: Securely triggers the AuditLog to record this event.

AuditLog
A strict security and compliance component that records sensitive actions such as manual overrides and configuration changes.

- **Attributes:**
  - `logID: string`: Unique identifier for the log entry.
  - `timestamp: datetime`: The exact time the event occurred.
  - `actorID: string`: The user or system process that initiated the event.
  - `actionType: string`: Categorization of the action (e.g., "Configuration Change", "Gate Override").
  - `description: string`: Detailed context of the event.
  - `isReadOnly: boolean`: An immutable flag ensuring the log cannot be altered or deleted.
- **Methods:**
  - `writeLog()`: Appends a new, immutable record to the system logs.
  - `exportLog(format)`: Generates a downloadable file (PDF/CSV) for management review.

10.3 Billing System

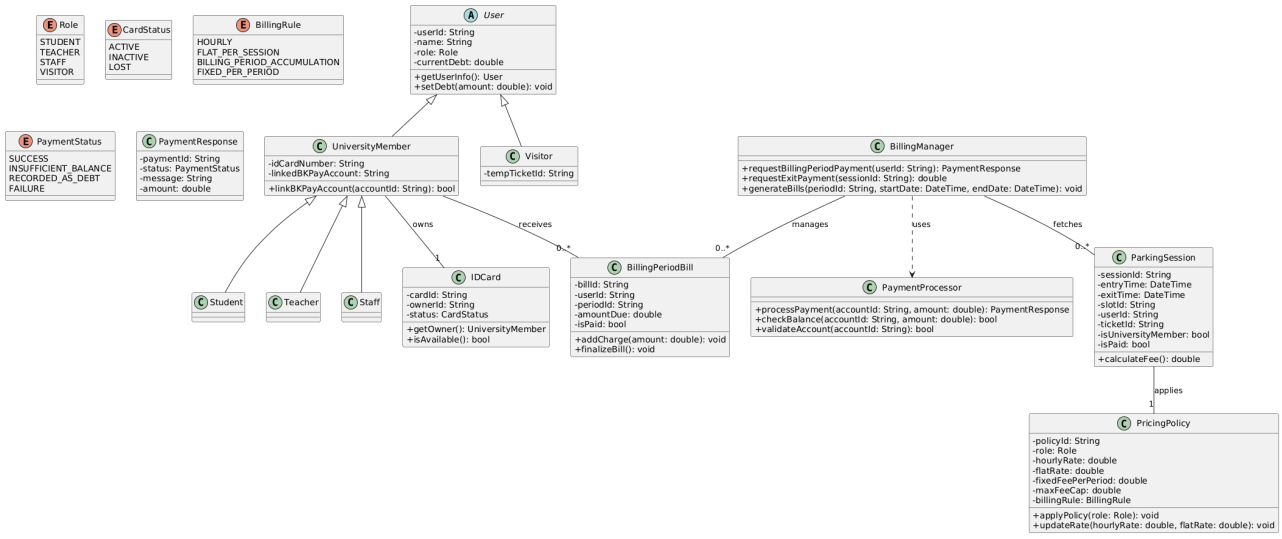

Figure 10.3: Class diagram for Billing system

User (Abstract)

The base class representing any individual interacting with the parking system.

- **Attributes:**
  - `userId: String`: A unique identifier assigned to the user.
  - `name: String`: The full name of the user.
  - `role: Role`: An enumeration representing the specific role of the user (STUDENT, TEACHER, STAFF, or VISITOR).
  - `currentDebt: double`: The outstanding financial balance the user owes to the parking system.
- **Methods:**
  - `getUserInfo(): User`: Retrieves the comprehensive profile and details of the current user.
  - `setDebt(amount: double): void`: Updates the user's current outstanding debt with the specified amount.

UniversityMember

Inherits from `User`. Represents individuals formally affiliated with the university (Students, Teachers, and Staff).

- **Attributes:**
  - `idCardNumber: String`: The unique alphanumeric code printed on the member's physical university ID card.
  - `linkedBKPayAccount: String`: The account identifier for the university's internal payment gateway (BKPay) linked to this member.
- **Methods:**
  - `linkBKPayAccount(accountId: String): bool`: Binds a BKPay digital wallet or account to the university member's profile for automated billing. Returns true if successful.

Student, Teacher, Staff

Inherit from `UniversityMember`. These are concrete implementations for specific university roles.

- **Attributes & Methods:**
  - Inherit all attributes and methods from `UniversityMember`. Currently acts as structural subclasses to allow role-specific logic or policy application in the future.

Visitor

Inherits from `User`. Represents external guests who park on campus without a formal university affiliation.

- **Attributes:**
  - `tempTicketId: String`: The identifier of the temporary paper or digital ticket issued to the visitor upon entry.

IDCard

Represents the physical or digital identification card owned by a `UniversityMember`, used for accessing the parking lot.

- **Attributes:**
  - `cardId: String`: Internal system identifier for the card record.
  - `ownerId: String`: The `userId` of the `UniversityMember` who owns this card.

  - `status: CardStatus`: The current state of the card (ACTIVE, INACTIVE, LOST).

- **Methods:**
  - `getOwner(): UniversityMember`: Retrieves the university member object associated with this ID card.
  - `isAvailable(): bool`: Checks if the card is currently active and permitted to open the parking barrier.

PaymentResponse

A data transfer object (DTO) that encapsulates the result of a financial transaction.

- **Attributes:**
  - `paymentId: String`: A unique transaction receipt or tracking number.
  - `status: PaymentStatus`: The outcome of the transaction attempt (SUCCESS, INSUFFICIENT_BALANCE, RECORDED_AS_DEBT, FAILURE).
  - `message: String`: A human-readable description of the payment result.
  - `amount: double`: The exact financial amount processed or attempted.

PaymentProcessor

A centralized component responsible for interfacing with external gateways and processing financial transactions.

- **Methods:**
  - `processPayment(accountId: String, amount: double): PaymentResponse`: Executes a debit operation against the specified account for the given amount, returning the transaction result.
  - `checkBalance(accountId: String, amount: double): bool`: Verifies if the specified account contains sufficient funds to cover the given amount.
  - `validateAccount(accountId: String): bool`: Checks if the provided payment account identifier is valid, active, and capable of transactions.

BillingManager

The core controller for orchestrating billing cycles, computing user debts, and issuing payment requests.

- **Methods:**
  - `requestBillingPeriodPayment(userId: String): PaymentResponse`: Triggers the payment collection process for a user's accumulated periodic bill.
  - `requestExitPayment(sessionId: String): double`: Calculates and processes the immediate payment required for a user exiting the parking lot (typically for visitors or per-session billing).
  - `generateBills(periodId: String, startDate: DateTime, endDate: DateTime): void`: Scans all unpaid parking sessions within the specified date range and generates `BillingPeriodBill` instances for applicable university members.

BillingPeriodBill

Represents a consolidated invoice generated for a user over a specific billing cycle (e.g., a month).

- **Attributes:**
  - `billId: String`: Unique identifier for the invoice.
  - `userId: String`: The user responsible for paying the bill.
  - `periodId: String`: Identifier for the specific time period this bill covers.
  - `amountDue: double`: The total accumulated fee required to settle the bill.

  - `isPaid: bool`: Flag indicating whether this bill has been successfully settled.

- **Methods:**
  - `addCharge(amount: double): void`: Increments the `amountDue` by the specified charge (e.g., appending a newly completed parking session to the active bill).
  - `finalizeBill(): void`: Locks the bill from further modifications, preparing it for dispatch and payment processing.

ParkingSession

Tracks the lifecycle of a single vehicle's stay in the parking facility from entry to exit.

- **Attributes:**
  - `sessionId: String`: Unique tracking identifier for this parking instance.
  - `entryTime: DateTime`: The exact timestamp when the barrier opened for the vehicle.
  - `exitTime: DateTime`: The exact timestamp when the vehicle departed (null if still parked).
  - `slotId: String`: The physical parking space or zone occupied by the vehicle.
  - `userId: String`: Identifier of the user to whom this session belongs.
  - `ticketId: String`: The temporary ticket number (applicable mostly to visitors).
  - `isUniversityMember: bool`: Quick reference flag to determine the rule set to apply.
  - `isPaid: bool`: Flag indicating if this specific session has been paid for (either immediately upon exit or via a compiled bill).
- **Methods:**
  - `calculateFee(): double`: Computes the cost of the session based on the duration (entry to exit) and the applicable `PricingPolicy`.

PricingPolicy

Defines the financial rules and rates applied to parking sessions based on user roles and time.

- **Attributes:**
  - `policyId: String`: Unique identifier for this configuration of pricing rules.
  - `role: Role`: The specific user category this policy targets.
  - `hourlyRate: double`: The cost per hour of parking.
  - `flatRate: double`: A static fee charged per parking session regardless of duration.
  - `fixedFeePerPeriod: double`: A bulk subscription fee (e.g., monthly parking pass).
  - `maxFeeCap: double`: The maximum allowable charge per day or session to protect users from exorbitant fees.
  - `billingRule: BillingRule`: Determines how the policy is applied (HOURLY, FLAT_PER_SESSION, BILLING_PERIOD_ACCUMULATION, FIXED_PER_PERIOD).
- **Methods:**
  - `applyPolicy(role: Role): void`: Assigns and configures the active pricing tier for a specific user role.
  - `updateRate(hourlyRate: double, flatRate: double): void`: Administrative method to adjust the cost parameters of the policy.

10.4 Access and Gate control

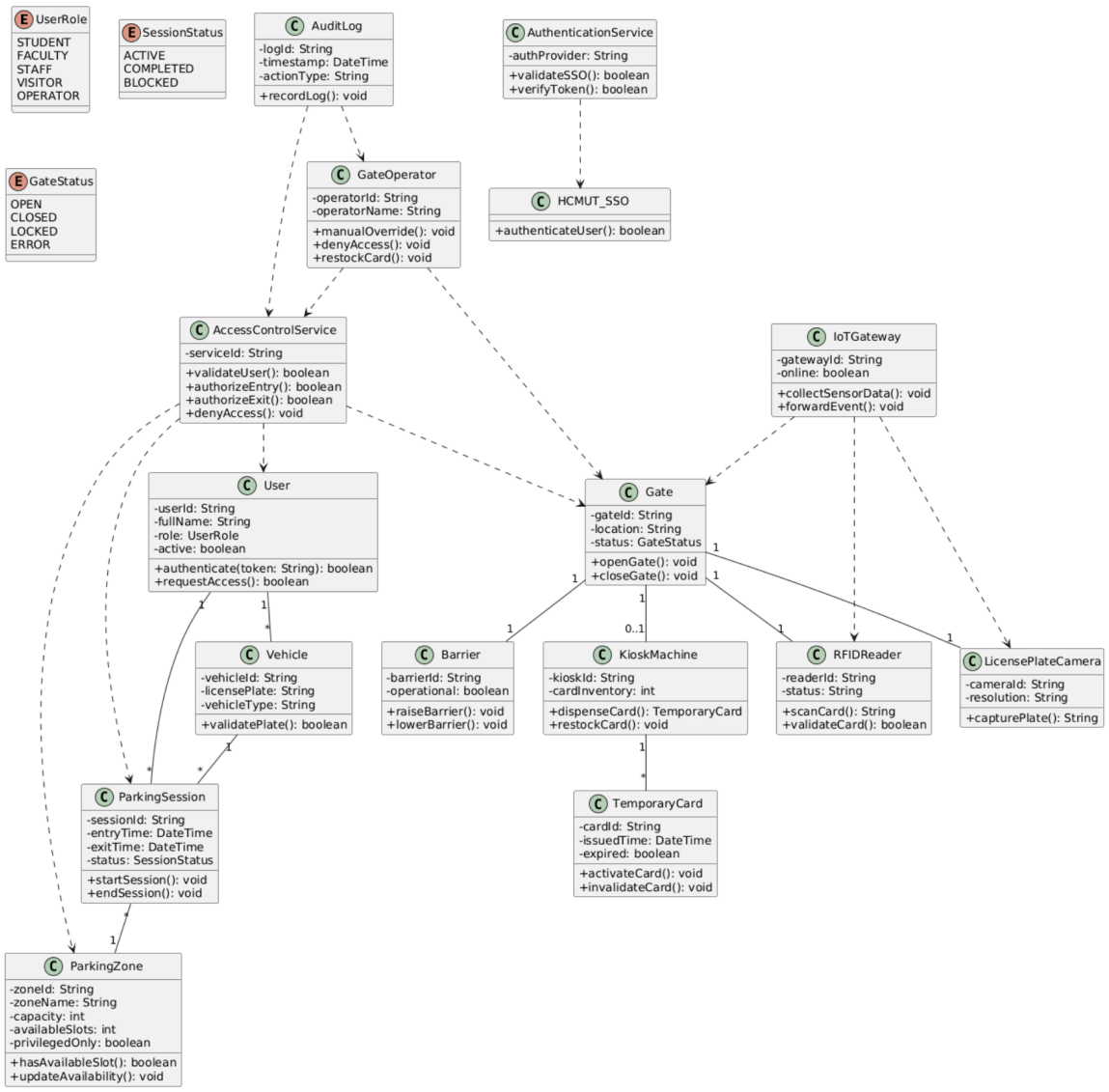

Figure 10.4: Class diagram for access and gate control.

User

The core entity representing users interacting with the parking access system, including students, faculty members, staff, visitors, and operators.

- **Attributes:**
  - `userId: String` - Unique user identifier.
  - `fullName: String` - Full name of the user.
  - `role: UserRole` - User role within the system.
  - `active: boolean` - Account status.
- **Methods:**
  - `authenticate(token: String): boolean` - Validates authentication token.
  - `requestAccess(): boolean` - Requests parking access.

Vehicle

Represents vehicles registered in the parking system.

- **Attributes:**
  - `vehicleId: String` - Vehicle identifier.
  - `licensePlate: String` - Vehicle plate number.
  - `vehicleType: String` - Vehicle category.
- **Methods:**
  - `validatePlate(): boolean` - Verifies plate validity.

ParkingSession

Represents an active or completed parking access session.

- **Attributes:**
  - `sessionId: String` - Parking session identifier.
  - `entryTime: DateTime` - Entry timestamp.
  - `exitTime: DateTime` - Exit timestamp.
  - `status: SessionStatus` - Current session state.
- **Methods:**
  - `startSession(): void` - Starts a parking session.
  - `endSession(): void` - Ends a parking session.

ParkingZone

Represents parking areas inside the campus.

- **Attributes:**
  - `zoneId: String` - Parking zone identifier.
  - `zoneName: String` - Parking zone name.
  - `capacity: int` - Maximum number of vehicles.
  - `availableSlots: int` - Current available slots.
  - `privilegedOnly: boolean` - Indicates whether access is restricted.
- **Methods:**
  - `hasAvailableSlot(): boolean` - Checks slot availability.
  - `updateAvailability(): void` - Updates available slot count.

AccessControlService

Handles authorization and validation for gate access operations.

- **Attributes:**
  - `serviceId: String` - Service identifier.
- **Methods:**
  - `validateUser(): boolean` - Verifies user identity.
  - `authorizeEntry(): boolean` - Grants entry permission.
  - `authorizeExit(): boolean` - Grants exit permission.
  - `denyAccess(): void` - Denies gate access.

AuthenticationService

Handles authentication through HCMUT_SSO.

- **Attributes:**
  - `authProvider: String` - Authentication provider name.
- **Methods:**
  - `validateSSO(): boolean` - Validates SSO authentication.
  - `verifyToken(): boolean` - Verifies access token.

GateOperator

Represents parking management personnel handling exceptional cases.

- **Attributes:**
  - `operatorId: String` - Operator identifier.
  - `operatorName: String` - Operator name.
- **Methods:**
  - `manualOverride(): void` - Opens gate manually.
  - `denyAccess(): void` - Denies suspicious access.
  - `restockCard(): void` - Refills temporary cards.

Gate

Represents a physical parking entry or exit gate.

- **Attributes:**
  - `gateId: String` - Gate identifier.
  - `location: String` - Physical location.
  - `status: GateStatus` - Current gate state.
- **Methods:**
  - `openGate(): void` - Opens the gate.
  - `closeGate(): void` - Closes the gate.

Barrier

Represents the physical gate barrier mechanism.

- **Attributes:**
  - `barrierId: String` - Barrier identifier.
  - `operational: boolean` - Barrier operational state.
- **Methods:**
  - `raiseBarrier(): void` - Raises the barrier.
  - `lowerBarrier(): void` - Lowers the barrier.

RFIDReader

Handles ID card scanning operations.

- **Attributes:**
  - `readerId: String` - RFID reader identifier.
  - `status: String` - Reader operational state.
- **Methods:**
  - `scanCard(): String` - Reads card information.
  - `validateCard(): boolean` - Validates scanned card.

LicensePlateCamera

Captures vehicle license plate information.

- **Attributes:**
  - `cameraId: String` - Camera identifier.
  - `resolution: String` - Camera resolution.
- **Methods:**
  - `capturePlate(): String` - Captures license plate number.

TemporaryCard

Represents temporary visitor access cards.

- **Attributes:**
  - `cardId: String` - Temporary card identifier.
  - `issuedTime: DateTime` - Card issuance timestamp.
  - `expired: boolean` - Expiration status.
- **Methods:**
  - `activateCard(): void` - Activates card access.
  - `invalidateCard(): void` - Disables card usage.

KioskMachine

Handles temporary card dispensing for visitors.

- **Attributes:**
  - `kioskId: String` - Kiosk identifier.
  - `cardInventory: int` - Remaining card quantity.
- **Methods:**
  - `dispenseCard(): TemporaryCard` - Issues temporary card.
  - `restockCard(): void` - Refills kiosk inventory.

IoTGateway

Represents the IoT gateway managing gate device communication.

- **Attributes:**
  - `gatewayId: String` - Gateway identifier.
  - `online: boolean` - Connection state.
- **Methods:**
  - `collectSensorData(): void` - Collects gate device data.
  - `forwardEvent(): void` - Sends events to the central system.

AuditLog

Stores operational and security-related activity logs.

- **Attributes:**
  - `logId: String` - Log identifier.
  - `timestamp: DateTime` - Event timestamp.
  - `actionType: String` - Action category.
- **Methods:**
  - `recordLog(): void` - Records activity log.

11 Testcases

The following test cases are derived from the functional requirements specified in the use-case diagrams (UC-01 through UC-13). Each test case is identified by a unique ID of the form **T*xx*-*nn***, where *xx* denotes the related use case group and *nn* is the sequential test number within that group. Priority levels follow the convention: **P0** (Critical), **P1** (High), **P2** (Medium), **P3** (Low).

11.1 Entry & Exit Gate (UC-01, UC-02)

Field	**T01-01**	**T01-02**	**T01-03**	**T01-04**	**T01-05**
Title	Valid student entry (general lot)	Staff entry to privileged lot	Student denied access to privileged lot	Entry denied – lot full	Valid ID card exit
Priority	P0	P0	P0	P1	P0
Preconditions	General lot has available slots; system online	Privileged lot has available slots; system online	System online; user is a student	General lot at 100% capacity	User has an active parking session
Test Data	card_id= STU001; role= Student	card_id= FAC001; role= Staff	card_id= STU002; role= Student; target= Privileged	General lot capacity= 0	card_id= STU001; session_id active
Steps	Swipe valid student card at entry gate	Swipe valid staff card at privileged entry gate	Student swipes at privileged lot entry gate	Any user swipes at general entry gate when full	Swipe valid card at exit gate
Expected Result	Session created; gate opens; slot count decremented; gate closes after vehicle passes	Session created; gate opens; privileged slot count decremented	"Access Denied – Restricted Area" displayed; gate remains closed; session not created	"Parking Area Full" notification displayed; gate remains closed	Session ended with exit timestamp; gate opens; slot count incremented; gate closes
Type	Functional/ Positive	Functional/ Positive	Negative/ Authorization	Negative/ Exception	Functional/ Positive

Table 15: Entry & Exit Gate Test Cases

11.2 Payment Processing (UC-03)

Field	T03-01	T03-02	T03-03
Title	Successful periodic billing via BKPay	Payment with insufficient balance	External cash payment at exit
Priority	P0	P1	P1
Preconditions	User has valid BKPay account; billing period ends	User BKPay balance is below total fee	Visitor has completed parking session
Test Data	user_id= STU001; fee= 15,000 VND; balance= 50,000 VND	user_id= STU002; fee= 20,000 VND; balance= 5,000 VND	visitor_card= V0042; cash= 10,000 VND
Steps	System triggers billing at end of period; user confirms payment	System attempts to deduct fee from BKPay account	Visitor inserts cash at exit kiosk cash register
Expected Result	Payment deducted from BKPay; transaction record created with status "Paid"; receipt issued	System notifies user of insufficient balance; debt record created; exit gate opens	Cash payment validated; session closed; receipt printed; gate opens
Type	Functional/ Positive	Negative/ Exception	Functional/ Alternative

Table 16: Payment Processing Test Cases

11.3 Temporary Card Issuance (UC-04)

Field	T04-01	T04-02	T04-03
Title	Visitor card issued successfully	Dispenser empty – management alerted	Visitor entry blocked – lot full
Priority	P0	P1	P0
Preconditions	Kiosk stocked; ALPR sensor online; general lot has slots	Kiosk dispenser has 0 cards remaining	General lot at 100% capacity; visitor arrives
Test Data	plate= 51A-12345; no match in DB	dispenser_count= 0	lot_capacity= 0
Steps	Unregistered vehicle triggers gate sensor; system captures license plate	Visitor vehicle triggers sensor; system attempts to dispense card	Visitor vehicle triggers sensor; system checks lot capacity
Expected Result	Temporary visitor profile created; card dispensed; entry timestamp written; gate opens after card swipe	System notifies Parking Management of empty dispenser; no card dispensed; gate remains closed	"Parking Lot Full" notification displayed; no card dispensed; gate remains closed
Type	Functional/ Positive	Resilience/ Negative	Negative/ Exception

Table 17: Temporary Card Issuance Test Cases

11.4 Parking Dashboard (UC-05)

Field	T05-01	T05-02	T05-03
Title	Dashboard loads real-time occupancy	Filter dashboard by parking zone	Dashboard shows hardware health status
Priority	P0	P1	P0
Preconditions	Manager logged in; IoT gateways active and reporting	Dashboard fully loaded; multiple zones configured	At least one IoT gateway is offline
Test Data	manager_id= PM001	zone= "Faculty Lot A"	gateway_id= GW-03; status= offline
Steps	Manager navigates to parking management URL; system renders dashboard	Manager selects zone "Faculty Lot A" from dropdown	Manager views the hardware health panel on dashboard
Expected Result	Dashboard displays real-time occupancy counts, active sessions, and slot availability per lot	Dashboard filters to show only Faculty Lot A occupancy and hardware status; other zones hidden	Offline gateway GW-03 shown in red/error state; online gateways shown as healthy
Type	Functional/ Positive	Functional/ Filter	Integration/ Status

Table 18: Parking Dashboard Test Cases

11.5 Exception Handling & Manual Override (UC-06)

Field	T06-01	T06-02	T06-03
Title	Manual override – open gate for valid staff	Manual override – deny access for suspected fraud	Dispenser empty restocking workflow
Priority	P0	P0	P1
Preconditions	Mismatch_Plate_Error alert received; live camera feed available	Alert received; plate and ID do not match database record	Dispenser_Empty alert received by Parking Management
Test Data	alert_id= EX-001; vehicle= 51B-99901; staff confirmed via intercom	alert_id= EX-002; plate mismatched; no valid identity confirmed	alert_id= EX-003; kiosk_id= K-02
Steps	Officer opens Event Detail; views camera footage; selects "Manual Override: Open Gate"; enters reason	Officer opens Event Detail; views footage; confirms mismatch; selects "Deny Access"; enters reason	Officer acknowledges alert; physically restocks dispenser at kiosk K-02; confirms restocking in system
Expected Result	Gate opened; audit log records officer ID, action "Open Gate", reason, and timestamp	Gate remains closed; audit log records officer ID, action "Deny Access", reason, and timestamp	Dispenser count updated; restocking event logged with officer ID and timestamp; kiosk resumes normal operation
Type	Functional/ Override	Negative/ Security	Functional/ Maintenance

Table 19: Exception Handling Test Cases

11.6 Pricing Policy Configuration (UC-07)

Field	T07-01	T07-02	T07-03
Title	Finance officer sets valid pricing policy	Submission rejected for negative fee value	Policy activates globally after save
Priority	P0	P0	P0
Preconditions	Finance Officer authenticated in Finance Module	Finance Officer authenticated	New policy just saved successfully
Test Data	base_fee= 5,000 VND; hourly_rate= 3,000 VND; grace= 15min	hourly_rate= -1,000 VND	next parking event after policy save
Steps	Officer enters valid fee values and submits; system validates and saves	Officer enters negative hourly rate and attempts to submit	A student parks after the policy is saved; system computes fee
Expected Result	Policy saved and activated globally; confirmation message displayed	System displays validation error "Fee values must be non-negative"; policy not saved	Fee computed using the newly activated pricing rules; old pricing no longer applied
Type	Functional/ Positive	Negative/ Validation	Integration/ Activation

Table 20: Pricing Policy Configuration Test Cases

11.7 Refund Processing (UC-08)

Field	T08-01	T08-02	T08-03
Title	Successful refund via BKPay	Refund rejected – already refunded	Refund for internal balance payment
Priority	P0	P0	P1
Preconditions	Finance Officer authenticated; transaction status is "Paid"; payment via BKPay	Finance Officer authenticated; transaction status is already "Refunded"	Finance Officer authenticated; transaction paid via internal card balance
Test Data	txn_id= TXN-2201; amount= 10,000 VND; method= BKPay	txn_id= TXN-2100; status= Refunded	txn_id= TXN-2305; amount= 8,000 VND; method= Internal
Steps	Officer searches TXN-2201; selects "Initiate Refund"; enters reason; confirms	Officer searches TXN-2100; selects "Initiate Refund"	Officer searches TXN-2305; selects "Initiate Refund"; enters reason; confirms
Expected Result	Refund request sent to BKPay; transaction status updated to "Refunded"; audit log entry created with actor ID and timestamp	System rejects request with message "Transaction already refunded"; no status change	Internal card balance restored by 8,000 VND; transaction status updated to "Refunded"; audit log entry created
Type	Functional/ Positive	Negative/ Validation	Functional/ Alternative

Table 21: Refund Processing Test Cases

11.8 User Role Management (UC-09)

Field	T09-01	T09-02	T09-03
Title	Assign role to new staff member	Assign role fails – user not found	Role conflict warning on assignment
Priority	P0	P1	P1
Preconditions	System Admin authenticated with Super Admin privileges; user account exists	System Admin authenticated; employee ID does not exist in system	System Admin authenticated; target user already holds a conflicting role
Test Data	employee_id= EMP-501; new_role= Finance Officer	employee_id= EMP-999 (non-existent)	employee_id= EMP-305; current_role= System Admin; new_role= Finance Officer
Steps	Admin searches EMP-501; selects "Assign Role"; chooses Finance Officer; confirms	Admin searches EMP-999 and attempts to proceed	Admin searches EMP-305; selects "Assign Role"; chooses Finance Officer
Expected Result	Role assigned; access control matrix updated; audit log records old role, new role, actor ID, and timestamp	System displays "User not found" error; no role change occurs	System displays role conflict warning; admin can choose to proceed or cancel
Type	Functional/ Positive	Negative/ Validation	Functional/ Warning

Table 22: User Role Management Test Cases

11.9 External API & IoT Data Integration (UC-10)

Field	T10-01	T10-02	T10-03
Title	SSO token validated on card swipe	SSO authentication failure – request rejected	Nightly DATACORE batch sync updates user privileges
Priority	P0	P0	P1
Preconditions	HCMUT_SSO endpoint active; valid API token	HCMUT_SSO temporarily unavailable or token expired	HCMUT_DATACORE available; scheduled sync time reached
Test Data	token= valid_jwt; card_id= STU-001	token= expired_jwt	sync_schedule= 02:00 AM daily
Steps	IoT gateway sends encrypted JSON packet on card swipe; system validates token with SSO	IoT gateway sends packet with expired token; system attempts SSO validation	System clock triggers nightly batch sync job at 02:00 AM
Expected Result	Token verified; user profile retrieved from local DB; appropriate entry/exit action executed; ACK sent to gateway	System rejects packet; returns authentication error response; event logged; no entry action taken	User profiles updated from DATACORE; graduated student access revoked; new faculty plates added; sync completion logged
Type	Integration/ Positive	Negative/ Security	Integration/ Scheduler

Table 23: External API & IoT Integration Test Cases

11.10 System Parameter Configuration (UC-11)

Field	T11-01	T11-02	T11-03
Title	IT Management updates gate timeout value	Invalid ALPR threshold value rejected	Config push fails for unreachable IoT gateway
Priority	P0	P1	P1
Preconditions	IT Management authenticated; IoT gateways all online	IT Management authenticated	IT Management authenticated; gateway GW-05 offline
Test Data	gate_timeout= 10s; grace_period= 15min; alpr_confidence= 85%	alpr_confidence= 150% (out of range)	config= valid; target_gateways= [GW-01, GW-02, GW-05]
Steps	IT Management opens System Settings; enters valid parameters; submits	IT Management enters ALPR confidence of 150% and submits	IT Management submits valid config; system pushes to all gateways
Expected Result	Configuration saved; pushed to all IoT gateways; system confirms successful deployment with gateway acknowledgement list	System displays validation error “Value out of accepted range”; configuration not saved	GW-01 and GW-02 acknowledge; GW-05 fails; system logs failure and alerts IT Management to retry for GW-05
Type	Functional/ Positive	Negative/ Validation	Resilience/ Partial Failure

Table 24: System Parameter Configuration Test Cases

11.11 Operational Log Monitoring (UC-12)

Field	T12-01	T12-02	T12-03
Title	Finance officer views payment logs only	Log export to CSV	Access denied for out-of-role log view
Priority	P0	P1	P0
Preconditions	Finance Officer authenticated; payment records exist in DB	Parking Manager authenticated; entry/exit logs exist	Student account authenticated; attempts to access admin logs
Test Data	actor_role= Finance Officer; date_filter= 2025-05-01	actor_role= Parking Manager; export_format= CSV	actor_role= Student; target= Admin Log
Steps	Finance Officer accesses log monitoring section; system filters by role	Parking Manager applies date filter; selects "Export" option; chooses CSV	Student navigates to admin log URL directly
Expected Result	Only Payment and Refund logs displayed; Entry/Exit and System logs not visible	CSV file generated and downloaded; all columns and row counts match on-screen data	System returns "Access Denied" message; no log data exposed; access violation logged
Type	Authorization/ Filtering	Functional/ Export	Negative/ Security

Table 25: Operational Log Monitoring Test Cases

11.12 Personal Parking History (UC-13)

Field	T13-01	T13-02	T13-03
Title	User views parking and payment history	No parking history available	SSO session expired – re-login prompted
Priority	P0	P1	P0
Preconditions	User logged in via HCMUT_SSO; at least two prior parking sessions exist	New user account; no prior parking sessions	User session token expired
Test Data	user_id= STU-003; sessions= 3	user_id= STU-999; sessions= 0	token= expired_session
Steps	User navigates to "History" section on self-service portal	New user navigates to "History" section	User attempts to load History page with expired session
Expected Result	Records displayed including Date, Duration, Fee, and Payment Method; user can tap a session to view digital receipt	System displays "No parking records found" message; no error thrown	System detects expired token; redirects user to SSO login page with message; no history data exposed
Type	Functional/ Positive	Functional/ Edge Case	Security/ Session

Table 26: Personal Parking History Test Cases

12 Public Source Code and Demo Website

To ensure transparency, reproducibility, and future extensibility of the system, the complete implementation of the Parking System has been published online.

- GitHub repository (source code): https://github.com/TienHwng/SPS-Website-Demo
- Public demo website: https://tienhwng.github.io/SPS-Website-Demo/

The GitHub repository hosts the full project source code and configuration files used during development and deployment. The public website provides a live demo of the system's main functionalities, allowing users, administrators, and evaluators to directly experience the system from relevant usage perspectives.

These resources are intended to support future maintenance, feature extension, and reuse of the system in similar smart parking or parking support scenarios.

13 Tools Used and Scope of Generative AI Contribution

13.1 Tools Used

During the development of the Parking System, Generative AI technologies were employed strictly as auxiliary tools to enhance productivity and streamline the workflow. All core system logic, architectural decisions, and final validations were performed entirely by team members.

- ChatGPT-5 / Claude: Used as conversational assistants to support brainstorming, refine conceptual approaches, review architectural designs, and improve clarity and academic tone of technical documentation.
- GitHub Copilot: Utilized to accelerate coding through intelligent autocompletion and boilerplate generation, reducing repetitive manual programming tasks.

13.2 Scope of Application and Level of Contribution

The application of Generative AI tools was carefully limited to drafting, revising, and suggesting improvements. All functional correctness, decision-making, and validation were performed manually by the team. The detailed scope of AI usage is summarized as follows.

13.2.1 Requirements Analysis

- Initial use case descriptions were drafted manually.
- AI tools assisted in refining phrasing of functional requirements for clarity and identifying missing edge cases.
- All AI-suggested edge cases were manually reviewed and validated against the original requirements before inclusion.

13.2.2 Diagram Design

- Core interaction flows and structural logic were determined manually.
- AI assistance was limited to improving naming conventions, enhancing consistency, and suggesting suitable design patterns.

13.2.3 Architecture Design

- AI tools were used to summarize the advantages and disadvantages of architectural alternatives.
- Final architectural decisions were made exclusively by the development team.

13.2.4 Implementation Phase

- AI tools were employed to generate boilerplate code such as entity definitions, standard CRUD structures, and skeleton unit tests.
- AI provided suggestions for syntax corrections and assisted in debugging non-functional errors.
- All business logic, data processing flows, and integration logic were written and validated manually.

Overall, Generative AI served solely as a supportive mechanism to enhance efficiency, while the intellectual and technical ownership of the system remained entirely with the team members.